National University of Science and Technology Politehnica Bucharest

Faculty of Computer Science
Master's programme in Artificial Intelligence

Benchmarking Visual SLAM Components on a Fixed Budget

A controlled study of feature extraction, loop closure, backend and depth across four hardware conditions

Master's Thesis

Petru Costin Zincenco

Scientific coordinator

Dan-Marius Novischi

Bucharest, 2026

Abstract

Visual SLAM is the perception backbone of autonomous vehicles and edge robots, yet most algorithm comparisons report accuracy on capable workstations and say little about whether a method deploys at all on the hardware these systems carry. I ask a different question: how much computation is required for each component to function, and how is its accuracy affected when the hardware components change?

I hold one SLAM system fixed, an ORB-SLAM2-style pipeline hosted by pySLAM, and swap one interchangeable block at a time: feature extraction, loop closure, and the optimization back end, with monocular neural depth tested separately as an alternative to the stereo rig. The resulting 46 configurations run on four KITTI-360 driving sequences across four hardware conditions that differ by exactly one factor each, for 1,683 full-SLAM and 236 visual-odometry runs. Every run is scored for accuracy on the two standard trajectory metrics, ATE and RPE, alongside CPU, RAM and GPU load, and is labeled based on how it finished the sequence.

This thesis presents several key findings. Changing the CPU platform while holding memory fixed at 14 GB roughly doubles throughput, though it does not change the number of runs that end due to out-of-memory errors. Doubling the memory budget from 14 GB to 28 GB removes all of those failures and raises completion from 56.8 % to 70.8 %. Raising the thread budget from 12 to 32 does almost nothing at all, as the pipeline saturates at about seven busy threads. Which method is more accurate stays consistent across every machine, with rank correlations between 0.89 and 0.95 across conditions; which method actually finishes varies significantly, dropping to correlations between 0.53 and 0.81.

Contents

Abstract **i**

1 Introduction **1**
1.1 Practical Limits of Visual SLAM . . . 1
1.2 Problem statement . . . 2
1.3 Why pySLAM as the Base of Research . . . 2
1.4 Contributions . . . 3
1.5 Scope and delimitation . . . 3
1.6 Outline . . . 3

2 Background: Visual SLAM and the System Under Test **4**
2.1 The ecosystem . . . 4
2.2 The visual-SLAM branch . . . 5
2.3 Anatomy of the pipeline . . . 6
2.3.1 Feature extraction and tracking ① . . . 7
2.3.2 Place recognition and loop closure ② . . . 8
2.3.3 The optimization back end ③ . . . 10
2.3.4 The depth source ④ . . . 11

3 The Interchangeable Components **12**
3.1 Feature extraction ① . . . 12
3.1.1 How hand-crafted detection and description work . . . 12
3.1.2 How CNN detect-and-describe networks work . . . 13
3.1.3 How attention-based matching works . . . 14
3.1.4 The same frame, three front ends . . . 15
3.1.5 The catalogue . . . 17
3.2 Loop closure ② . . . 18
3.2.1 How bag-of-words place recognition works . . . 19
3.2.2 How learned global descriptors work . . . 19
3.2.3 How foundation-model descriptors work . . . 19
3.2.4 The catalogue . . . 20
3.3 Back end ③ . . . 21
3.4 Depth estimation ④ . . . 22

4 The KITTI-360 Benchmark **23**
4.1 The dataset and the recording platform . . . 23
4.2 What this study uses, and what it discards . . . 24
4.3 The four sequences . . . 25
4.4 Turns and low-overlap manoeuvres . . . 27
4.5 Where the runs end . . . 28
4.6 Revisit structure, and what loop closure has to work with . . . 30
4.7 Map growth: what the sequences cost in memory . . . 30
4.8 What this dataset cannot test . . . 32

5 Experimental Design **33**
5.1 Protocol . . . 33
5.1.1 The independent vocabulary . . . 33
5.1.2 How the combined pairs were chosen . . . 34
5.1.3 Two passes, and the skip list . . . 34
5.1.4 Two ways of reading the same data . . . 35
5.2 The hardware ladder . . . 35
5.3 Budget control, run limits and safeguards . . . 36
5.4 Evaluation metrics . . . 36
5.4.1 Absolute Trajectory Error . . . 36
5.4.2 Relative Pose Error . . . 37
5.4.3 Resource monitoring . . . 37
5.5 Outcomes and statistical treatment . . . 37
5.6 How many runs this comes to . . . 38
5.6.1 Additional remarks regarding run counts . . . 38

6 Results I: Component Comparison **39**
6.1 The reference condition, and one bias to keep in mind . . . 39
6.2 How to read the tables . . . 39
6.3 The architecture axis at family level . . . 40
6.4 Feature extraction ① . . . 42
6.4.1 Accuracy and completion . . . 42
6.4.2 What each front end costs . . . 45
6.5 Loop closure ② . . . 46
6.5.1 Accuracy and completion . . . 46
6.5.2 What each loop closer costs . . . 48
6.6 Back end ③ . . . 49
6.7 Combined configurations ① + ② . . . 50
6.8 Depth estimation ④ . . . 53
6.9 How each family uses the machine . . . 56
6.10 Calibration against published systems . . . 56

7 Results II: The Hardware Ladder **60**
7.1 Changing the platform: a faster machine, but not for everyone . . . 60
7.2 Changing the memory capacity: where the failures actually went . . . 62
7.3 Changing the core count: a negative result . . . 64
7.4 What does not move . . . 65
7.5 The resource ordering . . . 66

8 Results III: Where Hardware and Algorithm Interact **67**
8.1 Loop closure is a liability when starved and an asset when fed . . . 67
8.2 A failure label is not a diagnosis . . . 68
8.3 Neither block can be judged without the other . . . 71

9 Discussion **72**
9.1 RQ1: how the component families compare at fixed hardware . . . 72
9.2 RQ2: what changes when the hardware changes . . . 73
9.3 RQ3: decomposing “better hardware” . . . 73
9.4 RQ4: is footprint reduction a substitute for capacity? . . . 74
9.5 What transfers beyond pySLAM . . . 74
9.6 A decision guide . . . 75

9.7 Threats to validity 76

10 Conclusion **77**

A How pySLAM Works **78**
A.1 Overview and structure 78
A.2 Local features: the front end 78
A.3 Frames, keyframes and initialisation 79
A.4 Tracking 79
A.5 Local mapping 80
A.6 Loop closing 80
A.7 Back ends, the C++ core and parallelism 81
A.8 Modules added in this work 81
A.9 Why the framework shapes the results 82

B Environment and Known Issues **84**
B.1 The guest configuration 84
B.2 Organisation and dataset preparation 84
B.3 The variant table 85
B.4 Running one sequence 85
B.5 Batch execution, limits and aggregation 86
B.6 Known issues and honest mistakes 87

C The Two Machines **89**
C.1 The two systems 89
C.2 The processors 90
C.3 DDR4 against DDR5 90
C.4 What capacity costs in 2026 91
C.5 The confound I could not remove 92

D The Full Run Matrix **93**

Bibliography **96**

CHAPTER 1

Introduction

1.1 Practical Limits of Visual SLAM

Simultaneous Localisation and Mapping (SLAM) lets a moving camera build a map of an unknown scene while tracking its own position inside it. The two halves feed each other: a better map sharpens localisation, and better localisation sharpens the map. That coupling is what makes the problem hard and what gives the field its name. SLAM is used on almost all autonomous driving and mobile robotics, and on those platforms it has to run on whatever compute budget the vehicle can carry: often a modest embedded processor with a small GPU, a tight power budget, and no second chance if the tracker fails mid-route.

Over the last decade deep learning models have reached into nearly every block of the pipeline. There are learned feature detectors and descriptors, learned place recognition for loop closure, and neural depth estimation that promises metric scale from a single camera. Published comparisons are almost always framed around accuracy, and almost always measured on capable desktop or server GPUs [1]. That leaves a gap precisely where it matters most for deployment. A method that lowers trajectory error by a few percent is of little use to an edge robot if it cannot sustain real-time throughput, saturates the GPU, exhausts system memory, or simply loses tracking and never recovers.

While this thesis was being written ,through late 2025 and into 2026, DRAM demand from AI datacentre buildout pulled manufacturing capacity away from consumer memory, and prices rose to levels the desktop market has not seen before: the 32 GB DDR5 kit used in the current machine of this study cost roughly three times its 2024 price (Appendix C). For a technology that is supposed to end up on vehicles and robots built to a unit cost, this matters directly. If a perception stack only works with generous memory headroom, and headroom is the component whose price is rising fastest, then knowing exactly how much memory each algorithm needs stops being an implementation detail and becomes a design constraint.

The SLAM systems that actually ship are still the classical ones: the ORB-SLAM family and its derivatives [2], [3], [4]. The closest prior work benchmarks whole open-source SLAM systems against each other with similar practical concerns [1]. I do the opposite: I hold one system fixed and compare the interchangeable components inside it, so that a difference in the numbers is attributable to the block I swapped rather than to two entirely different codebases.

1.2 Problem statement

The central question is empirical: *how is memory capacity, single-core performance, and multithreading constraints affecting deep-learning SLAM components compared to classical baselines?*

This question splits into four research questions, and every results chapter answers one or more of them. Section 9.1 onwards returns to each in turn.

RQ1 At a fixed hardware condition, how do the component families compare on accuracy, resource cost, and completion, and is a single pooled ranking the right thing to report, or does the answer depend on what the route contains?

RQ2 Which of those comparisons change when the hardware changes, and which are invariant?

RQ3 Can the effect of "better hardware" be decomposed into platform speed, memory capacity, and core count, which of the three binds, and what is it inside the pipeline that consumes the binding resource?

RQ4 Is intrinsic footprint reduction (running without loop closure) substitutable for extrinsic capacity increase (more RAM)?

Three words recur throughout and are worth fixing now. A *variant* is one configuration of the pipeline, such as AKAZE features with an OBIndex2 loop closer. A *cell*, equivalently a *variant and sequence pair*, is one variant run on one particular drive; a cell is normally three repetitions. A *condition* is one of the four hardware settings. The distinction matters when counting: a variant is recommended or not, but it is cells that succeed, fail, or get rescued by a hardware change.

1.3 Why pySLAM as the Base of Research

This study targets feature-based, sparse-map visual SLAM in the ORB-SLAM2 lineage [3]. Its modular architecture allows isolated component swapping across three core blocks (feature extraction, place recognition, and back-end optimization) plus depth source evaluation (stereo vs. monocular with neural depth). End-to-end systems like DROID-SLAM [5] or NeRF-based SLAM [6] lack these modular seams, preventing controlled component-level comparisons.

pySLAM [7] provides the unified benchmarking platform. It wraps an ORB-SLAM2-style C++ core with Python interfaces for modern deep modules, ensuring all variants share identical tracking, map management, keyframe policies, and metrics. While native implementations would be faster, pySLAM isolates algorithmic differences from software engineering quality. Timings and memory are therefore evaluated relatively; platform overhead, framework internals, and published performance gaps are detailed in Section 6.10 and Appendix A.

1.4 Contributions

pySLAM provided the baseline platform; I built the benchmarking infrastructure on top of it, including the batch execution pipeline, metric recovery, wall-time constraints, crash-proof logging and out-of-memory handling. Using that harness, this thesis evaluates feature extractors (integrating LiftFeat, RDD and DeDoDe) and loop closers (integrating SALAD, CricaVPR and SuperVLAD) up to the 2024–2025 generation of methods, across four hardware profiles that isolate platform speed, memory capacity and core count. Treating compute resources as primary metrics alongside accuracy shows that accuracy rankings stay stable across platforms while completion rates do not; separating budget-driven from algorithmic failures shows that added memory removes crash deaths while leaving tracking loss untouched. Both results bound how far a single-machine benchmark can generalise.

1.5 Scope and delimitation

The purpose is to push the *visual* components of the pipeline to their limit and record where they break, so the sensing is deliberately minimal: one camera, or two, and nothing else. No inertial unit, no wheel odometry, no GNSS, no LiDAR. This is worth being explicit about, because the largest category of failure in the results is tracking loss at sharp turns, which is precisely what an IMU is good at mitigating. Fusing inertial data would have improved almost every number here, and a deployed system should do it. It would also have hidden what I set out to measure: with an IMU carrying the pose through a low-overlap corner, a front end that cannot match across that corner and one that can look far more similar than they are. The results are the floor a purely visual pipeline provides, not the performance of a complete navigation stack.

The study is bounded in three further ways. It covers stereo and monocular pinhole cameras only, with no fisheye support and no radiance-field methods. The dataset is KITTI-360 [8] with four selected sequences, which fixes the context as ground autonomous driving; Chapter 4 characterises it and states what it does not contain. I had hoped to add a UAV dataset for a different perspective, but most are recorded with fisheye optics that pySLAM does not support.

1.6 Outline

Chapter 2 places the work in the SLAM landscape and walks the pipeline once, so the four interchangeable blocks have names before anything is compared, and Chapter 3 says what can fill them. Chapter 4 describes the dataset and where its difficulty sits. Chapter 5 sets out the protocol, the four hardware conditions, the metrics and the statistical treatment.

The results occupy three chapters. Chapter 6 compares the components at the most capable condition, where a failure belongs to the method rather than to the machine, and checks the measurements against published figures. Chapter 7 reports what changing the machine did across all four conditions. Chapter 8 reports the findings that belong to neither half, the ones a single-machine benchmark cannot produce. Chapter 9 then answers RQ1–RQ4, states what transfers beyond pySLAM, gives a decision guide and sets out the threats to validity, and Chapter 10 closes. Four appendices carry the detail: the platform internals (Appendix A), the environment and its known issues (Appendix B), the two machines and what capacity cost in 2026 (Appendix C), and the full run matrix behind every completion figure (Appendix D).

CHAPTER 2

Background: Visual SLAM and the System Under Test

This chapter does two jobs. It locates the study in the SLAM landscape and it then walks the pipeline once so that the four interchangeable blocks have names and positions before Chapter 3 compares the methods that fill them.

2.1 The ecosystem

Figure 2.1 sketches the top level of the field. It splits immediately into two questions: what the robot senses with, and how the estimate is computed.

The first is the sensor. *Visual* SLAM uses one or more cameras, which are cheap, light and information-rich but sensitive to scale, lighting and motion blur. *Visual-inertial* systems fuse an inertial measurement unit to survive fast motion and resolve monocular scale. *LiDAR* SLAM measures geometry directly and precisely at higher cost, *LiDAR-inertial* systems add inertial support to it, *multi-sensor fusion* combines all three, and a final branch collects the specialised modalities: radar, event cameras, sonar and thermal. This study stays entirely on the purely visual branch, for the reason given in Section 1.5.

The second is how the estimation is solved, and it has only two members. *Filter-based* methods keep a single probability distribution over the current pose and map and update it recursively as each measurement arrives, as in EKF-SLAM [9], [10] and the particle-filter FastSLAM [11]. *Optimization-based* or graph methods treat SLAM as a large least-squares problem over many frames at once: bundle adjustment over points and poses, and pose-graph optimization over keyframes. Solving jointly over a window of history gives more accuracy per unit of computation, and it has been the default since that comparison was made directly [12]. It is the family this work uses, and it is why the optimization back end is one of the blocks I swap.

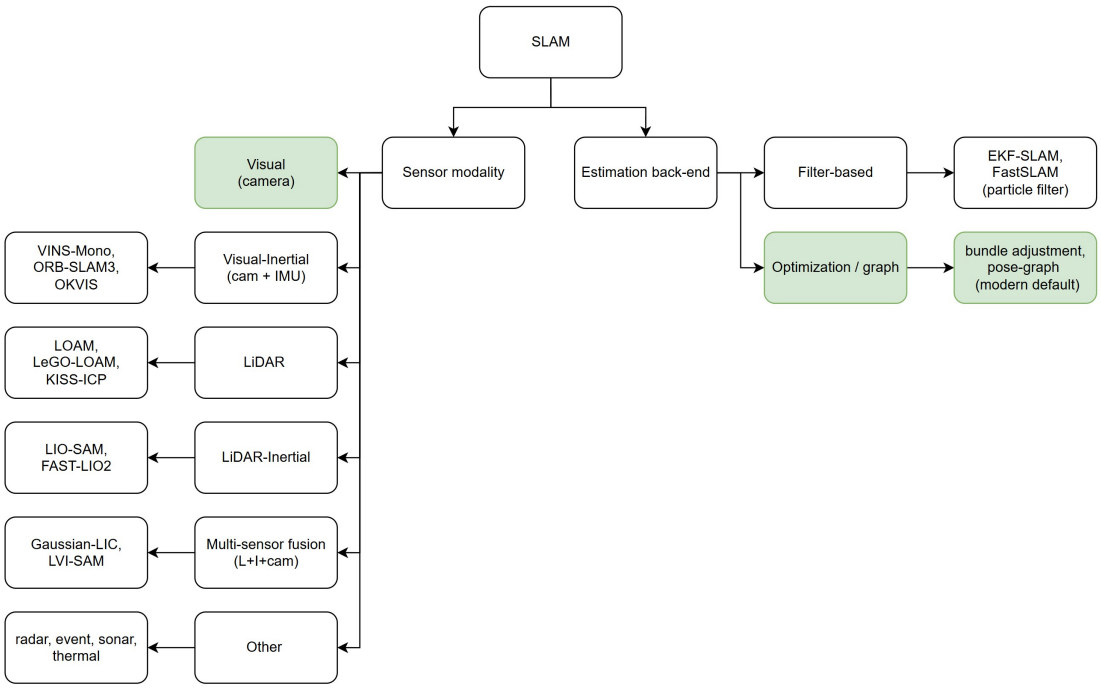


Figure 2.1: The SLAM landscape at a glance. Green marks the branches this study operates in: the purely visual sensor branch, and the optimization-and-graph back end, meaning bundle adjustment and pose-graph optimization.

2.2 The visual-SLAM branch

Inside visual SLAM the field organises along three independent axes rather than one (Figure 2.2).

The *camera type* axis runs from stereo, where depth comes from left–right disparity, through monocular, which has no depth cue at all, to RGB-D, where a sensor supplies depth directly. A fourth option has appeared recently and is treated as its own branch here: monocular plus neural depth, in which a network predicts a dense depth map from the single image and the pipeline consumes it as though it came from an RGB-D sensor. Whether that fourth option is a usable substitute for the first is the question the depth phase answers.

The *map density* axis is how much of the world the map retains: a sparse map keeps a few thousand isolated 3D points, a semi-dense map keeps the pixels with strong intensity gradients, and a dense map keeps a full surface or volume. Density tends to follow the paradigm in practice, but it is worth reading as a separate axis, because it determines whether the map fits in memory on a long drive, which turns out to matter a great deal in Chapter 7.

The *paradigm* axis is how the pixels are used. The *geometric* methods are the classical ones. *Indirect* or feature-based systems detect a sparse set of keypoints, describe them, match them across frames, and minimise the resulting reprojection error; this is the PTAM [13] and ORB-SLAM [2], [3] lineage. *Direct* systems skip features and minimise photometric error over image intensities [14], [15], while *semi-direct* systems blend the two [16]. Alongside them sit the *learning-based* methods, which either replace individual stages with neural networks or learn the whole system end to end [5]. The newest paradigm is *radiance-field* SLAM, which represents the scene as a NeRF or a set of 3D Gaussians and optimises a photorealistic dense map jointly with the trajectory [6].

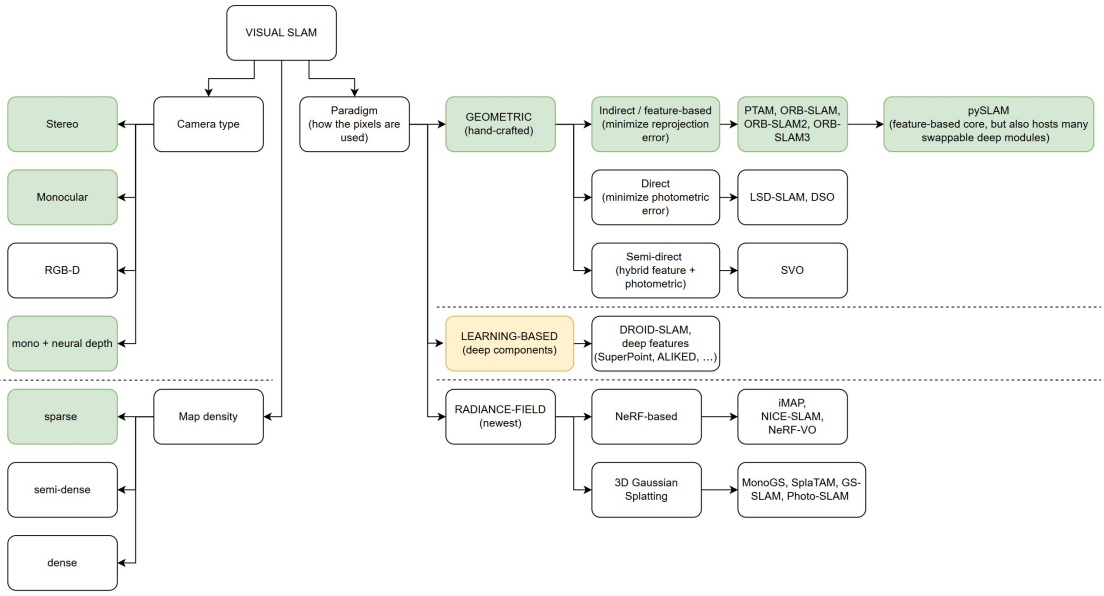


Figure 2.2: The visual-SLAM branch. Green marks where this pipeline lives; yellow marks the learning-based paradigm it borrows components from without joining.

Taken together, the two figures give a one-line description of the system under test: a sparse, feature-based, optimization-backed visual SLAM pipeline in the ORB-SLAM2 lineage, run in stereo by default and in monocular-plus-neural-depth for one phase. Everything is compared against the ORB2 baseline configuration: ORB2 features, DBoW3 place recognition [17] and a g2o back end [18], in stereo.

2.3 Anatomy of the pipeline

The baseline is the ORB-SLAM2 architecture [3], shown in Figure 2.3. It has four stages that pass a single shared map between them. *Tracking* estimates the camera pose for every incoming frame. *Local mapping* turns selected frames into new map geometry and refines it. *Loop closing* detects when the camera returns to a known place and undoes the drift that has built up. A *full bundle adjustment*, triggered once a loop has closed, polishes the entire map at once. Two modules sit underneath and are shared: a place-recognition database, which includes a visual vocabulary plus a recognition index, and the map itself, consisting of the keyframes, the sparse 3D points, and the covisibility graph recording which keyframes observe common points.

One frame's journey runs left to right through the diagram: it enters Tracking, may be promoted to a keyframe that Local Mapping expands, and if it closes a loop it triggers Loop Closing and a full bundle adjustment, all while reading from and writing to the shared map.

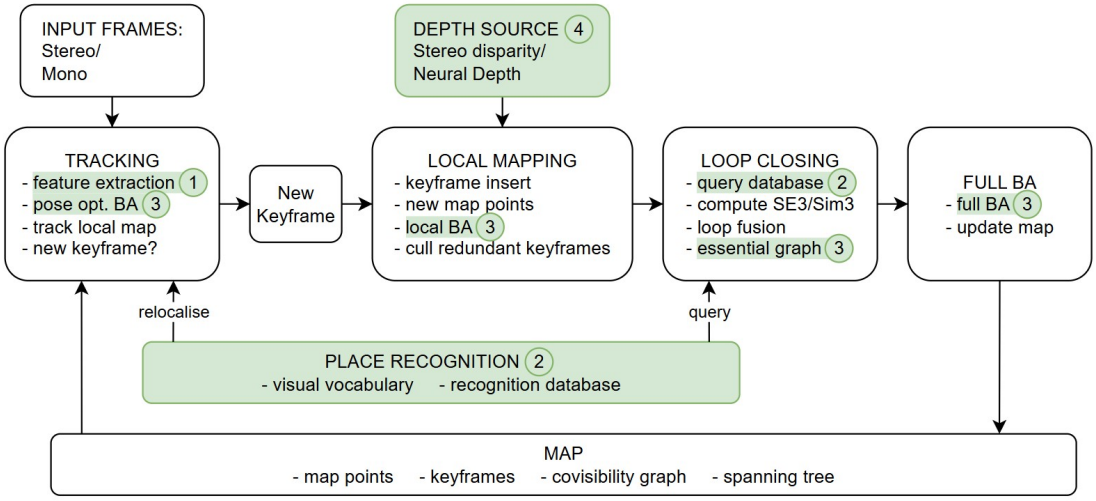


Figure 2.3: The ORB-SLAM2 architecture [3]. Green marks the four blocks this study varies, ① to ④. None is a standalone stage; the optimiser ③ appears at four points in the diagram.

2.3.1 Feature extraction and tracking ①

Feature extraction finds the points in an image worth remembering. For each incoming frame it detects a sparse set of keypoints, corners and blobs that should be recognisable again from a different position, and computes for each a compact numerical descriptor of the surrounding pixels, so the same physical feature can be matched between frames. Those matches are the raw material for everything downstream: comparing where a point appeared in the last frame against where it appears now is what lets the system infer how the camera moved, and triangulating it across views is what places it in the map. Everything after this step sees only these described points, never the full image, which is what keeps the map sparse and the system real-time.

Figure 2.4 shows this on a driving frame, with each keypoint carrying a trail marking where the same point sat in the preceding frames. Two properties of the trail pattern return later. The trails are long near the image edges and almost stationary near the centre, which is the signature of forward motion and the reason a straight drive is easy to track. And a keypoint is only useful while its trail survives; when the camera turns quickly, points sweep out of view before they can be matched, the trails break, and the pose estimate has nothing left to stand on. Chapter 4 locates exactly where on each sequence that happens, and Chapter 6 shows failures concentrating there.



Figure 2.4: Feature extraction and tracking in progress, visualised by pySLAM during a run on this study's setup. Trail length grows towards the image edges under forward motion; a broken trail is a lost correspondence.

The baseline uses ORB, multi-scale FAST corners tagged with an orientation and a compact binary descriptor [19], chosen because thousands of them can be extracted and matched in milliseconds on a CPU. The output is a few thousand described points per frame.

The rest of Tracking then estimates where the camera is. It predicts a pose from a constant-velocity motion model and projects the local map's points into the frame to find matches. If the track is lost it relocalises by querying the place-recognition module for a similar past keyframe. With matches in hand it refines the pose by motion-only bundle adjustment, the first appearance of the optimiser ③, and decides whether the frame is new enough to become a keyframe and be passed to Local Mapping. In the stereo case each keypoint also receives a depth immediately, which separates well-constrained nearby points from distant ones.

2.3.2 Place recognition and loop closure ②

Even an accurate tracker drifts. Small per-frame pose errors are individually invisible and cumulatively fatal, so after a few thousand frames a loop that should close instead leaves a visible gap in the map. Loop closure is the correction: it recognises that the camera has returned somewhere it has already been, and redistributes the accumulated error across the whole trajectory.

Recognition is the hard half, and it leans on the shared place-recognition module. Each new keyframe is summarised into a compact signature, and that signature is queried against every past keyframe. The result of every such query is the similarity matrix in Figure 2.5. The bright diagonal is uninformative, being each keyframe matching itself and its immediate neighbours; everything interesting is off-diagonal, where a bright patch means two keyframes recorded at very different times look alike. That is either a revisited place or a false alarm from repeated architecture, a similar row of trees, or an identical stretch of road.

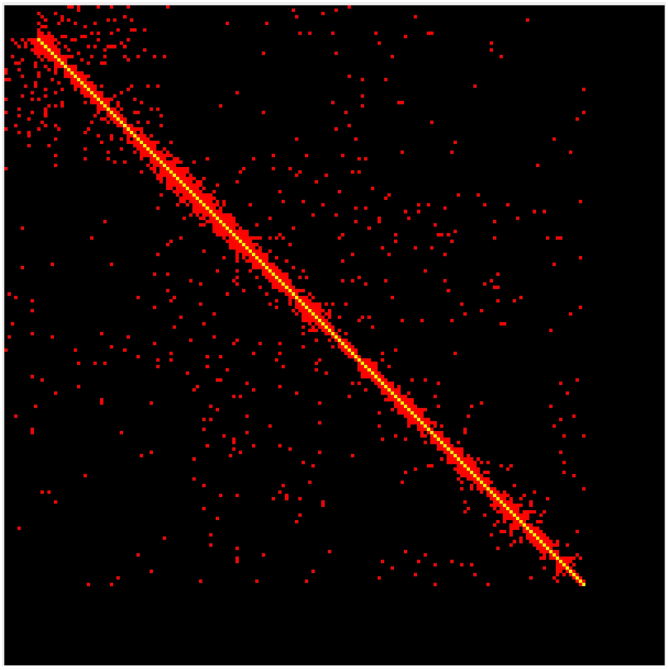


Figure 2.5: The similarity matrix produced by the place-recognition module, generated by pySLAM on this study's setup. Each entry is the descriptor similarity between two keyframes; off-diagonal brightness is a candidate revisit.

Similarity alone is not enough to act on, so a raw match is only ever a candidate. The system requires that a candidate be temporally consistent, appearing across several successive queries rather than in one isolated frame, and then geometrically verified, meaning enough feature correspondences between the two keyframes admit a single rigid transform. Most candidates do not clear both tests, and Figure 2.6 shows a typical rejection. When a candidate does clear them, Loop Closing computes the aligning transform, fuses the duplicated map points, and optimizes the pose graph over keyframes, optionally followed by full bundle adjustment. The same module serves a second purpose: when Tracking loses the camera entirely, relocalisation is the same query.



Figure 2.6: A query that produced no closure. The current keyframe is at the top in colour; below it in grey are the three past keyframes the place-recognition module returned as its nearest matches. None of them was similar enough to be accepted, so nothing was fused and the pose graph was left alone. Most of the off-diagonal brightness in Figure 2.5 ends here.

2.3.3 The optimization back end ③

Every pose and every map point in this system is an estimate, and estimates from different frames disagree. The back end reconciles them. It expresses the disagreement as a graph, with camera poses and 3D points as nodes and observations as edges, and finds the configuration that best explains all measurements at once by minimising reprojection error in a nonlinear least-squares sense. This is the machinery behind all four optimiser boxes in Figure 2.3: motion-only bundle adjustment refines a single pose against fixed map points, local bundle adjustment jointly refines a window of covisible keyframes and their points, essential-graph optimization spreads loop-closure error over the keyframe graph, and full bundle adjustment refines the entire map. They differ in what is held fixed and how large the problem is, not in what they are. Because these problems are large, sparse and solved constantly, they are delegated to a dedicated solver library rather than implemented inline.

2.3.4 The depth source ④

A single camera cannot recover metric scale: a small, close scene and a large, distant one project to the same image. Every new map point therefore needs a depth from somewhere, both to fix the scale of the reconstruction and to tell the tracker which points are well constrained. In the stereo baseline it comes from disparity, the horizontal offset of the same feature between the rectified left and right images.

In a rectified stereo pair the cameras are aligned so that corresponding points share the same image row, which reduces the search for a feature from two dimensions to one. The horizontal shift, or disparity d, yields depth directly as $Z = fb/d$ for focal length f and baseline b. Two consequences matter. Since depth is inversely proportional to disparity, distant estimates are highly unreliable, which is why this study discards depths beyond 50 m. And the physical baseline b provides absolute scale, making stereo depth a true measurement rather than an inference. Figure 2.7 illustrates the correspondence step this depends on.

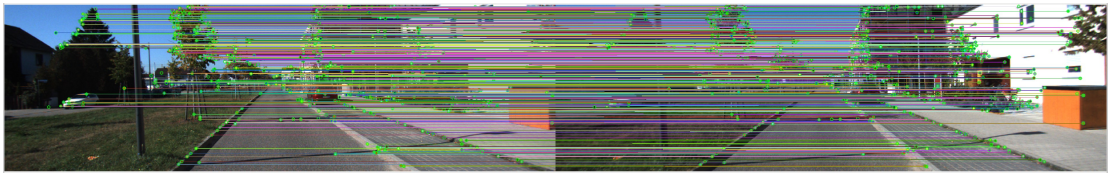


Figure 2.7: Feature matching between two views, from pySLAM's matching demonstration. The scene is not from this study's dataset.

CHAPTER 3

The Interchangeable Components

Chapter 2 named four blocks. This chapter says what can go in them. For the two front-end blocks the range of methods is wide enough to deserve both an explanation and a taxonomy, so each begins with how the three architectural families actually work before cataloguing the specific methods; the back end and the depth source are two-option forks and get a section each.

One organising axis runs through both taxonomies: from hand-crafted methods, through convolutional networks, to transformer- and foundation-model-based methods. The premise the literature offers is that moving along that axis buys robustness at a steadily rising cost in computation and memory. Measuring whether it does is what this thesis is for.

Most of the methods below ship with pySLAM [7] and are selected through its configuration interface. Six do not. Three feature extractors, LiftFeat [20], RDD [21] and DeDoDe [22], and three loop closers, SALAD [23], CricaVPR [24] and SuperVLAD [25], were integrated as part of this study, so that the comparison reaches the 2024 and 2025 generation of methods. They are marked in the tables, and what their integration required is documented in Appendix A.

3.1 Feature extraction ①

3.1.1 How hand-crafted detection and description work

A hand-crafted extractor is two hand-designed algorithms bolted together, and neither of them learns anything. The detector's job is to find pixels whose neighbourhood is distinctive enough to be re-found from a different viewpoint, which in practice means corners rather than edges or flat regions: an edge can slide along itself without changing appearance, and a flat patch matches everywhere.

The FAST test that ORB builds on is the cheapest possible version of that idea. It examines a ring of sixteen pixels around a candidate point and accepts the point if a contiguous arc of them is uniformly brighter or uniformly darker than the centre by a threshold. No gradients, no convolutions, just comparisons, which is why thousands of candidates can be tested per millisecond. Scale invariance is added on afterwards by repeating the test over an image pyramid, and rotation invariance by computing an orientation for each keypoint from the intensity centroid of its patch and then rotating the descriptor sampling pattern to match.

The descriptor then converts the patch around the keypoint into a number that can be compared. The two families differ in what kind of number. A *binary* descriptor, as in ORB, BRISK and AKAZE, samples a fixed pattern of pixel pairs inside the oriented patch and writes one bit per pair recording which of the two is brighter; the result is a 256-bit string, and matching two of them is a XOR and a population count, which modern processors do in a single instruction. A *float* descriptor, as in SIFT, RootSIFT and KAZE, instead builds a histogram of local gradient orientations over a spatial grid and stores it as a 128-dimensional vector, matched by Euclidean distance. Figure 3.1 shows

the whole sequence on one keypoint, and Figure 3.2 contrasts the two descriptor spaces. The float form retains far more information about the patch and is correspondingly more invariant to viewpoint and illumination change, but it is heavier to compute and heavier to match, and it is not what the platform's C++ projection matcher is built for.

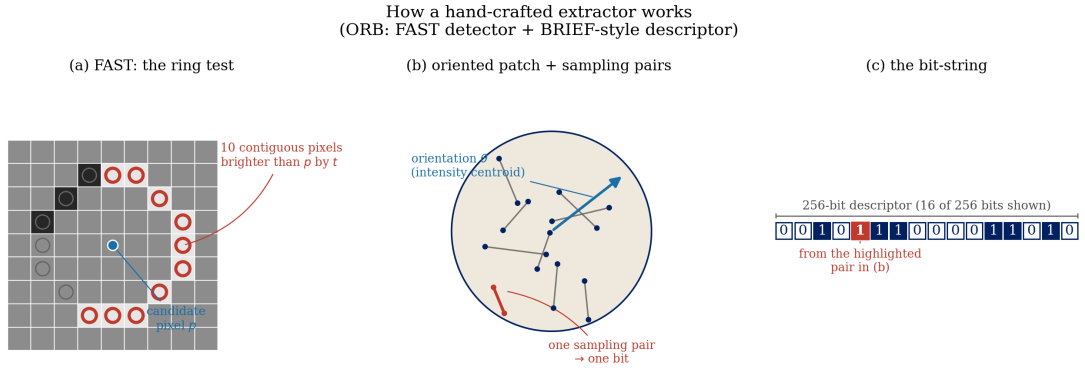


Figure 3.1: How a hand-crafted extractor works, using ORB. (a) FAST corner test. (b) Orientation from the intensity centroid. (c) Binary descriptor from rotated sampling pairs.

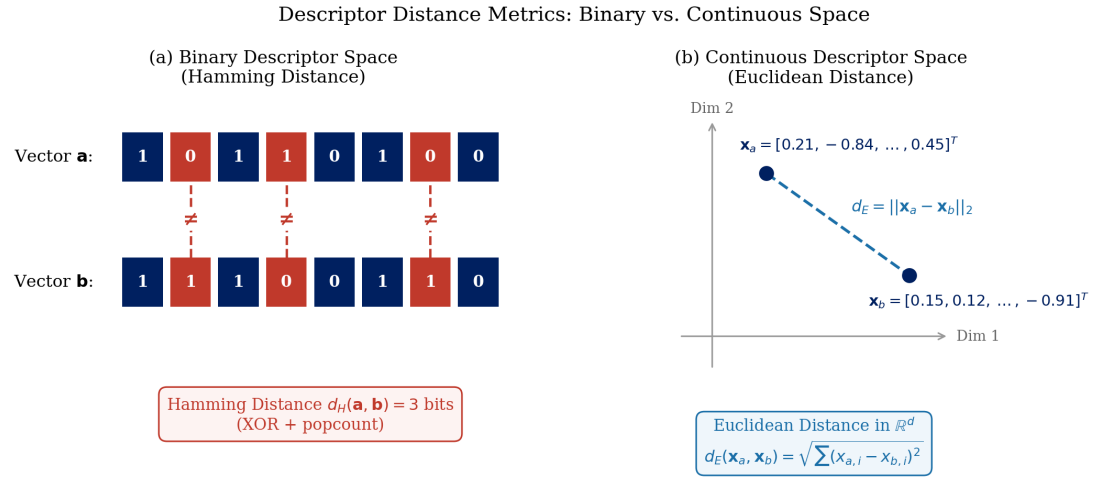


Figure 3.2: The two descriptor spaces: Hamming distance over bit strings against Euclidean distance in a float space. Most CNN extractors emit the latter.

3.1.2 How CNN detect-and-describe networks work

The learned alternative removes the hand-designed pieces and trains a single network to do both jobs at once. The architecture that dominates the field, established by SuperPoint [26], is a shared convolutional encoder feeding two heads. The encoder downsamples the input image into a feature map that has learned, over training, which local structures are worth attending to. The *detection head* converts that feature map into a dense score map, one score per pixel location, saying how keypoint-like that location is; non-maximum suppression then reduces the score map to a discrete set of keypoints. The *descriptor head* converts the same feature map into a dense descriptor tensor, one vector per location, from which the descriptors at the selected keypoint positions are sampled and normalised.

Figure 3.3 shows the architecture. Three consequences follow, and all three show up in the measurements. First, detection and description are *joint*: the descriptor is not computed from the raw patch after the fact but read out of a representation that was trained together with the detector, which is the source of the family's claimed repeatability advantage. Second, the entire cost is a forward pass over the whole image regardless of how many keypoints are wanted, so the compute is a function of resolution rather than of feature count, and it lands on the GPU. Third, the descriptors are floats, so a pipeline built for binary matching pays a conversion cost somewhere.

The family is far from uniform in how heavy that forward pass is. XFeat [27], ALIKED [28] and LiftFeat [20] are deliberately lightweight: XFeat built for raw speed, ALIKED using a deformable descriptor head to stay both light and accurate, LiftFeat adding 3D-geometry cues to a light backbone. DISK [29], R2D2 [30] and D2-Net [31] are the heavy, dense-leaning members. Key.Net [32] is the odd one out, being detector-centric and paired with a separate descriptor, so it behaves unlike the joint methods around it.

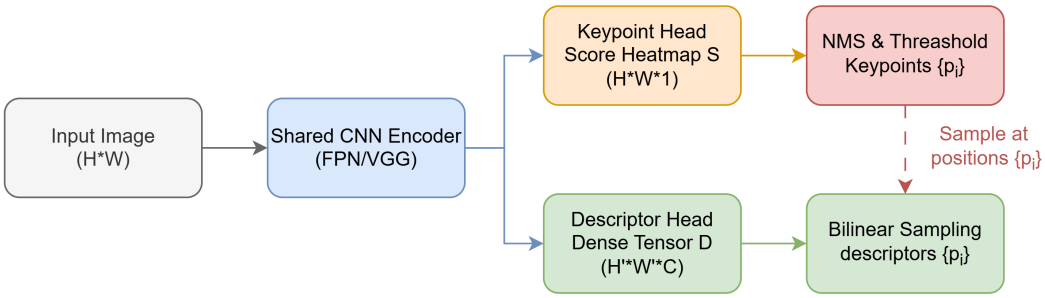


Figure 3.3: How a CNN detect-and-describe network works, using SuperPoint. A shared encoder feeds a keypoint-score head and a dense descriptor head; the cost is one forward pass on the GPU, independent of keypoint count.

3.1.3 How attention-based matching works

The third family attacks a different part of the problem. Everything above produces descriptors that are then matched by nearest neighbour: for each descriptor in image A, find the closest descriptor in image B, and keep the match if it is sufficiently better than the second-closest. That procedure treats every feature independently and knows nothing about the rest of the image.

An attention-based matcher replaces it with a network that reasons over both feature sets jointly. The keypoints of both images, together with their positions, are embedded as tokens. *Self-attention* layers let the features within one image exchange information, so that a feature can be interpreted in the context of the whole scene rather than only its own patch. *Cross-attention* layers let features in one image attend to features in the other, so the matcher can weigh a candidate correspondence against every competing candidate at once. The output is a soft assignment matrix over all pairs, from which the final matches and their confidences are read (Figure 3.4).

This is strictly more informed than nearest-neighbour matching and strictly more expensive, because attention is quadratic in the number of features. LightGlue [33] and XFeat+LightGlue keep a convolutional front end and replace only the matching stage this way. The other two push attention into the feature stage itself: RDD [21] with a deformable transformer, DeDoDe [22] with a Vision-Transformer descriptor.

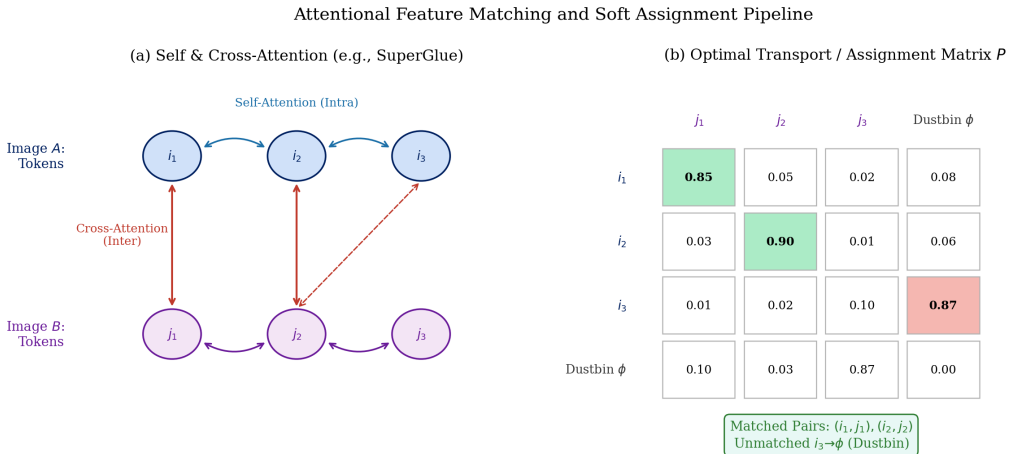


Figure 3.4: How an attention-based matcher works, using LightGlue. Self- and cross-attention produce a soft assignment matrix rather than independent nearest-neighbour decisions, at quadratic cost in the number of features.

3.1.4 The same frame, three front ends

The three mechanisms above produce visibly different results on the same input, and Figure 3.5 is worth more than the three preceding pages for a reader who wants an intuition. The hand-crafted detector clusters on high-contrast structure and leaves textureless road and sky empty; the CNN detector spreads more evenly and fires on structures that are not corners in any classical sense; the attention matcher produces fewer but more confident correspondences, with the difference concentrated exactly where nearest-neighbour matching is ambiguous.

Two things in the figure are worth pointing out. All detectors avoid the flat wall surfaces almost completely. A blank wall has no local intensity variation, so there is nothing for either mechanism to lock onto, and neither wastes budget there. Where they part company is the vegetation. ORB2 puts a large share of its keypoints inside the foliage, because leaves and branches produce exactly the high-contrast corner response it is built to find, even though those points sit on a surface that moves, self-occludes and looks different from the next viewpoint. SuperPoint and LightGlue spend less there and more on the boundaries of solid structures, where a point is likely to still be a point after the camera has moved. The detectors are not disagreeing about what is textured. They are disagreeing about what is worth tracking.

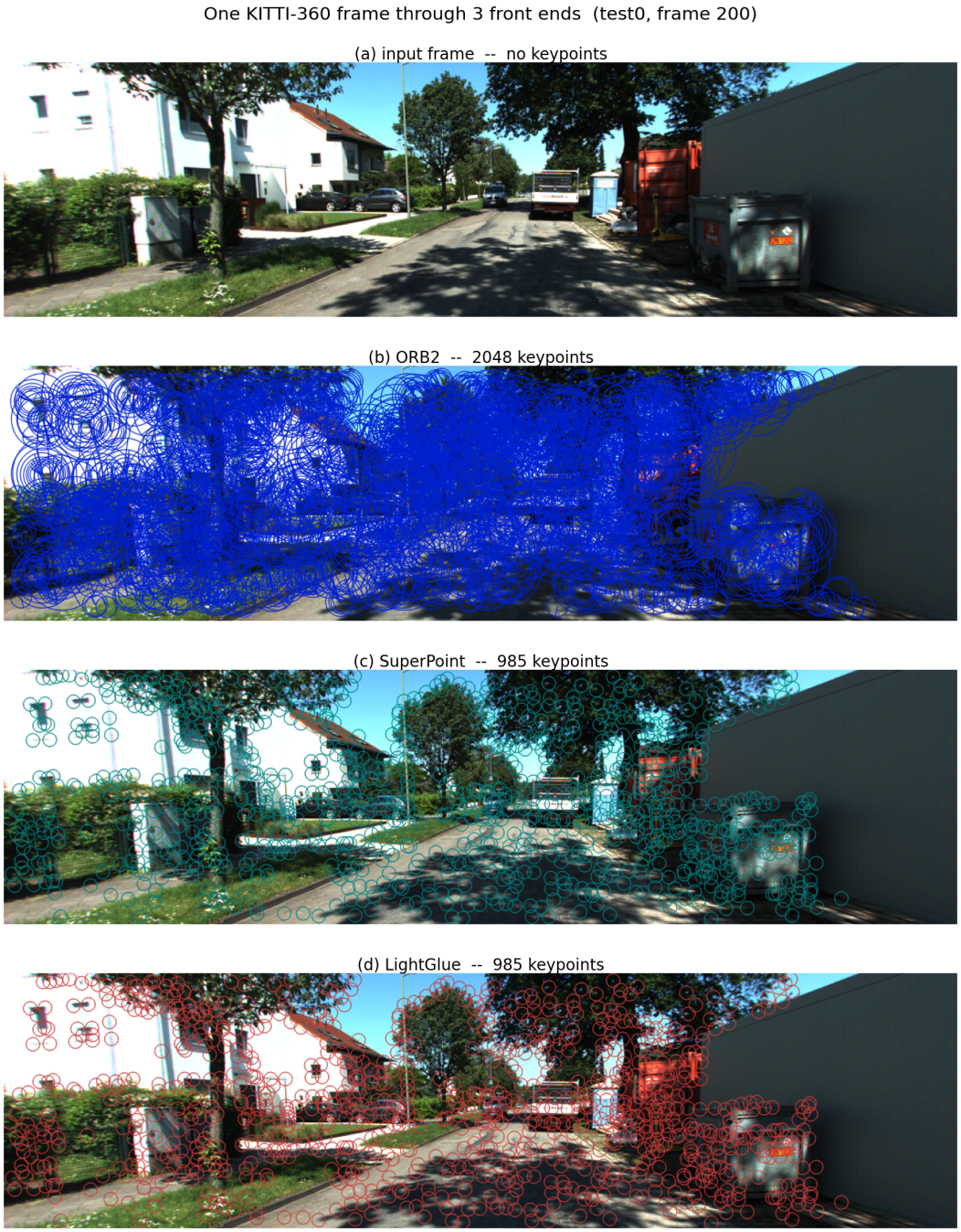


Figure 3.5: One KITTI-360 frame through three front ends, from pySLAM on this study's setup. Top: the frame as recorded. Then ORB2, then SuperPoint, then LightGlue matching against the next frame. All three front ends are asked for 2048 keypoints.

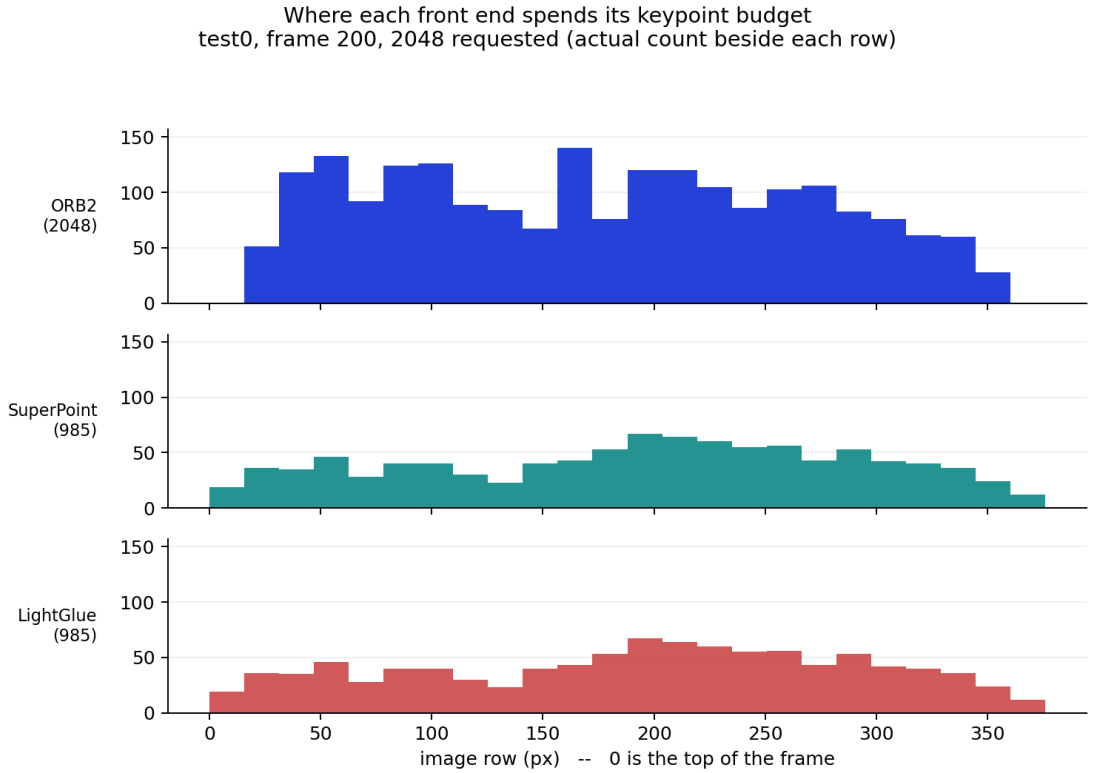


Figure 3.6: Where each front end spends its keypoint budget, binned by image row. The road surface fills the bottom of every driving frame and carries almost no texture worth detecting, so a detector spending its budget there is not sampling the geometry the tracker needs. LightGlue and SuperPoint are the same curve drawn twice, because LightGlue is configured on a SuperPoint detector so the matcher changes what is done with the keypoints, not which ones exist.

3.1.5 The catalogue

Figure 3.7 groups the field into three architectural families, and Table 3.1 lists the twenty extractors that fill them, one line each.

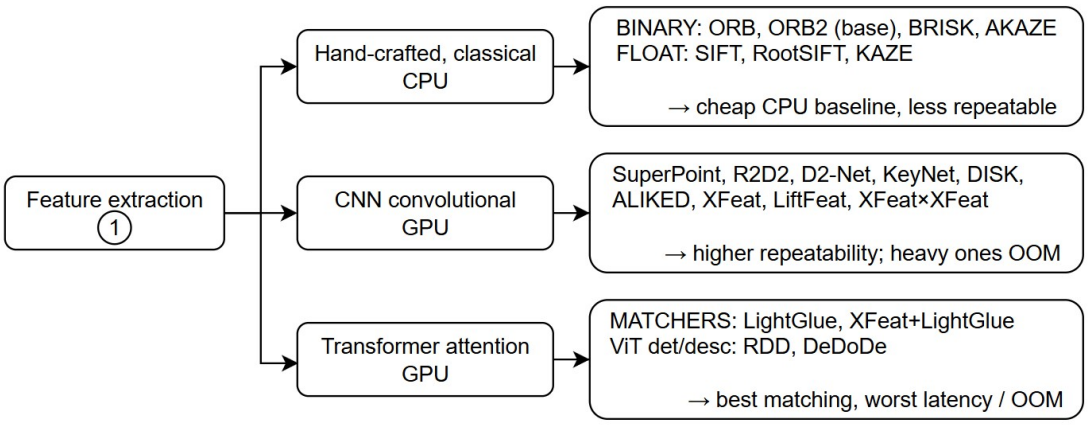


Figure 3.7: Feature-extraction families compared in this study, ordered by architecture. Robustness and compute cost are both expected to rise from left to right; Chapter 6 tests whether they do.

Table 3.1: The twenty feature extractors compared in this study, one line each. Rows run baseline, Cls, CNN, Trf. ⋆ marks a module integrated into pySLAM for this study rather than shipped with it.

Method	fam.	type	in one line
ORB2 (baseline) [3]	Cls	binary	ORB2 as shipped, the reference every other row is measured against
ORB [19]	Cls	binary	the original oriented FAST keypoint with a rotated BRIEF descriptor
BRISK [34]	Cls	binary	scale-space FAST detector, descriptor sampled on concentric rings
AKAZE [35]	Cls	binary	nonlinear diffusion keeps edges sharp across scales
SIFT [36]	Cls	float	the gradient-histogram reference, matched in Euclidean space
RootSIFT [37]	Cls	float	SIFT under a Hellinger metric, a one-line change and usually better
KAZE [38]	Cls	float	the float-descriptor ancestor of AKAZE, kept for the timing comparison
SuperPoint [26]	CNN	joint	the well-rounded reference: one network detects and describes
XFeat [27]	CNN	joint	built for raw speed on a small backbone
XFeat×XFeat [27]	CNN	semi-dense	XFeat paired with its own semi-dense matcher instead of nearest neighbour
ALIKED [28]	CNN	joint	a deformable descriptor head keeps it light without losing accuracy
LiftFeat⋆ [20]	CNN	joint	3D-geometry cues added to a lightweight backbone
DISK [29]	CNN	dense	policy-gradient training over dense features, heavy to run
R2D2 [30]	CNN	dense	predicts repeatability and reliability as separate maps
D2-Net [31]	CNN	dense	detection deferred until after description, on a dense feature map
Key.Net [32]	CNN	detector	detector-centric and paired with a separate descriptor, unlike its neighbours
LightGlue [33]	Trf	matcher	convolutional features, attention matching over both images jointly
XFeat+LightGlue [33]	Trf	matcher	the same attention matcher on XFeat's cheaper front end
RDD⋆ [21]	Trf	feature	attention pushed into the feature stage via a deformable transformer
DeDoDe⋆ [22]	Trf	feature	a Vision-Transformer descriptor, detection and description trained apart

3.2 Loop closure ②

Loop closure recognises when the camera revisits a place, and its methods fall along the same architectural axis. One practicality sets this block apart from feature extraction: the classical methods reuse the ORB descriptors the tracker has already computed and so cost almost nothing extra, whereas every deep method needs its own forward pass over each keyframe, paid on top of whatever the feature extractor already spends. That asymmetry is the reason to expect a wider cost gap between families here than in the front end.

3.2.1 How bag-of-words place recognition works

The classical approach borrows an idea from text retrieval. Offline, a very large collection of local descriptors is clustered into a vocabulary of *visual words*, organised as a tree so that assigning a descriptor to its nearest word costs a handful of comparisons rather than a scan of the whole vocabulary. An image is then summarised by quantising each of its descriptors to a word and counting how often each word occurs, producing a sparse histogram weighted so that rare words count for more than common ones. Two images are compared by comparing their histograms, and an inverted index makes it possible to retrieve the most similar past keyframes without comparing against all of them.

The strengths and the weakness both follow from the construction. It is extremely cheap, because the descriptors already exist and the histogram is sparse; it scales to thousands of keyframes; and it is completely insensitive to where in the image the words appeared, which makes it robust to moderate viewpoint change but also unable to distinguish two scenes made of the same parts arranged differently. DBoW2 and the DBoW3 baseline [17] use a vocabulary trained offline, while iBoW and OBIndex2 [39] build theirs incrementally as the run proceeds, which removes the offline training requirement at the cost of an index that grows. VLAD [40] aggregates the local descriptors into a single vector rather than a histogram, and SAD compares whole down-sampled images directly, with no descriptor stage at all.

3.2.2 How learned global descriptors work

The learned alternative replaces the whole pipeline with a network that maps an image directly to a single fixed-length vector, trained so that images of the same place land close together and images of different places land far apart. NetVLAD [41] is the architecture that established the family: a convolutional backbone produces a dense grid of local features, and a differentiable aggregation layer, structurally a soft version of VLAD, pools them into one vector by accumulating each feature's residual against a set of learned cluster centres. Because the aggregation is differentiable, the backbone and the pooling are trained end to end on a place-recognition objective rather than on a proxy.

Retrieval then becomes a nearest-neighbour search in that vector space, which is fast. The cost is entirely in producing the vector, which is one forward pass per keyframe on the GPU. AlexNet [42] is included as a deliberately weak, older baseline; CosPlace [43] and EigenPlaces [44] are efficient modern networks trained specifically for place recognition.

3.2.3 How foundation-model descriptors work

The newest family keeps the global-descriptor idea and changes what produces the local features. Instead of a convolutional backbone trained on a place-recognition dataset, they use a frozen DINOv2 [45] Vision Transformer, a model trained self-supervised on a very large image corpus, and train only the aggregation on top of its patch tokens. The idea is that a general-purpose visual representation of this scale already encodes what makes a place a place, and generalises better than a backbone fitted to one dataset.

The four members differ only in the aggregation. SALAD [23] treats the assignment of patch tokens to clusters as an optimal-transport problem, CricaVPR [24] adds correlation across images in a batch, SuperVLAD [25] compresses a VLAD-style descriptor to fewer parameters, and MegaLoc [46] is a recent all-rounder trained across many datasets at

once. The property they are all reaching for is invariance to viewpoint and appearance change of a kind a word histogram cannot survive; Section 6.5 identifies the one sequence in this study where that property is what is actually needed. Figure 3.8 places the three mechanisms side by side.

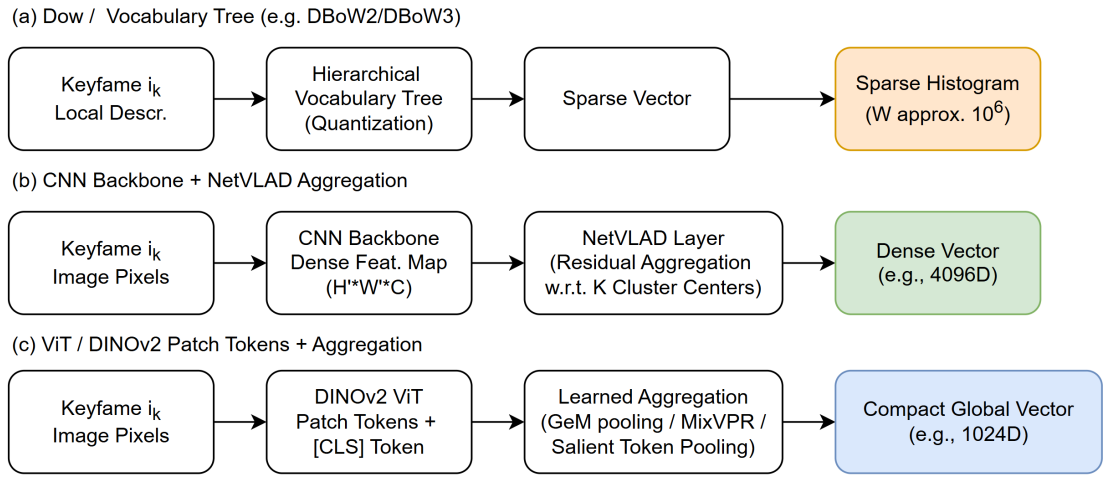


Figure 3.8: Three ways to summarise a keyframe. (a) Bag of words: local descriptors quantised against a vocabulary tree. (b) Learned global descriptor: trained pooling over a CNN backbone. (c) Foundation-model descriptor: trained pooling over frozen DINOv2 tokens.

3.2.4 The catalogue

Figure 3.9 places the fourteen closers on the architectural axis, and Table 3.2 lists them one line each.

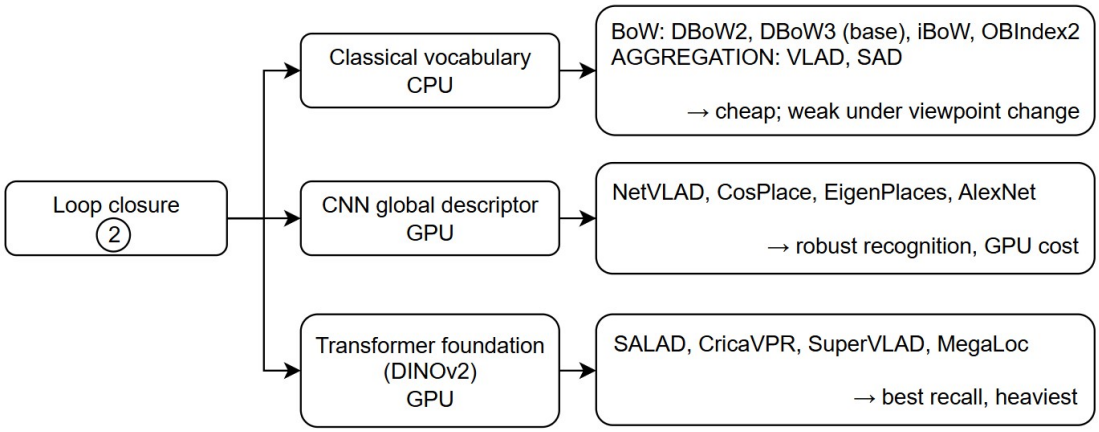


Figure 3.9: Loop-closure (place-recognition) families compared in this study, ordered by the same architectural axis as the feature extractors.

Table 3.2: The fourteen loop closers, one line each. Columns and row order as in Table 3.1. The classical rows reuse descriptors the tracker has already computed; every deep row needs its own forward pass per keyframe.

Method	fam.	type	in one line
DBoW3 (baseline) [17]	Cls	BoW, offline	DBoW3 as shipped, reusing the tracker's own ORB descriptors
DBoW2 [17]	Cls	BoW, offline	the 2012 bag of binary words, vocabulary trained ahead of the run
iBoW [39]	Cls	BoW, online	builds its vocabulary incrementally as the drive proceeds
OBIndex2 [39]	Cls	BoW, online	the incremental index behind iBoW, used directly
VLAD [40]	Cls	aggregation	local descriptors pooled into one vector, no vocabulary lookup
SAD	Cls	whole image	down-sampled images compared directly, no descriptor stage at all
AlexNet [42]	CNN	global desc.	the deliberately weak older baseline for the learned descriptors
NetVLAD [41]	CNN	global desc.	the classic learned aggregation layer over convolutional features
CosPlace [43]	CNN	global desc.	trained for place recognition as a classification problem
EigenPlaces [44]	CNN	global desc.	trained on viewpoint-diverse views of the same place
SALAD* [23]	Trf	DINOv2	optimal-transport aggregation over the frozen backbone
CricaVPR* [24]	Trf	DINOv2	adds cross-image correlation between candidate keyframes
SuperVLAD* [25]	Trf	DINOv2	a VLAD-style descriptor compressed to a compact vector
MegaLoc [46]	Trf	DINOv2	the recent all-rounder, trained across many place-recognition datasets

3.3 Back end ③

The back end is the solver behind every optimization step in the pipeline. Unlike the front-end blocks it is a straight two-way comparison rather than a family tree, and there is no deep-learning alternative, which makes it the one block where the comparison is between two mature engineering libraries rather than between paradigms.

g2o [18] is the solver ORB-SLAM2 ships with: a general framework for graph optimization with sparse block-structured Jacobians, hand-tuned for exactly the bundle-adjustment and pose-graph problems this pipeline generates. GTSAM [47] formulates the same problems as factor graphs and solves them by variable elimination into a Bayes tree, which supports incremental re-solution when only part of the graph changes. Both minimise the same nonlinear least-squares objective and both are used in shipping systems, so the question is narrowly empirical: on this workload and this hardware, is either faster, lighter or more accurate than the other?

3.4 Depth estimation ④

The depth source fixes the metric scale of new map points, and the comparison is again a fork. The stereo baseline derives depth from left–right disparity, which is a measurement anchored to a known 0.5942 m baseline. The monocular alternative discards the second camera and predicts a dense depth map from a single image with a neural network: DepthAnything V2 [48] and DepthAnything V3 [49] here. Note what changes about the nature of the number: stereo depth is triangulated from a measured baseline, while predicted depth is inferred from appearance alone, with nothing physical fixing its scale.

Figure 3.10 shows what the tracker actually receives on the substitute path. The prediction is plausible: the road surface recedes smoothly, parked cars and the kerb line separate cleanly from the background, and the near field is confidently close. The failure is at the edges. Around every foreground object the network leaves a band of intermediate depth where the true discontinuity is abrupt, so a keypoint landing on a car silhouette or a pole receives a depth interpolated between the object and the wall behind it. Those points are geometrically wrong but numerically unremarkable, which is exactly the kind of error the tracker cannot reject. The bottom panel is the same map after shadow filtering, which discards points straddling large depth jumps; this is the form fed to Local Mapping in every depth variant reported here. Filtering removes the worst offenders but thins the map precisely where the structure is, and it cannot supply the one thing stereo gets for free.

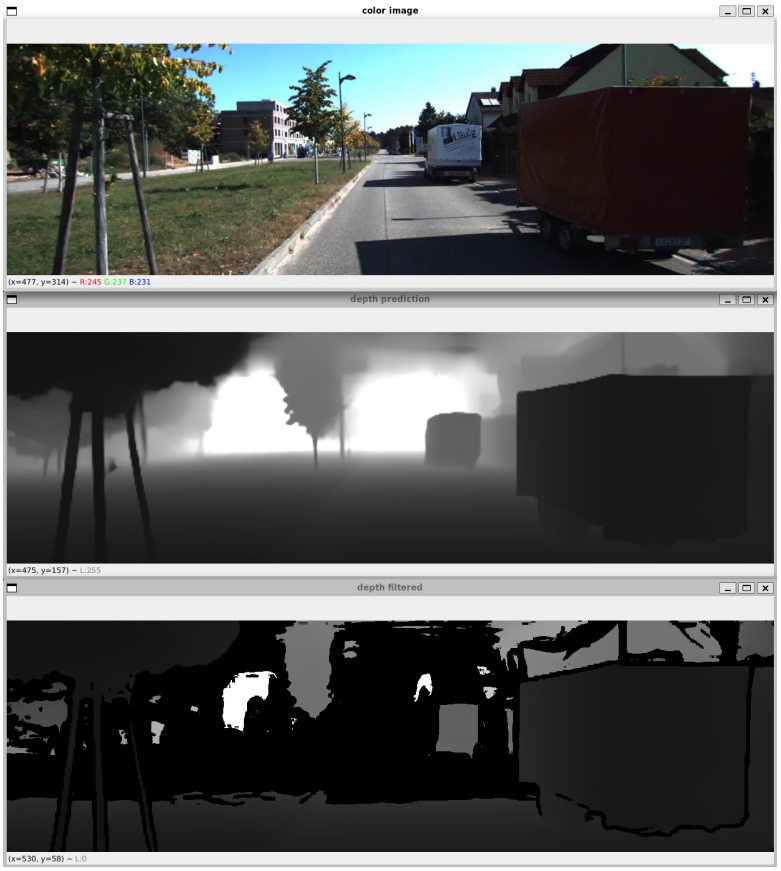


Figure 3.10: Monocular depth on a KITTI-360 frame with DepthAnything V3, at the 50 m cutoff. Top: input. Middle: raw prediction. Bottom: after shadow filtering, which is what the tracker consumes.

CHAPTER 4

The KITTI-360 Benchmark

This chapter says what those stretches contain, where their difficulty is concentrated, and what they cannot test, because a benchmark result that is not read against its data is not a result.

4.1 The dataset and the recording platform

KITTI-360 [8] is the successor to KITTI, recorded from a station wagon driven through the suburbs of Karlsruhe. It covers roughly 73.7 km of driving, with over 320,000 images and 100,000 laser scans, and adds to its predecessor dense semantic and instance annotations in both 2D and 3D, richer 360-degree sensing, and accurate geolocalised vehicle and camera poses.

The vehicle carries two forward-facing perspective cameras, two side-facing fisheye cameras, a Velodyne HDL-64E scanner and a SICK pushbroom laser, with an OXTS GNSS/IMU unit supplying pose (Figure 4.1). The ground-truth poses this study evaluates against come from that unit fused with the laser data, which is what makes the dataset usable as a SLAM benchmark rather than only as a perception dataset.

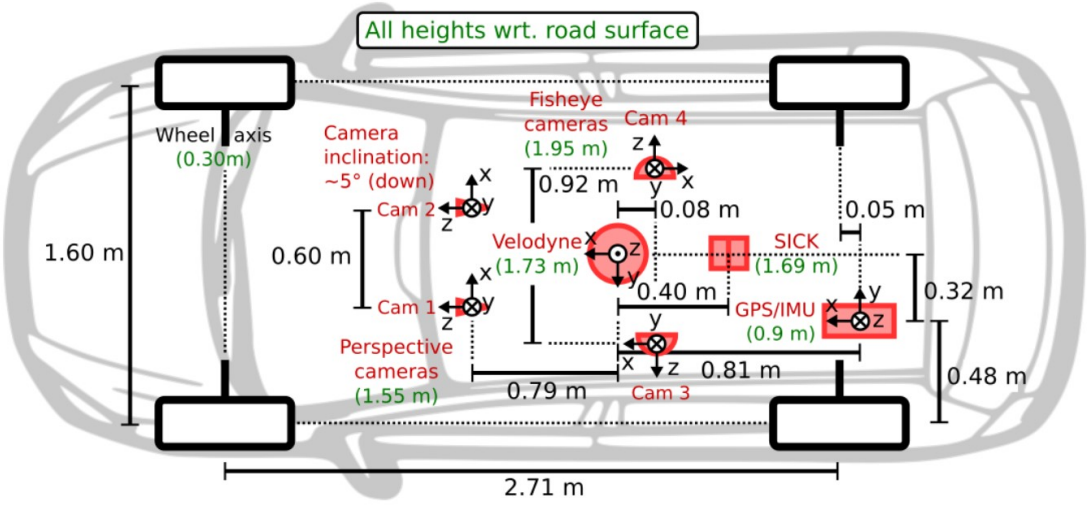


Figure 4.1: The KITTI-360 sensor layout [8]. Only the forward-facing perspective pair (Cam1 & Cam2) is used here; the inertial unit supplies ground truth only.

4.2 What this study uses, and what it discards

This study uses the forward-facing perspective pair and nothing else. The two cameras are separated by a baseline of 0.5942 m and deliver rectified images at 1408×376 px and 10 Hz. Since the pair is distributed already rectified, a point in the left image lies on the same row in the right, and the horizontal offset between them is the disparity from which stereo depth is computed (Figure 4.2).

Everything else is discarded, and each omission is deliberate. The LiDAR and the pushbroom laser are excluded because this is a study of visual SLAM. The inertial unit is excluded as an *input* for the reason given in Section 1.5: it would rescue precisely the failure mode being measured. The fisheye cameras are excluded because pySLAM does not support their projection model, which is also the reason the scope stays with a ground vehicle: the aerial datasets I had hoped to add are largely recorded with fisheye optics.

Two settings are held constant across the entire study, so that only the swapped block can account for a difference in the results. The front end is asked for 2048 keypoints per frame, and depth beyond 50 m is discarded, because at that range a disparity of one or two pixels already dominates the estimate.



(a) `test0`, left



(b) `test0`, right



(c) `test3`, left



(d) `test3`, right

Figure 4.2: Two stereo pairs as supplied by KITTI-360, each pair stacked left above right. The images arrive rectified, so a point in the left image lies on the same row in the right, and the horizontal offset between the two is the disparity from which depth is computed.

4.3 The four sequences

The four sequences come from the dataset's test-SLAM split, which is a set of windows cut out of much longer recorded drives rather than the drives themselves. Two of them, `test0` and `test1`, are separate windows from the same drive `0008`: frames 1482 to 2414 and 2992 to 4633 respectively. `test2` is frames 54 to 2905 of drive `0004`, and `test3` is frames 15874 to 18996 of drive `0002`. Everything outside those ranges is unused, so the sequences are considerably shorter than the drives they are named after.

Table 4.1 characterises them. They form a difficulty ladder that runs left to right in every figure of this thesis, and that ladder is deliberate: a single easy sequence would rank almost every method as equivalent, and a single hard one would fail almost all of them. The spread between them is what separates the methods.

Table 4.1: The four sequences, characterised. Route geometry is computed from the ground-truth poses; a manoeuvre is a contiguous stretch above 20 deg/s sweeping more than 45°, and a revisit is a pose pair within 15 m and at least 20 s apart. Completion is the share of full-SLAM runs that finished at the reference condition `R9-12t-28GB`.

	`test0`	`test1`	`test2`	`test3`
Frames	933	1547	2848	3012
Duration (s)	97.2	171.2	298.0	326.7
Path length (m)	771	1340	2667	2478
Mean speed (m/s)	7.9	7.8	8.9	7.6
Manoeuvres $> 45°$	5	7	9	16
Peak yaw rate (deg/s)	42	38	52	52
Revisited segments	53	199	129	217
of which $> 60°$ apart	37	98	0	2
Variants attempted	45	42	26	26
Completion at `R9-12t-28GB` (%)	95.6	60.9	67.5	47.5

Figure 4.3 plots all four ground-truth trajectories at a common scale. Read alongside Table 4.1, it says what the numbers cannot: `test0` is a short, largely straight window; `test1` is longer and contains the study's clearest case of the vehicle re-entering ground it has already covered from a different heading; `test2` and `test3` are the long drives, and length here brings the sharp, low-overlap manoeuvres that the shorter windows never contain.

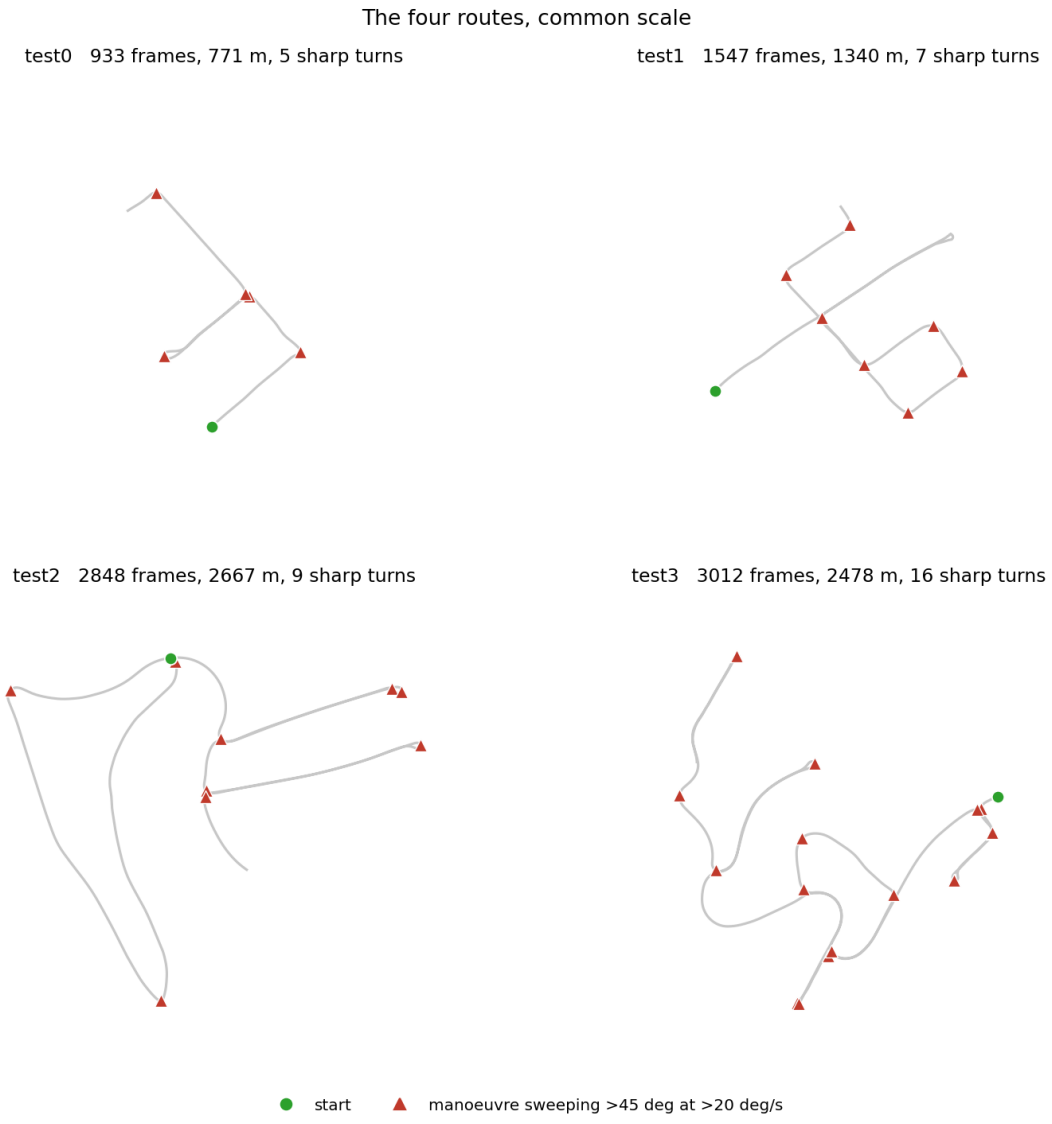


Figure 4.3: The four ground-truth trajectories at a common scale, with revisited segments highlighted and sharp manoeuvres marked. The difficulty ladder runs left to right and top to bottom.

4.4 Turns and low-overlap manoeuvres

The single most useful thing to know about these four sequences is that their difficulty is not spread evenly along them. It is concentrated in a small number of manoeuvres, and everything else is almost straight road.

The mechanism is the one Section 2.3.1 described. Under forward motion, the scene flows outward from the image centre and most of the frame remains visible from one frame to the next, so a keypoint's trail survives for many frames and the tracker has an abundance of correspondences. Under rotation, the entire frame translates sideways at once. At a 10 Hz capture rate and a realistic yaw rate for a car turning into a side street, a large fraction of the field of view is replaced between consecutive frames, and the overlap on which matching depends collapses. If the front end cannot establish enough correspondences across that gap, the pose estimate has nothing to work with, and after ten consecutive such frames the run is declared lost.

A manoeuvre is therefore counted here as a contiguous stretch of yaw rate above 20 deg/s that sweeps more than 45° in total. At 10 Hz it is only two degrees of rotation per frame, which no single frame pair would notice, so on its own it would flag every lane change and every drift in the road. The sweep condition is what does the selecting: it keeps only rotation that is sustained long enough to replace the view.

Figure 4.4 plots yaw rate against frame index for each sequence, which turns that description into a measurement: the spikes are the manoeuvres, and their height is how much of the field of view is being replaced per frame.

Two things stand out once the manoeuvres are isolated this way. First, they are almost all real corners. Very few of the stretches that pass the filter stop near the 45° floor; the great majority sweep past 90°, which is what a vehicle does when it turns into a cross street rather than when it follows a bend. There is no meaningful population of mild turns in these sequences. A manoeuvre here is either absent or severe, which is why the failures later group so sharply instead of tapering.

Second, the vehicle drives at close to the same speed throughout, between 7.6 and 8.9 m/s on average across all four sequences, and that average is not hiding much variation within any of them. This matters for reading the figure. Translation between frames is roughly constant, so the overlap available to the tracker is set by rotation and almost nothing else. The yaw-rate trace is not a proxy for something else happening at the same time; it is the difficulty signal on its own.

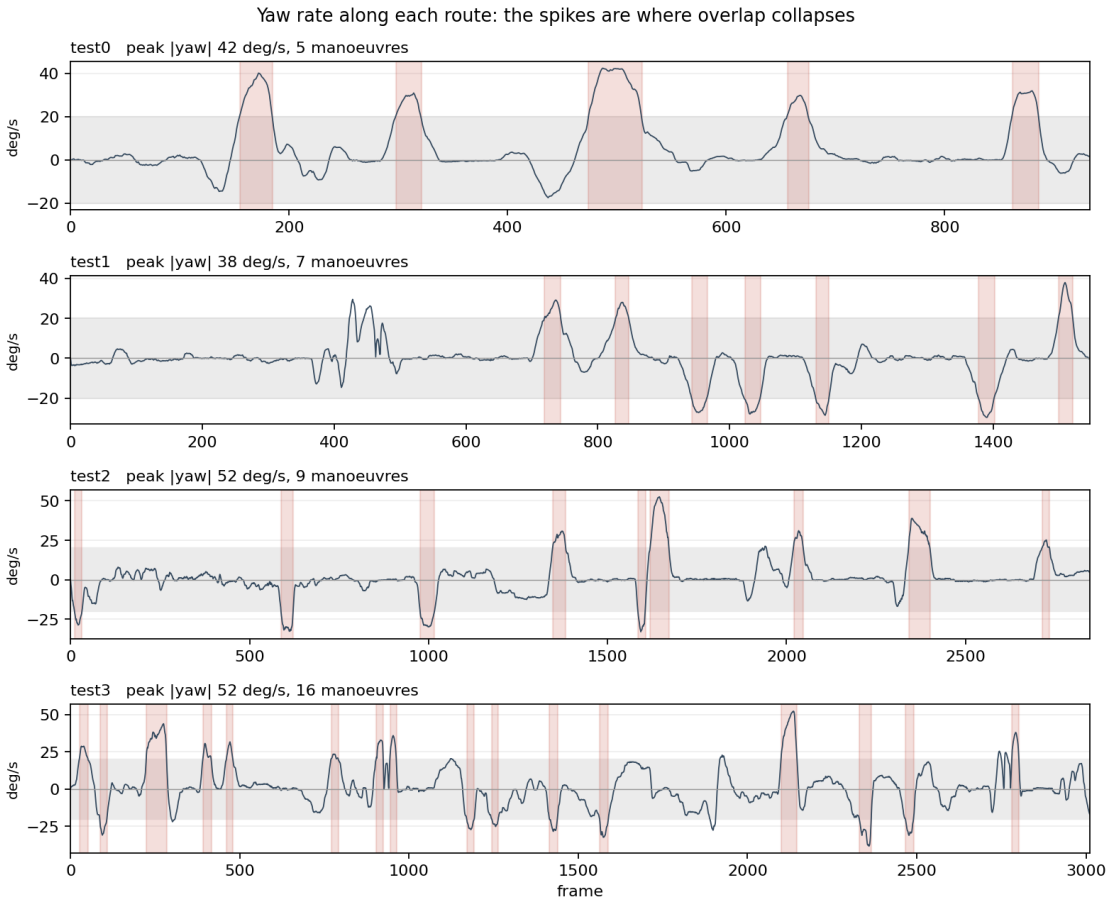


Figure 4.4: Yaw-rate profile per sequence, from the ground-truth poses. The line marks 20 deg/s. Difficulty is concentrated in a handful of spikes rather than distributed across the drive.

4.5 Where the runs end

Because the harness records how far each run progressed before it stopped, every failure can be placed on the map. Figure 4.5 does that: each sequence's ground-truth path is coloured by how many runs terminated at each point, across every variant at the reference condition.

If failures were caused by accumulated drift, they would be spread evenly along the route with a mild bias toward the end. They are not. On `test0`, `test1` and `test3` the terminations sit on the manoeuvres or in the short stretch just after them, lining up with the spikes in Figure 4.4. The lag is worth noting: a run often survives the corner itself and dies a second or two later, because the tracker limps out of the turn on a thin set of correspondences and then runs out of them on the straight that follows. The corner does the damage; the straight collects the body.

`test2` does not behave that way, and it is the more interesting case. Its failures cluster on a stretch that is close to straight, well away from any of its nine manoeuvres. What is there instead is vegetation. The road is lined with dense foliage on both sides, and foliage is the worst kind of texture for this pipeline: it produces a strong corner response, so detectors spend their budget on it happily, but the points sit on surfaces that move in the wind, occlude each other and change shape with a small change of viewpoint. The tracker is not short of keypoints there. It is short of keypoints that mean

the same thing two frames later. A handful of terminations elsewhere have a similar character, usually severe light bloom washing out one side of the frame.

So the difficulty on `test2` is environmental rather than geometric, and it is the one sequence where the yaw-rate trace does not predict the failures. This is worth carrying into the results. A method that survives `test2` has demonstrated something different from a method that survives `test3`: robustness to weak and unstable texture, rather than the ability to match across a viewpoint gap. Chapter 6 reports which methods manage which, and they are not always the same methods.

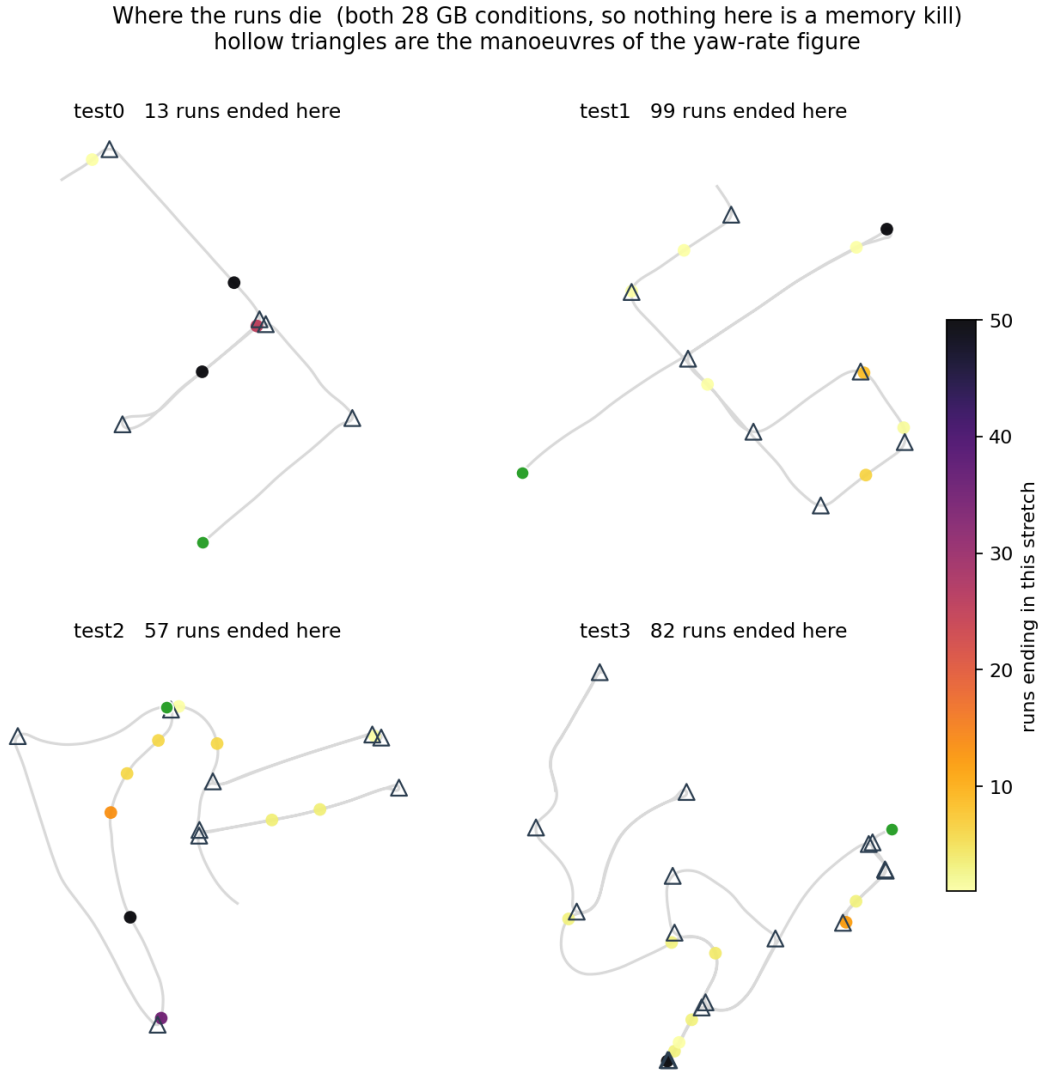


Figure 4.5: Where runs die. Each sequence's ground-truth path is coloured by how many runs terminated at that point, pooled over every variant at the reference condition `R9-12t-28GB`. On `test0`, `test1` and `test3` the clusters sit on or just after the manoeuvres; on `test2` they sit on a straight, heavily vegetated stretch. Termination points are recovered from the per-frame progress log, which survives even a killed run.

4.6 Revisit structure, and what loop closure has to work with

Loop closure can only correct drift where the vehicle actually revisits ground, and the four sequences differ in how they do so. The distinction that matters is not how much ground is revisited but from what heading. A revisit approached along the same direction of travel presents the place-recognition module with two images that already look alike, and a bag of visual words handles it comfortably. A revisit approached from a materially different heading presents two images of the same place that share little pixel-level appearance, and recognising it requires a descriptor with genuine viewpoint invariance.

Table 4.1 records the revisit counts; Figure 4.6 shows their geometry. `test1` is the sequence where the heading difference is large, and it is precisely the sequence on which the DINOv2-based closers post their only wins in this study (Section 6.5). On the other three, revisits are closer to head-on, the invariance is not needed, and the cheap closers take the lead back. Reading the loop-closure results without this figure produces the wrong conclusion, which is why it is here rather than in an appendix.

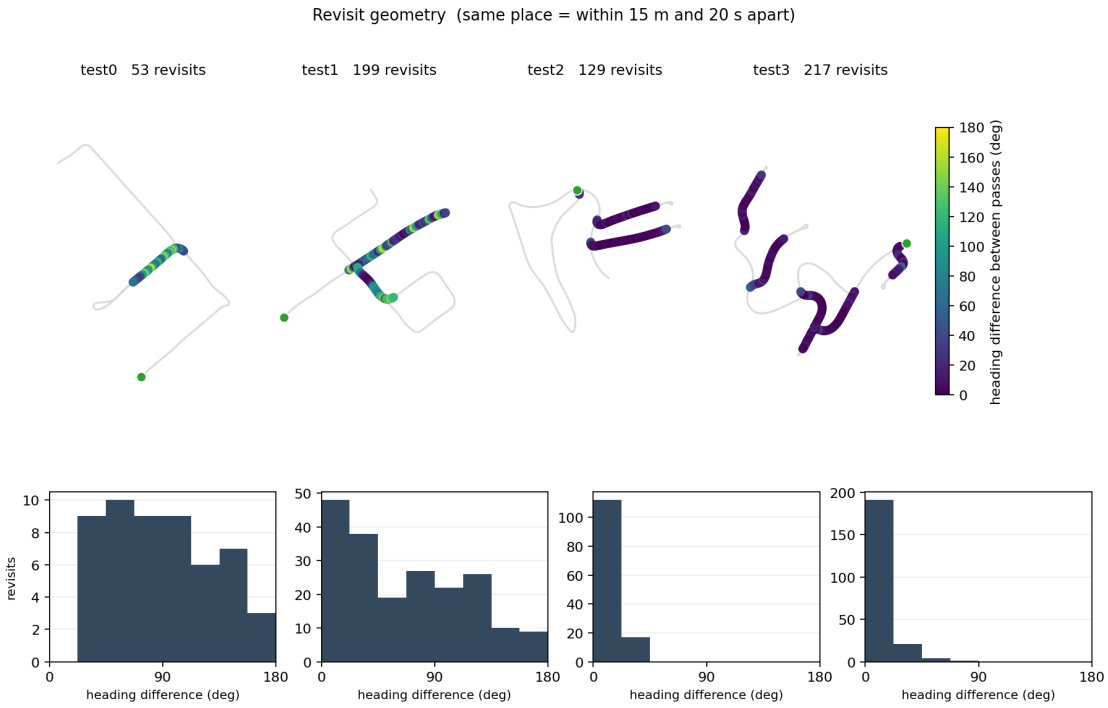


Figure 4.6: Revisit geometry per sequence, coloured by the heading difference between passes. `test1` holds the study's only substantial population of large-heading revisits.

4.7 Map growth: what the sequences cost in memory

One property of the data determines the entire hardware half of this thesis, and it belongs here rather than in the results. The pipeline's memory requirement is a function of *how far the vehicle drives*, not of how hard the scene is. Keyframes accumulate as the route extends, each costing roughly 8 to 12 MB under the independent vocabulary of Section 5.1.1, and they are never discarded within a run.

Figure 4.7 shows keyframes accumulating against distance driven, and Figure 4.8 shows the consequence: across the 139 cells at the reference condition, peak resident

memory correlates with keyframe count at $r = 0.78$. That is the direct evidence for a claim Chapter 7 rests on, namely that what exhausts memory is the map rather than the model. It also explains why the memory ceiling binds on `test2` and `test3` and not on `test0`: the long drives simply accumulate more map.

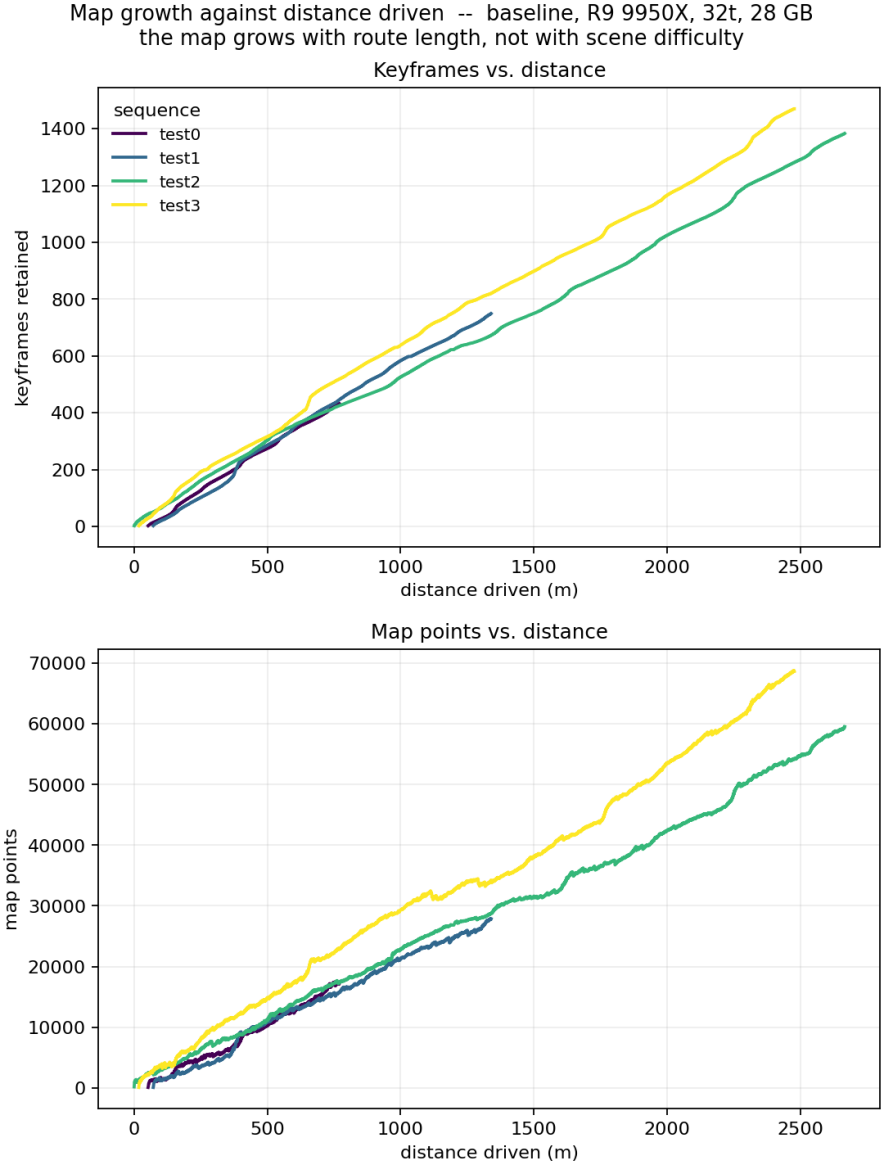


Figure 4.7: Map growth against distance driven, baseline configuration. Keyframe count tracks route length, and the map is never pruned within a run.

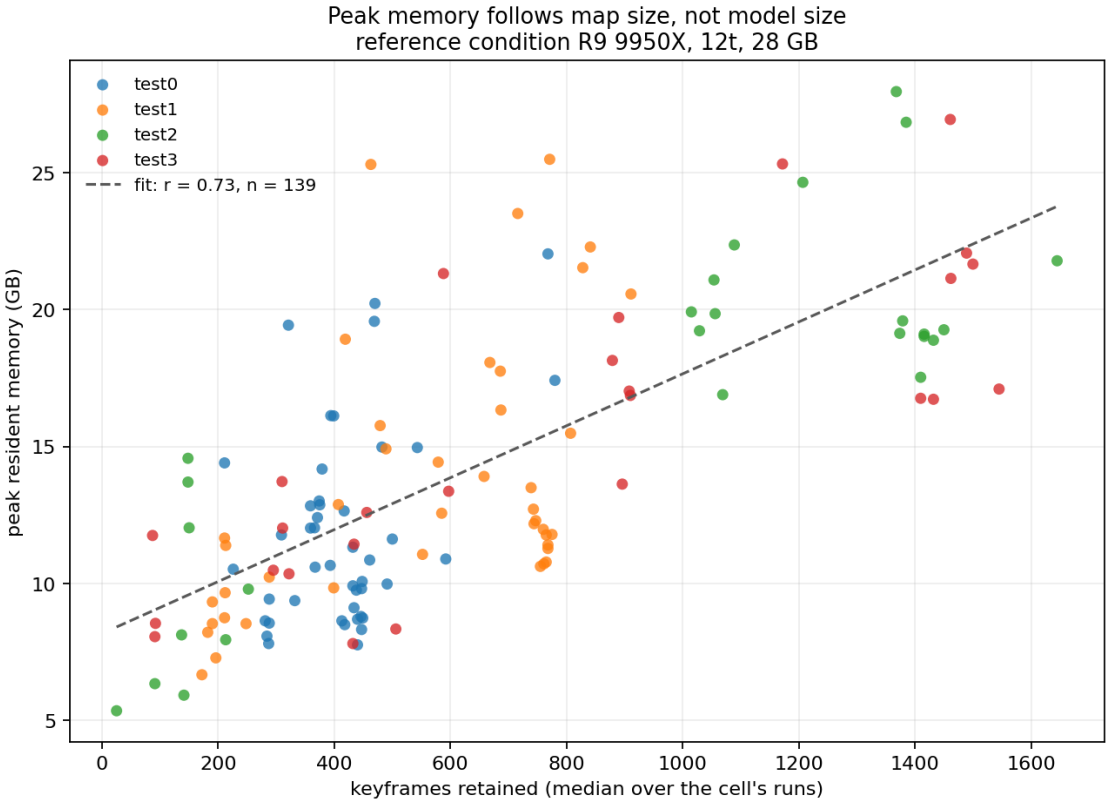


Figure 4.8: Peak memory against map size, one point per cell at `R9-12t-28GB` ($r = 0.78$, $n = 139$). What exhausts memory is the size of the map, not the size of the model.

4.8 What this dataset cannot test

Four limitations bound every conclusion drawn from it, and they are stated here so that Section 9.7 can refer back rather than repeat.

One environment, one season, one time of day. All four sequences are daytime suburban driving in the same city, recorded in the same conditions. Nothing here tests appearance change across seasons, weather or illumination, which is exactly the axis on which the learned place-recognition methods claim their largest advantage. A DINOv2 descriptor that fails to justify its cost on this dataset might well justify it on a multi-session one.

One motion profile. A car on a road turns in a constrained way and never rolls, pitches sharply or moves sideways. Results here should not be read as predicting behaviour on a handheld, aerial or legged platform.

Two of four sequences share a drive. `test0` and `test1` are separate windows from drive `0008`, so they are not fully independent samples of the environment.

Short windows relative to real deployment. The longest sequence is just over five minutes. The memory finding in Chapter 7 is a statement about a five-minute drive; on a route lasting hours, with the same unpruned map growth, the ceiling would be reached far earlier.

CHAPTER 5

Experimental Design

5.1 Protocol

The study follows a single rule: hold the entire pipeline fixed and change exactly one block at a time. Every variant is measured against one common baseline (ORB2 features, DBoW3 for loop closure, and a g2o back end, in stereo) and each phase swaps a single component of that baseline while leaving the rest untouched (Table 5.1). Because only one thing changes, a difference in accuracy or cost is attributable to that change rather than to the scene, the implementation, or interactions between blocks. The baseline is run once per sequence per condition and reused as the reference point for the feature-extraction, loop-closure and back-end phases.

Table 5.1: The five experiment phases and what each holds fixed versus varied. Every phase in the stereo column uses the DBoW3-independent vocabulary, including the baseline.

Phase	**Varied**	**Fixed**	**Mode**
1. Feature extraction	FE method	loop closer, g2o	stereo
2. Loop closure	LC method	ORB2 FE, g2o	stereo
3. Back end	g2o vs. GTSAM	ORB2 FE, DBoW3	stereo
4. Combined	best FE + LC pairs	back end	stereo
5. Depth	FE + depth model	loop closer	mono + neural

5.1.1 The independent vocabulary

One detail of the loop-closure setup matters for fairness. The DBoW3 vocabulary shipped with the platform was trained on ORB descriptors, so only ORB and ORB2 can use it. Every other feature type, and the deep extractors especially, produces incompatible descriptors, so for those the system builds its own vocabulary during the run: the DBoW3-independent setting.

The important point is that this setting is used by *every* variant in the feature-extraction, back-end and depth phases, including the baseline itself. Had the baseline been allowed the cheaper pre-trained vocabulary while the deep extractors paid for an online one, the ratios between them would have measured the vocabulary rather than the front end. Making the baseline pay the same cost keeps the relative numbers comparable, at the price of a baseline that is slower and heavier than a tuned ORB-SLAM2 would be. It is not free in another sense either: the independent vocabulary needs ORB2 descriptors of its own, so every non-ORB front end runs its own network *and* an ORB2 pass on each keyframe. Appendix A explains why, and Section 9.7 states what it does not fix.

5.1.2 How the combined pairs were chosen

The combined phase is the one phase not designed on a principle. The original intention was a full compatibility matrix, but twenty extractors against fourteen closers is 280 configurations, three repetitions each, four sequences, which fits in no run budget I had. The matrix became a sample of eight pairs.

The loop-closure phase already covered one classical front end, ORB2, against the full range of closers. What was missing was the same question asked of a different classical front end and of a deep one. AKAZE was the classical choice, because it had done well on the visual-odometry pass over `test3` and I wanted to know whether the closer rankings carried across a different descriptor set. LightGlue was the deep choice, because it was fast and because bare LightGlue had been dying part way through `test2` and `test3`: if a strong closer can carry a fragile front end to the end of a hard drive, that is the cleanest demonstration of what loop closure is for. Each was paired with four closers spanning the architectural range: OBIndex2, NetVLAD, MegaLoc and SALAD.

I picked these by eye from the two preceding phases rather than by any rule, so Section 6.7 reports the best of an informed selection rather than the best available pairing.

5.1.3 Two passes, and the skip list

The front end can be run as pure visual odometry, with loop closure off and no vocabulary loaded, which is the lightest configuration the pipeline has. On the pilot machine I started there: VO exposes raw drift before global correction hides it, and on 14 GB I needed to know which extractors could finish a sequence at all before committing to the heavier phases. That pass answered the question, and not encouragingly. The remaining phases then ran in full SLAM with nothing pre-emptively skipped, so a failure is a data point rather than a gap.

Cells were skipped only on the pilot machine, and only where the VO pass had shown a guaranteed loss: memory exhausted within the first minutes of a long drive, or throughput so far below real time that the wall-clock limit was unreachable. Thirteen of the twenty extractors and five of the fourteen closers are skipped on the two long sequences on that basis, and the two DepthAnything V2 variants are restricted to `test0`. KAZE is the one method dropped outright: at roughly 0.2 fps it timed out everywhere, so it appears in the taxonomy of Chapter 3 but contributes no data.

The code, the variant definitions and the skip list were carried over unaltered to the current machine, so any difference between conditions is the hardware and not a change of experiment. The cost is that the skip list is a property of the *pilot* machine frozen into all four conditions, and since it is the single largest gap in the design, this is where it is stated in full rather than in pieces.

Two things were lost. First, 23 cells spanning nine variants (KAZE, Key.Net, AlexNet, CosPlace, EigenPlaces, iBoW, NetVLAD, and the two DepthAnything V2 depth pairings) ran on the pilot condition and were never re-run on any R9 condition. Second, 19 of the 45 variants that ran at `R9-12t-28GB` were restricted to `test0` and `test1`, so the two long sequences were attempted by 26 variants rather than 45, and every completion figure for `test2` and `test3` throughout this thesis is computed over that reduced set rather than over the full one.

The reduced set is not a random sample of the full one, and the bias is measurable in what survived: three of five classical front ends reached the long sequences, against two of nine CNN and one of four transformer. The deep families are therefore under-represented exactly where the phase separates. The direction of that bias is not simply pessimistic

for them, though. Every skipped configuration was skipped for failing at 14 GB, so the excluded set is drawn from the tail that performed worst, and re-running it at 28 GB could raise the deep families' completion by an unknown amount but could not lower it. Since Section 8.2 goes on to show that 14 GB verdicts are unreliable in precisely the direction that would exclude a recoverable configuration, some of those exclusions were certainly wrong and this thesis cannot say which.

The honest bound, which every later chapter is written against: pooled family completion figures are lower bounds for the deep families and approximately unbiased for the classical ones. That is why the family-level claim in Section 6.3 is stated as one about which methods survive long drives in this pipeline, rather than as a ranking of architectures. Closing this properly means re-running the skipped set at 28 GB, and it is the first thing a replication should do.

5.1.4 Two ways of reading the same data

The skip list matters to one half of this thesis and not to the other, so the two halves read the data differently. The hardware analysis uses everything: its question is what a change of machine does to a fixed set of configurations, and the runs that died at 14 GB are the evidence, not noise to be cleaned away. It therefore works on the cells present under all four conditions and compares them condition by condition, with failures counted rather than excluded. The software analysis does the opposite: its question is which component is the better choice, and a component that failed for want of memory has not been given a fair test, so those comparisons are made at the full-budget condition alone.

5.2 The hardware ladder

The spine of the thesis is four hardware conditions, each differing from the previous by exactly one factor (Table 5.2).

Table 5.2: The four hardware conditions. Each rung differs from the previous by one factor. Memory figures are the guest allocation rather than the physical install, and the GPU (RTX 4060 Ti, 8 GB) is held constant throughout, which makes it a useful control.

ID	Condition	CPU	`processors`	Memory	Isolates
1	`i7-12t-14GB`	i7-8700K	12	14 GB	baseline
2	`R9-12t-14GB`	R9 9950X	12	14 GB	platform
3	`R9-12t-28GB`	R9 9950X	12	28 GB	capacity
4	`R9-32t-28GB`	R9 9950X	32	28 GB	core count

condition 1 to 2 isolates the platform: microarchitecture, clocks, cache, and memory technology (DDR4 against DDR5). condition 2 to 3 isolates memory capacity. condition 3 to 4 isolates core count.

condition 1 to 2 carries one composite factor I cannot remove. Both machines were given twelve logical processors, but the i7-8700K has six physical cores with two SMT threads each while the 9950X has sixteen, so twelve on the pilot is about six cores' worth of execution resources and twelve on the current machine is up to twelve. No configuration setting separates that from the generational difference, and the pilot machine is no longer available to me. Section 7.3 shows why the confound turns out to be small in practice, and Appendix C describes both machines in full.

5.3 Budget control, run limits and safeguards

Both variable factors are set in one place. The guest's memory allocation and its logical processor count are two keys in the WSL2 configuration file, so each condition is a single edit followed by a restart of the guest, verified from inside it before a batch begins. Swap size and the memory-reclaim policy are held constant across all four conditions, so neither can contribute to a difference between conditions.

Because the experiments run unattended through a batch runner, every run is bounded. The wall-clock limit is twenty minutes, roughly four times the natural duration of the longest sequence, so a run that hits the limit is not unlucky, it is at least four times slower than the camera it is supposed to keep up with. A run is declared lost after ten consecutive tracking failures. A RAM-growth early abort catches swap-thrash deaths that would otherwise masquerade as timeouts. Each run is followed by a three-minute cooldown so the operating system can reclaim memory before the next one starts, and the batch runner sleeps a further sixty seconds between variants. Appendix B documents the safeguards and the quirks of running under WSL2.

5.4 Evaluation metrics

Each run is scored on three things at once: how accurate the trajectory is, what it cost the machine, and whether it finished at all. This section gives the definitions and, just as importantly, what each number hides.

5.4.1 Absolute Trajectory Error

An estimated trajectory and the ground truth are first associated by nearest timestamp, keeping only pairs within 50 ms, then aligned with the closed-form Umeyama solution [50], which finds the rotation R, translation t and scale s minimising $\|x_{\text{ref}} - (sRx_q + t)\|^2$. Alignment is computed with scale throughout, and the recovered s is reported alongside the errors: it says how much larger or smaller the reconstruction is than the real world, so for a stereo run it should sit at 1.0 and any deviation reads as scale drift. This matters most for the monocular-depth runs, where scale is exactly what the network is trusted to supply.

After alignment, the ATE is the per-frame Euclidean residual, summarised by its RMSE [51]:

$$\text{ATE}_{\text{rmse}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\|x_{\text{ref},i} - (sRx_{q,i} + t)\|^2}.$$

ATE measures global consistency, which makes it the metric most sensitive to loop closure: a successful loop pulls the whole trajectory back onto the ground truth and collapses the ATE.

What it hides is threefold. The alignment absorbs any global offset, rotation and scale, so a badly scaled trajectory can still score well once s soaks the error up, which is why s is always read next to the ATE. It is a whole-trajectory average, so one bad excursion in an otherwise clean run and a uniformly mediocre run score identically. And on a partial run it is computed only over the frames that exist, which is the completion trap: dying early on the easy opening stretch produces a small and meaningless ATE. That last property is why accuracy and completion are never combined into a single score anywhere in this thesis.

5.4.2 Relative Pose Error

RPE evaluates local consistency. Over every window of fixed duration Δt it compares the relative motion of the estimate against that of the ground truth and reports the RMSE of the translational and rotational residuals [51]. Two windows are used: $\Delta t = 0.5\,\mathrm{s}$ for short-range jitter and $\Delta t = 2\,\mathrm{s}$ for slow local drift.

Because loop closures barely change relative motion over half a second, short-window RPE reflects mostly the front end's frame-to-frame quality while ATE reflects mostly the loop closer's global correction. That is a tendency rather than a rule, since a front end bad enough to lose tracking destroys both. What RPE hides is the global picture: a trajectory can drift arbitrarily far while keeping every local segment clean, and RPE will not notice.

5.4.3 Resource monitoring

A background thread samples the machine every 0.5 s throughout each run: process-level CPU and RAM via `psutil`, and GPU utilisation, memory and power via NVML. Each run is summarised by the average and maximum of every channel, and the busy-thread counters used in Section 7.3 come from the same sampler. Two caveats. Averages understate the spikes that actually kill runs, since an out-of-memory kill comes from a momentary peak rather than a mean, which is why peak as well as mean memory is reported everywhere. And under WSL2 the memory accounting includes the guest's page-cache behaviour, so RAM figures are best compared between runs on the same setup rather than taken as absolute.

5.5 Outcomes and statistical treatment

Every run receives one of four labels: SUCCESS, LOST (ten consecutive tracking failures), TIMEOUT (twenty-minute wall limit) or OOM (killed by the operating system). The distinction is central to the study. LOST is the algorithm's failure; TIMEOUT and OOM are the machine's. Claims about a component are therefore scored on the *algorithm-attributable* subset, successes over successes plus tracking losses, and claims about hardware on the *budget-attributable* failures, out-of-memory plus timeout. Which group a claim is scored against is stated wherever a completion figure appears. The per-frame progress log survives crashes, so partial metrics are recoverable even from killed runs, which is what makes completion percentages available for failures at all. Section 8.2 explains why the boundary between LOST and OOM is less reliable than it looks.

Three consequences follow for how the numbers are handled. *Survivorship:* accuracy conditioned on completion is biased, since a run that dies early is scored only on the easy stretch it survived, so completion is reported beside every accuracy number and the cells that complete under every condition are analysed separately. *Censoring:* progress at failure is right-censored, and keeping the outcome label together with the completion percentage turns that into structure rather than discarding it, letting a run killed at 95 % still contribute evidence. *Variance:* three repetitions is thin, and in part of the data the standard deviation of ATE exceeds its mean, so I report medians, keep the spread visible, and flag the variants where run-to-run variance makes a single number meaningless rather than averaging over it silently.

5.6 How many runs this comes to

On each of the three current-machine conditions, the full-SLAM matrix covers 45 variants on `test0`, 42 on `test1`, 26 on `test2` and 26 on `test3`. That is 139 cells, each normally run three times:

$$(45 + 42 + 26 + 26) \times 3 = 417 \text{ SLAM runs per condition.}$$

The reference condition carries eight more, for the reason in Section 5.6.1, which takes it to 425. The VO pass covers 19 variants on each of the two shorter sequences and 7 on each of the two longer ones, run once each:

$$(19 + 19 + 7 + 7) \times 1 = 52 \text{ VO runs per condition.}$$

The pilot predates the settled skip list and attempted a wider set: 46 variants on each of the two shorter sequences and 35 on each of the longer ones, plus 20 VO variants on all four. Its SLAM count is not a clean product, because the repetitions there are uneven; it comes to 424 SLAM runs attempted and 80 VO runs. Across the ladder,

$$424 + 417 + 425 + 417 = 1{,}683 \text{ SLAM runs,} \qquad 80 + 3 \times 52 = 236 \text{ VO runs,}$$

for **1,919 runs** in total. Of the cells attempted, 139 SLAM cells and 52 VO cells exist under all four conditions, 191 together, and those form the matched subset behind every condition-to-condition comparison.

5.6.1 Additional remarks regarding run counts

The pilot machine is not balanced. It ran every variant before the skip list had settled, so it carries a handful of extra runs left over from working out which variants were viable, and is missing a handful of others. The missing ones are my own mistake and cannot be filled in, because the machine is no longer available. It does not distort the argument much, since the software comparison reads only the current machine and the hardware comparison reads the 139 cells that exist under all four conditions, but it is a real gap.

The reference condition has five baseline repetitions, not three. All three R9 conditions were balanced at three runs per cell, but at `R9-12t-28GB` the baseline lost one of its three runs on `test3`. Since every accuracy figure in Chapter 6 is read beside the baseline's, I ran two more repetitions there, on all four sequences rather than only on the one that disappointed. It did not change much: `test3` still comes back short, three completed of five with two lost, so that column keeps its dagger throughout.

CHAPTER 6

Results I: Component Comparison

This chapter reports how accurately each interchangeable block reconstructs the trajectory, how often it finishes, and what it costs the machine, with the hardware held fixed at the most capable condition available. Interpretation is deferred to Chapter 9; what follows is what happened.

6.1 The reference condition, and one bias to keep in mind

Every number in this chapter comes from `R9-12t-28GB`, the reference condition, which is the best configuration in the study. Of the 425 full-SLAM runs executed there, four timed out and not one was killed for exhausting memory. A configuration that fails here has therefore failed on its own merits, which is the only setting in which an algorithm comparison is honest.

There is one deliberate exception, flagged where it occurs: the back-end comparison in Section 6.6 reports both 28 GB conditions side by side, because its result is a null one and a null result resting on a single sweep is not worth much.

One bias needs stating before any table is read. pySLAM is an ORB-SLAM2 implementation, and its keyframe policy, relocalisation logic, matcher and default constants were all designed and tuned around ORB features. Forcing the baseline onto the independent vocabulary (Section 5.1.1) levels the vocabulary cost and nothing else. So expect ORB2 and ORB to look good here, and read that partly as a property of the bench: where the baseline wins, it wins *on its home ground*. Where a swapped component beats it anyway, as several do on the hard sequences, the result is the more interesting one for having been achieved against the platform's bias. Section 9.7 bounds how far the concern reaches; it is not repeated in between.

6.2 How to read the tables

What counts as a run. A *run* is one repetition of one variant on one sequence at the reference condition. Every cell is three repetitions, so a variant that attempted all four sequences has twelve runs behind it and a variant the skip list restricted to `test0` and `test1` has six. The baseline is the one exception, at five repetitions per sequence.

Table conventions. Accuracy is the mean ATE over the repetitions that completed, in metres, with that count printed beside it as *completed/attempted*, and the best figure in each column set in bold. A cell reading `lost @`*n*`%` was attempted but never finished, and reports how far it got instead of an error it has no right to report. A "–" means the pair was never attempted, so a row containing one has a completion figure computed over what it ran, which is not comparable with a full row's. A dagger marks a sequence on which the baseline itself did not complete every run, which in this study is `test3`, where it finishes three of five at 3.58 ± 1.78 m; every `test3` comparison inherits that uncertainty.

Why accuracy and completion stay separate. The two are never combined into a score, for the reason given in Section 5.4.1: a run that stopped at 30 % measured its error on a different and easier piece of road.

What each sequence is for. Completion falls from 95.6 % on `test0` to 60.9 % on `test1`, 67.5 % on `test2` and 47.5 % on `test3`. The first two figures are computed over 45 and 42 variants, the last two over 26 (Section 5.1.3), so the apparent rise from `test1` to `test2` is a change in who was attempting rather than a change in difficulty. Cross-sequence completion figures in this chapter are not like-for-like and should not be read as a difficulty curve. `test0` and `test1` are the accuracy lens, since enough methods finish that trajectories can be compared. `test2` and `test3` are the real tests: they are the long drives, and Chapter 4 says why that matters, since length, revisit structure and sharp low-overlap manoeuvres are all concentrated there.

6.3 The architecture axis at family level

Chapter 3 inherited one organising claim from the literature: moving from hand-crafted through CNN to transformer buys robustness. Checking it at the family level sets the expectation everything below is read against.

On accuracy it is not visible. On `test0`, where enough of every family finishes to compare, the three families land within a few centimetres of each other. *On completion the classical family leads and the axis does not order the other two*: across the feature-extraction phase, 83 % of classical runs finish against 67 % for transformer and 56 % for CNN. That pooled figure needs one qualification before it is read as an architectural result, because the families did not face the same sequences. Restricted to `test0` and `test1`, which every front end attempted, classical and transformer are level at 83.3 % and CNN trails at 66.7 %. Restricted to `test2` and `test3` the separation is absolute (83.3 % classical, 8.3 % CNN, 0 % transformer), but only 26 of the 45 variants were attempted there and the skip list that chose them is biased (Section 5.1.3). The ordering below is therefore a statement about which methods survive long drives in this pipeline, not a clean ranking of architectures.

Figure 6.1 makes the accuracy point in a way the ATE tables cannot. Short-window RPE, which measures frame-to-frame motion quality before any global correction, is essentially identical across the three families on every sequence: 6.07, 6.07 and 6.10 m for classical, CNN and transformer respectively on `test0`, and flat to within a few centimetres elsewhere. Whatever separates these families, it is not their ability to estimate local motion. They differ in global consistency and in survival, and Section 6.4 shows the survival gap is a wall rather than a gradient.

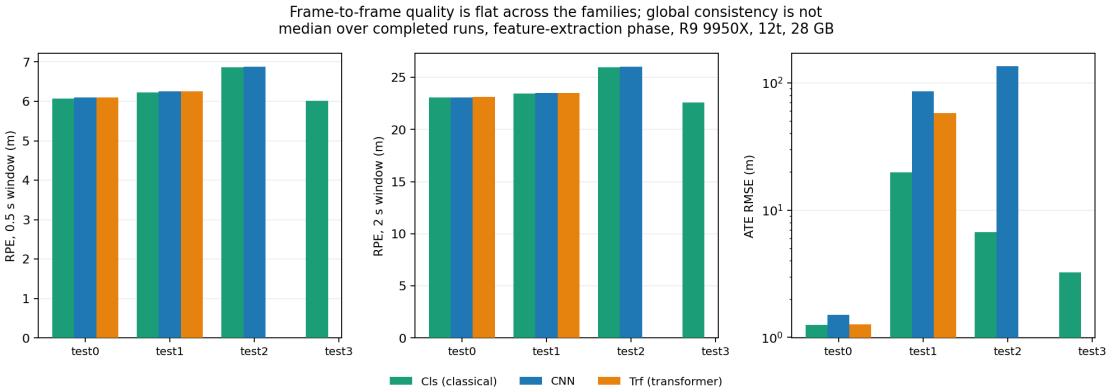


Figure 6.1: Local motion estimation does not separate the families. Short-window RPE (left) is flat across all three front-end families on every sequence; ATE (right) is not.

A second difference shows up on an axis nobody reports, and it does not follow the architecture. Figure 6.2 counts the frames each front end consumes before the map initialises and the first pose appears. Every family has a median of one, so the family medians say nothing; the split is within them. Thirteen of the nineteen configurations bootstrap immediately and six do not, and that six contains a member of each family. D2Net is the extreme, consuming 701 frames on `test1`, more than a minute of driving, identically in every repetition and at every hardware condition, so this is a deterministic property of the detector on that scene rather than a slow start. The baseline is in the same group at 46 and 56 frames. On a five-minute sequence that is a curiosity; on a robot that must localise before it can move it is a specification, and not one you can read off a method's family.

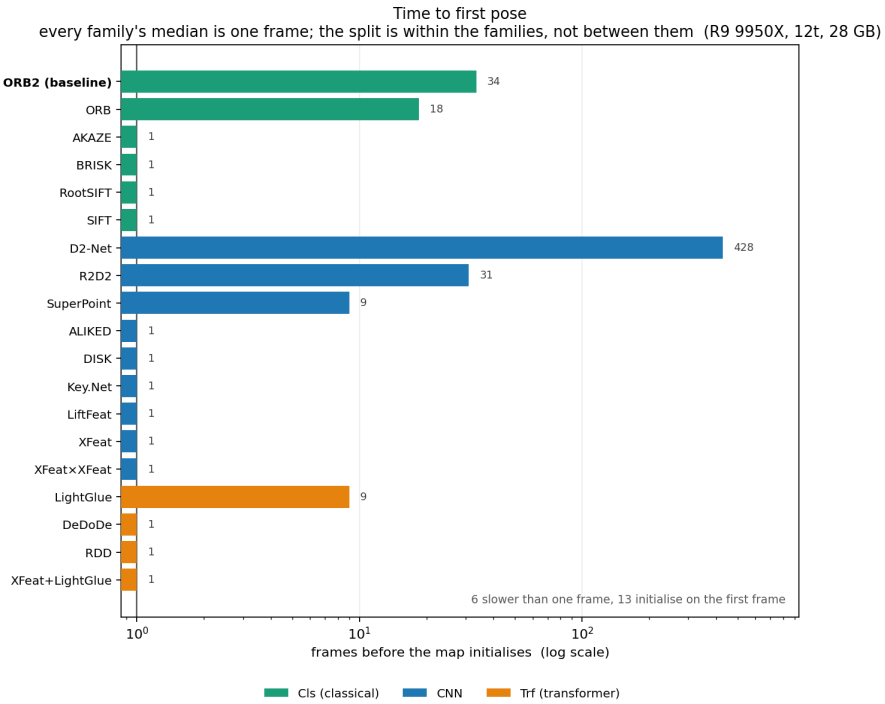


Figure 6.2: Frames required for map initialisation at `R9-12t-28GB`, by architectural family. Every family has a median of one frame; the six slow configurations span all three.

6.4 Feature extraction ①

6.4.1 Accuracy and completion

Table 6.1: Feature extraction at the reference condition. Rows run baseline, Cls, CNN, Trf. "Compl." is completed runs over attempts. Accuracy is the mean ATE over completed repetitions, in metres, with that count beside it; best per column in bold. A cell that never finished reports how far it got, a dash was never attempted, and † marks a sequence the baseline did not itself complete.

Method	fam.	compl.	test0	test1	test2	test3
ORB2 (baseline)	Cls	18/20	1.29 (5/5)	7.70 (5/5)	**2.74 (5/5)**	**3.58 (3/5)**†
BRISK	Cls	12/12	1.23 (3/3)	17.27 (3/3)	3.13 (3/3)	4.18 (3/3)†
RootSIFT	Cls	6/6	1.23 (3/3)	22.24 (3/3)	–	–
SIFT	Cls	6/6	1.29 (3/3)	29.88 (3/3)	–	–
AKAZE	Cls	9/12	1.23 (3/3)	lost @31%	35.08 (3/3)	9.57 (3/3)†
ORB	Cls	7/12	**1.18 (3/3)**	14.26 (1/3)	11.44 (3/3)	lost @25%
D2-Net	CNN	6/6	1.58 (3/3)	**1.42 (3/3)**	–	–
XFeat	CNN	6/6	1.76 (3/3)	71.19 (3/3)	–	–
DISK	CNN	6/6	1.45 (3/3)	89.09 (3/3)	–	–
LiftFeat	CNN	4/6	2.42 (3/3)	96.03 (1/3)	–	–
XFeat×XFeat	CNN	4/6	1.64 (3/3)	26.42 (1/3)	–	–
Key.Net	CNN	4/6	1.27 (3/3)	96.29 (1/3)	–	–
ALIKED	CNN	4/12	1.19 (3/3)	lost @60%	134.86 (1/3)	lost @42%
SuperPoint	CNN	3/12	1.27 (3/3)	lost @28%	lost @27%	lost @32%
R2D2	CNN	0/6	lost @76%	lost @28%	–	–
DeDoDe	Trf	6/6	1.33 (3/3)	39.80 (3/3)	–	–
XFeat+LightGlue	Trf	6/6	2.19 (3/3)	55.37 (3/3)	–	–
RDD	Trf	5/6	1.25 (3/3)	95.59 (2/3)	–	–
LightGlue	Trf	3/12	1.29 (3/3)	lost @29%	lost @15%	lost @31%

Table 6.1 gives the front-end results, and three things stand out.

On the easy sequence the choice barely matters. On `test0` twelve of the seventeen extractors that finish land between 1.18 and 1.45 m. ORB is nominally best at 1.18 m and ALIKED next at 1.19 m, a difference of one centimetre and well inside the run-to-run spread. Only D2-Net, the XFeat family and LiftFeat trail, at 1.6 to 2.4 m. When the sequence is friendly, the cheapest thing that finishes is the right choice, and any ranking published on a sequence like this one is measuring noise.

On the long drives the ranking becomes a completion ranking, and it inverts the architectural axis. On `test3`, **not one deep extractor completes a single run**: ALIKED, SuperPoint and LightGlue record 0/9 between them, and the other ten deep front ends were removed by the skip list before the long sequences. What this establishes is that no deep front end *survived to and through* `test3`, which is weaker than a clean thirteen-way comparison and stronger than it sounds, since the ten were excluded for failing earlier. What finishes is BRISK (3/3), AKAZE (3/3) and the baseline (3/5). On `test2` the picture is only slightly softer, with ALIKED scraping one run of three. Across all four sequences BRISK is the only swapped front end to complete every one of its twelve runs, AKAZE reaches 9/12 and ORB 7/12, while ALIKED manages 4/12, SuperPoint and LightGlue 3/12, and R2D2 nothing at all.

Figure 6.3 shows what happened to each of those runs, and the answer is almost uniformly tracking loss rather than a resource death, which is what makes this an algorithmic ordering rather than a memory one.

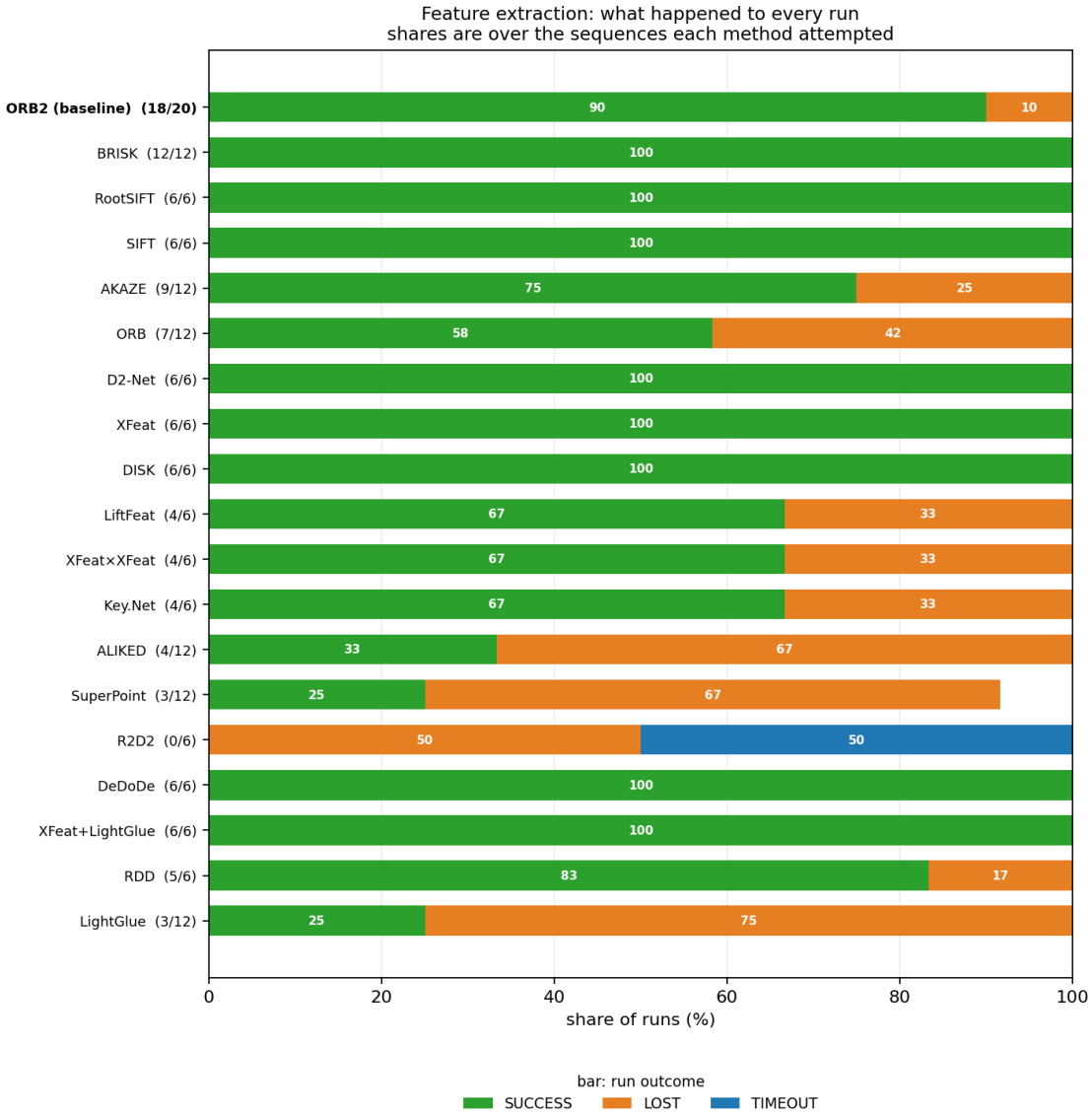


Figure 6.3: Feature extraction: the outcome of every run at the reference condition. Bar lengths differ because a skip-list-restricted variant contributes six runs rather than twelve.

Completing is not the same as being right. Several deep extractors finish `test1` with errors between 55 and 96 m: DISK at 89.1 m, Key.Net at 96.3 m, LiftFeat at 96.0 m, RDD at 95.6 m, XFeat at 71.2 m. These are complete trajectories that have drifted the width of several city blocks. The clearest single case is AKAZE on `test2`: it completes all three runs, so on the completion column it is indistinguishable from the baseline, and it posts 35.08 ± 40.31 m against the baseline's 2.74 ± 0.19 m. The standard deviation exceeding the mean is the giveaway. This is not a method consistently thirty-five metres out; two of its three runs land at 11.4 and 12.2 m and the third is far worse, so the mean averages over two quite different outcomes. Figure 6.4 shows where it breaks: loop closure fails to trigger during one of the hardest turns on `test2`, the manoeuvre identified in Figure 4.4. ORB does something milder in the same place, finishing 3/3 at 11.4 m.

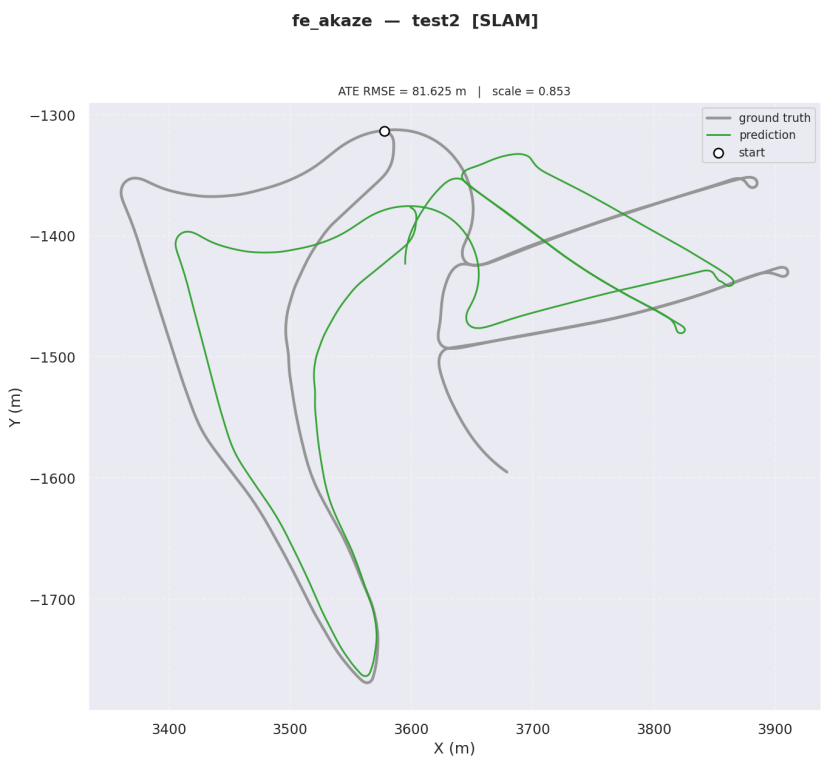


Figure 6.4: AKAZE on `test2`: three completed runs, mean 35.08 m with a standard deviation of 40.31 m. A finished drive is not a correct one, and the spread between repetitions says the outcome hinges on a single stretch of road rather than on accumulated drift.

There is one clean counter-example in this phase and it deserves its due. D2-Net posts 1.42 ± 0.18 m on `test1`, the best figure any front end achieves on that sequence and far better than the baseline's 7.70 ± 7.53 m. It only ever attempted the two short sequences, so it is untested where the phase separates, but on the ground it was given it beat the platform's own front end on the platform's own terms.

6.4.2 What each front end costs

Table 6.2: Machine cost of each feature extractor at the reference condition, absolute with the ratio to the ORB2 baseline in brackets. Rows run baseline, Cls, CNN, Trf; best per column in bold. CPU/frame is mean CPU load divided by throughput: a pipeline stalled on the GPU posts a low instantaneous CPU per cent while doing less work, and this column removes that illusion.

Method	fam.	GPU % (rel)	CPU % (rel)	fps (rel)	peak GB	CPU/frame
ORB2 (baseline)	Cls	5.3 (1.00×)	36.9 (1.00×)	**13.8 (1.00×)**	19.6	2.67
ORB	Cls	**4.6 (0.87×)**	26.1 (0.71×)	7.1 (0.51×)	21.8	3.70
RootSIFT	Cls	5.0 (0.96×)	44.8 (1.19×)	7.5 (0.54×)	18.1	5.93
SIFT	Cls	5.2 (1.00×)	45.4 (1.21×)	7.9 (0.57×)	17.8	5.74
BRISK	Cls	5.3 (1.01×)	30.4 (0.82×)	8.7 (0.62×)	25.3	3.52
AKAZE	Cls	6.1 (1.14×)	39.6 (1.07×)	10.4 (0.75×)	19.9	3.82
XFeat	CNN	9.4 (1.79×)	28.6 (0.76×)	7.6 (0.55×)	20.6	3.76
LiftFeat	CNN	10.1 (1.92×)	34.1 (0.91×)	3.2 (0.23×)	25.3	10.62
XFeat×XFeat	CNN	11.0 (2.10×)	21.6 (0.57×)	7.4 (0.53×)	21.5	2.93
R2D2	CNN	16.1 (3.09×)	**13.5 (0.36×)**	0.6 (0.04×)	15.0	22.91
ALIKED	CNN	24.1 (4.54×)	21.6 (0.58×)	8.9 (0.64×)	12.0	**2.43**
SuperPoint	CNN	33.2 (6.26×)	22.2 (0.60×)	9.1 (0.66×)	**10.7**	2.44
D2-Net	CNN	46.5 (8.90×)	29.2 (0.78×)	1.7 (0.12×)	22.3	17.50
Key.Net	CNN	52.8 (10.12×)	17.1 (0.45×)	2.2 (0.16×)	11.8	7.62
DISK	CNN	91.2 (17.46×)	14.0 (0.37×)	2.6 (0.19×)	15.8	5.39
XFeat+LightGlue	Trf	16.0 (3.07×)	18.9 (0.50×)	3.9 (0.28×)	23.5	4.82
LightGlue	Trf	30.9 (5.83×)	18.5 (0.50×)	6.4 (0.46×)	12.8	2.91
RDD	Trf	71.3 (13.67×)	17.7 (0.47×)	3.1 (0.22×)	18.9	5.77
DeDoDe	Trf	88.2 (16.88×)	24.4 (0.65×)	2.5 (0.18×)	25.5	9.91

The front-end cost table contains this chapter's most easily misread column, which is why it carries a fifth. Raw CPU per cent *falls* as the methods get deeper: the classical extractors sit near or above the baseline, the deep ones at 0.4 to 0.8 times it. Read naively that says the learned front ends hand the processor back for other work. They do not. They leave it idle waiting on the GPU while the pipeline runs at a fraction of the speed, and instantaneous CPU per cent is a rate that rewards exactly that. The last column divides CPU load by throughput to give work per frame, and on that measure the ordering reverses: D2-Net spends 17.5 units per frame against BRISK's 3.5.

That reframes the trade this thesis is measuring. The learned components do not simply cost more; they *move* the cost from one processor to the other, and whether that is a good trade depends on what else the platform has to do. On a workstation running SLAM alone, offloading the front end to an idle GPU is nearly free. On an edge robot the GPU is usually the contested resource, shared with detection, segmentation or a second camera, while cores sit idle, and the trade runs backwards.

6.5 Loop closure ②

Every run in this phase shares the ORB2 front end, so only the place-recognition module varies.

6.5.1 Accuracy and completion

Table 6.3: Loop closure at the reference condition, every run sharing the ORB2 front end so only the place-recognition module varies. Rows and columns as in Table 6.1.

Method	fam.	compl.	test0	test1	test2	test3
ORB2 (baseline)	Cls	18/20	1.29 (5/5)	7.70 (5/5)	2.74 (5/5)	3.58 (3/5)†
DBoW2	Cls	12/12	1.26 (3/3)	12.13 (3/3)	2.57 (3/3)	**2.52 (3/3)**†
OBIndex2	Cls	12/12	1.28 (3/3)	17.40 (3/3)	**2.42 (3/3)**	2.73 (3/3)†
VLAD	Cls	11/12	1.37 (3/3)	21.28 (3/3)	3.22 (3/3)	11.03 (2/3)†
iBoW	Cls	5/6	1.27 (3/3)	16.29 (2/3)	–	–
SAD	Cls	9/12	1.24 (3/3)	21.95 (3/3)	2.84 (3/3)	lost @31%
AlexNet	CNN	3/3	1.26 (3/3)	–	–	–
CosPlace	CNN	5/6	1.29 (3/3)	25.58 (2/3)	–	–
EigenPlaces	CNN	5/6	1.28 (3/3)	6.32 (2/3)	–	–
NetVLAD	CNN	5/6	1.30 (3/3)	26.43 (2/3)	–	–
CricaVPR	Trf	10/12	1.32 (3/3)	**5.55 (3/3)**	2.99 (3/3)	9.07 (1/3)†
MegaLoc	Trf	10/12	1.35 (3/3)	17.73 (3/3)	2.96 (3/3)	9.39 (1/3)†
SALAD	Trf	9/12	1.29 (3/3)	43.94 (1/3)	2.74 (3/3)	21.24 (2/3)†
SuperVLAD	Trf	7/12	**1.24 (3/3)**	lost @60%	2.84 (3/3)	7.00 (1/3)†

On `test0` the loop closer is irrelevant: all fourteen methods land within 1.24 to 1.37 m, because a drive that barely drifts leaves nothing to fix. The spread opens on the sequences with real drift, and it opens in an order the architecture does not predict.

On `test3`, the hardest drive, the two best corrections in the phase are DBoW2 at 2.52 m and OBIndex2 at 2.73 m, both completing three runs of three, and both beating the baseline's own DBoW3 at 3.58 m. These are a 2012 bag of binary words and its incremental successor, winning on the platform's home ground against the closer it ships with. Every deep closer that reaches the end of that sequence lands between 9.1 and 21.2 m, and none completes more than two runs of three. On `test2` the pattern is gentler but the ordering holds: everything that completes lands between 2.42 and 3.22 m, with OBIndex2 in front and the classical group taking three of the top four places.

`test1` inverts it. CricaVPR reaches 5.55 m across three completed runs and EigenPlaces 6.32 m on two of three, against 12 to 26 m for most of the classical field. The baseline's DBoW3 posts 7.70 ± 7.53 m over five runs, so both DINOv2 closers beat the shipped closer outright on that sequence, and CricaVPR does so while being the more consistent of the two: its spread is a fraction of the baseline's, which swings by more than seven metres between repetitions of identical code.

This is the strongest evidence in the study for deep place recognition, and Section 4.6 explains why it appears here and nowhere else. `test1` is the sequence where the vehicle revisits ground from a materially different heading, so recognising a place requires a descriptor that survives viewpoint change rather than one that matches the same corner from the same angle. That is exactly the property a DINOv2 backbone has and a bag of binary words does not. On the three sequences where revisits are closer to head-on, the property is not needed and the cheap closers take it back. Figure 6.5 shows all four orderings at once, and a method's rank on one panel predicts very little about its rank

on the next: a pooled table would average CricaVPR's win on `test1` against its losses elsewhere and report a mediocre closer, which is precisely the wrong conclusion.

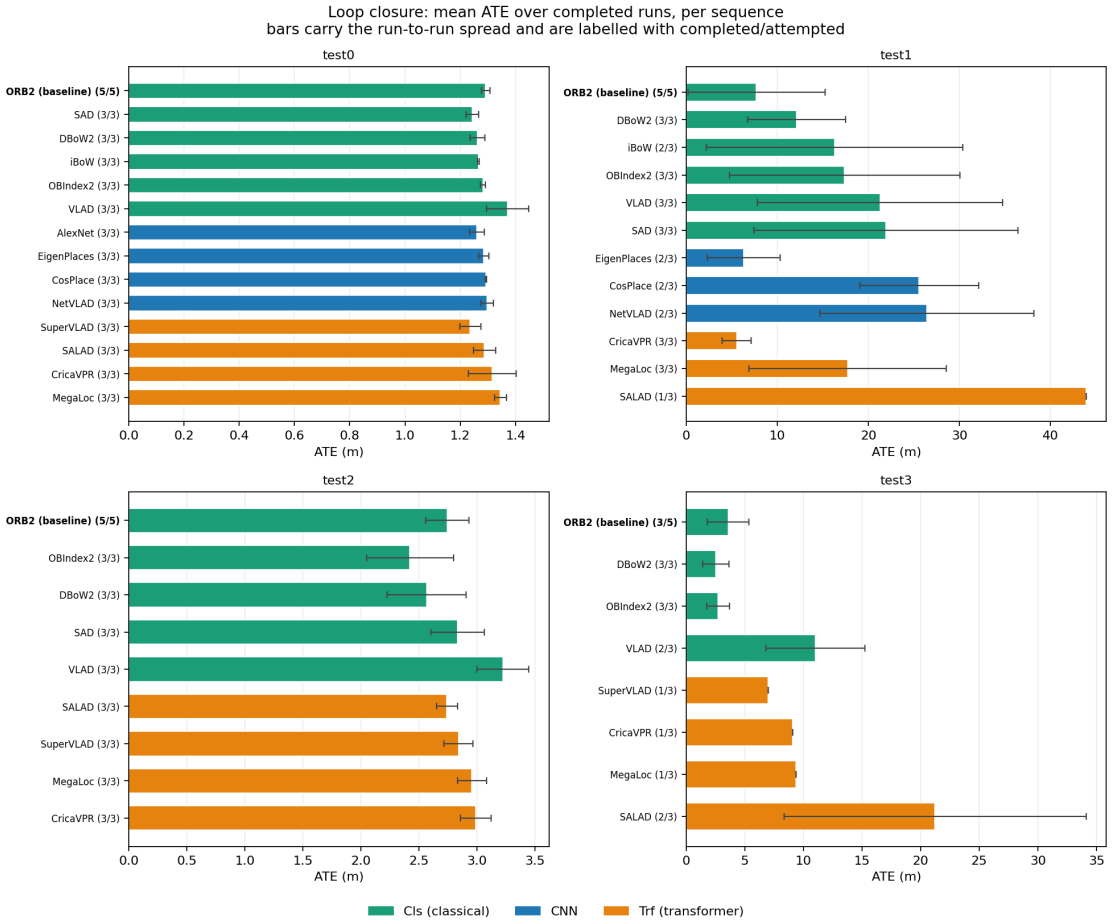


Figure 6.5: Loop closure: mean ATE over completed runs, per sequence. Bars carry the run-to-run spread and are labelled with completed over attempted. The ordering is different in all four panels, which is the argument against reporting any of this as a single pooled number.

Figure 6.6 shows the outcome side of the same phase, and it contains a result Chapter 8 returns to: a loop closer changes whether the *front end* finishes. An identical ORB2 tracker fails a sequence under one closer and completes it three times of three under another, with the tracking code unchanged between them.

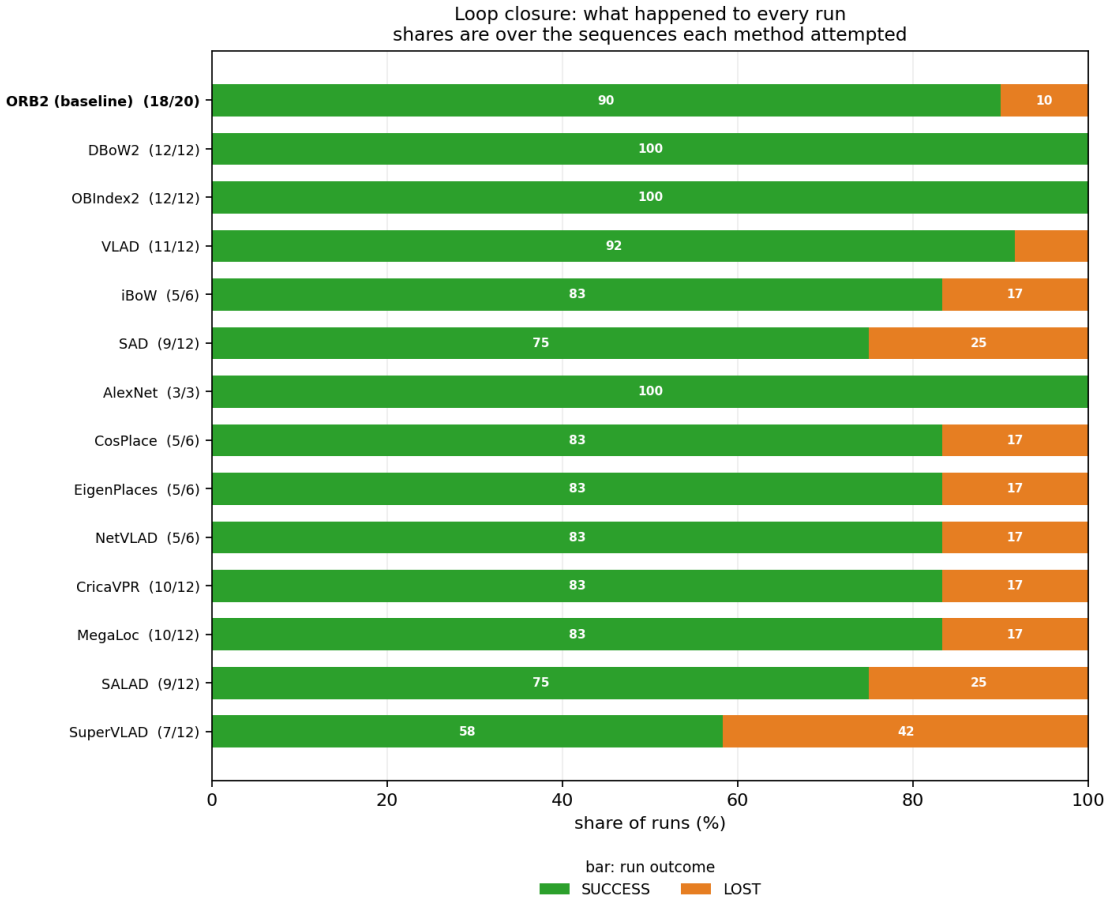


Figure 6.6: Loop closure: what happened to every run. Bar colour is the run outcome; the colour of each method name is its architecture family. As in the front-end phase, essentially all of the failure is tracking loss rather than a resource death.

6.5.2 What each loop closer costs

Table 6.4: Machine cost of each loop closer. Rows and columns as in Table 6.2.

Method	fam.	GPU % (rel)	CPU % (rel)	fps (rel)	peak GB	CPU/frame
ORB2 (baseline)	Cls	**5.3 (1.00×)**	36.9 (1.00×)	13.8 (1.00×)	19.6	2.67
DBoW2	Cls	6.6 (1.25×)	36.3 (0.98×)	13.7 (1.00×)	21.1	2.64
OBIndex2	Cls	6.7 (1.26×)	42.4 (1.15×)	13.6 (0.99×)	21.7	3.11
SAD	Cls	6.7 (1.27×)	35.3 (0.96×)	14.4 (1.05×)	19.1	2.45
VLAD	Cls	6.7 (1.28×)	36.0 (0.97×)	14.8 (1.08×)	19.3	2.43
iBoW	Cls	6.8 (1.30×)	41.4 (1.10×)	14.5 (1.04×)	13.5	2.86
AlexNet	CNN	6.2 (1.23×)	95.6 (2.55×)	9.9 (0.71×)	**9.8**	9.61
NetVLAD	CNN	27.7 (5.29×)	57.7 (1.53×)	13.9 (1.00×)	11.4	4.14
EigenPlaces	CNN	46.7 (8.86×)	58.2 (1.55×)	14.1 (1.01×)	10.7	4.13
CosPlace	CNN	51.5 (9.81×)	57.6 (1.53×)	14.4 (1.03×)	10.8	4.00
SuperVLAD	Trf	6.5 (1.24×)	**32.1 (0.87×)**	12.6 (0.94×)	28.0	2.56
CricaVPR	Trf	6.8 (1.29×)	34.5 (0.94×)	15.0 (1.08×)	17.5	2.30
SALAD	Trf	6.8 (1.29×)	33.9 (0.92×)	14.1 (1.04×)	26.8	2.40
MegaLoc	Trf	6.8 (1.29×)	34.4 (0.93×)	**15.4 (1.11×)**	18.9	**2.24**

The loop-closure costs invert the ordering the architecture predicts, and not in the direction of the front-end result above. The classical closers are effectively free, as

expected, because they reuse descriptors the tracker has already computed. But so are the DINOv2 transformers: CricaVPR, MegaLoc, SALAD and SuperVLAD all measure within three per cent of the baseline's GPU load. It is the mid-weight CNN descriptors that are expensive, at 4.1 times baseline for NetVLAD, 7.0 for EigenPlaces and 7.7 for CosPlace. The likely explanation is architectural rather than algorithmic: loop closing runs in its own process, and how much of a foundation model's forward pass the monitor attributes to the main process depends on how that model is invoked (Appendix A). The measurement is reported as taken, with the caveat that the comparison between the two deep families is less trustworthy here than the comparison between deep and classical.

6.6 Back end ③

The back-end phase is the one place in this chapter that reads *two* hardware conditions rather than one. Table 6.5 and Figure 6.7 both report the reference condition and the thirty-two-thread condition side by side, so that a null result does not rest on a single sweep. Since Section 7.4 shows accuracy to be independent of the machine, pooling the two costs nothing and doubles the evidence behind a claim that two solvers are indistinguishable.

Table 6.5: Back-end solver comparison on an otherwise identical ORB2 pipeline, at both 28 GB conditions. ATE is the mean ± standard deviation over completed runs, with completed over attempted beside it. The standard deviations are kept here because the whole claim of this phase is that the two solvers do not separate, and that cannot be judged without them.

Condition	Seq.	g2o (baseline)	GTSAM
R9/28	`test0`	1.29±0.02 (5/5)	1.31±0.05 (3/3)
	`test1`	7.70±7.53 (5/5)	7.32±1.21 (3/3)
	`test2`	2.74±0.19 (5/5)	3.06±0.00 (1/3)
	`test3`	3.58±1.78 (3/5)	5.29±0.21 (2/3)
R9/28/32t	`test0`	1.28±0.06 (3/3)	1.27±0.06 (3/3)
	`test1`	17.49±19.68 (3/3)	19.11±11.93 (3/3)
	`test2`	2.61±0.27 (3/3)	2.87±0.26 (2/3)
	`test3`	1.86±0.13 (3/3)	1.99±0.26 (3/3)

The two solvers are a wash. They agree to within two centimetres on `test0` (1.29±0.02 against 1.31 ± 0.05) and to within the repetition spread on `test2` and `test3`. On `test1`, which on three repetitions had looked like the one sequence where they separated, g2o posts 7.70 ± 7.53 m against GTSAM's 7.32 ± 1.21 m: the means are within half a metre and g2o's error bar swallows GTSAM's entirely. The apparent difference was a sampling artefact of a thin baseline, not a property of the solvers. Completion differs by two runs out of twelve in g2o's favour at the reference condition and by one at thirty-two threads, which is the size of the repetition noise.

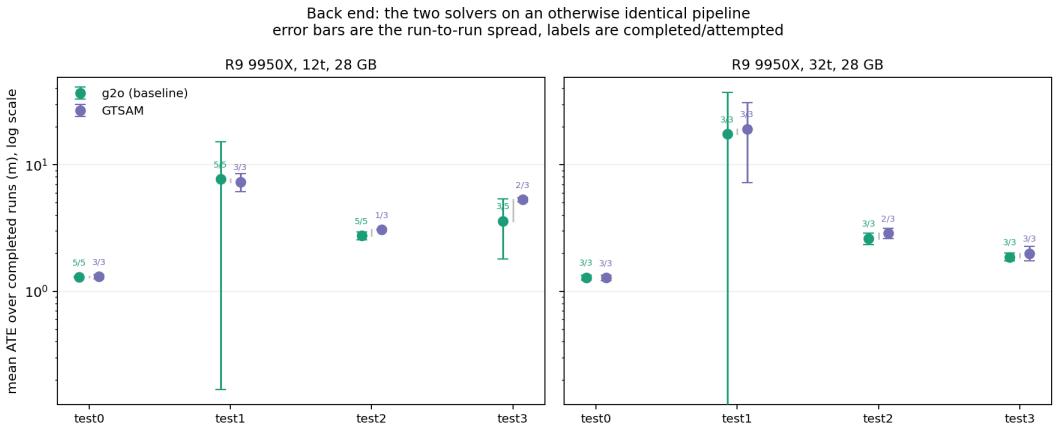


Figure 6.7: Back end: the two solvers on an otherwise identical pipeline, at both 28 GB conditions. Error bars are the run-to-run spread, because a null result has to be shown with its uncertainty or it looks like a comparison that was never run.

6.7 Combined configurations ① + ②

Section 5.1.2 explains where these eight pairs came from, so what follows is the best of an informed selection and nothing stronger. The question the phase was built to ask is whether the ranking of loop closers from Section 6.5, established on the ORB2 front end, survives being bolted to a different classical front end and to a deep one.

Table 6.6: Combined front-end and loop-closer pairs at the reference condition, with the plain ORB2 baseline and BRISK, the only front end from Table 6.1 to complete every sequence on its own, carried in for comparison. Rows and columns as in Table 6.1, apart from the family column, dropped here since every row but the reference is a front-end/loop-closer pair rather than a single family.

Configuration	compl.	test0	test1	test2	test3
ORB2 (baseline)	18/20	1.29 (5/5)	7.70 (5/5)	**2.74 (5/5)**	**3.58 (3/5)**†
AKAZE + SALAD	12/12	**1.23 (3/3)**	62.23 (3/3)	125.61 (3/3)	28.34 (3/3)†
AKAZE + NetVLAD	12/12	1.24 (3/3)	96.30 (3/3)	88.39 (3/3)	13.73 (3/3)†
AKAZE + MegaLoc	10/12	1.24 (3/3)	51.87 (2/3)	48.89 (3/3)	97.44 (2/3)†
AKAZE + OBIndex2	10/12	1.24 (3/3)	34.94 (1/3)	4.97 (3/3)	23.34 (3/3)†
LightGlue + MegaLoc	4/12	1.28 (3/3)	lost @28%	3.83 (1/3)	lost @17%
LightGlue + NetVLAD	4/12	1.28 (3/3)	lost @28%	4.04 (1/3)	lost @25%
LightGlue + SALAD	4/12	1.27 (3/3)	**3.26 (1/3)**	lost @12%	lost @17%
LightGlue + OBIndex2	3/12	1.32 (3/3)	lost @64%	lost @18%	lost @40%
for reference, from other phases					
BRISK	12/12	1.23 (3/3)	17.27 (3/3)	3.13 (3/3)	4.18 (3/3)†

Table 6.6 gives the results, with the plain ORB2 baseline and the best single loop closer from the previous phase carried in for comparison, and Figure 6.9 splits completion from accuracy.

The completion split is total, and it runs along the front end. Every AKAZE pair completes ten or twelve of twelve; every LightGlue pair completes three or four. The choice of closer moves completion by two runs; the choice of front end moves it by eight.

The LightGlue rows answer the phase's question with a flat no. Bare LightGlue, with no loop closure at all, dies at 29.5 % of `test1`, 15.3 % of `test2` and 30.5 % of `test3`, and bolting a closer on barely moves those stopping points. The same front end fails in the same stretches of road whether or not there is a closer behind it, which is what you would expect if the failure happens before any keyframe worth recognising has been produced. A closer cannot correct drift on a trajectory that has already stopped, and it cannot recognise a place the tracker never reached.

A deep closer can rescue completion, and charge for it in accuracy. The AKAZE rows contain the counter-example, and it is not a happy one. Bare AKAZE completes none of its three `test1` runs, stopping at 31.4 %. AKAZE with NetVLAD and AKAZE with SALAD both complete all three, and post 96.3 m and 62.2 m on that sequence. The closer genuinely did carry the run to the end of the drive. It carried it there with a map wrong by the width of a neighbourhood.

The accuracy column is a warning rather than a victory. AKAZE + NetVLAD and AKAZE + SALAD complete every single run and then post 88.4 m and 125.6 m on `test2`, almost certainly closing wrong or late loops on a front end whose descriptors the closer's geometric verification could not confirm. The spread is the giveaway again: AKAZE + NetVLAD's three `test2` runs have a standard deviation of 75 m, which is not a measurement but a coin flip. AKAZE + OBIndex2 is the only pair that is both mostly complete and reasonably accurate, at 4.97 m on `test2`, and even it drifts to 23.3 m on `test3`.

The same failure point, seen from the other side. AKAZE + MegaLoc on `test3` completes two runs of three and posts 97.44 ± 2.45 m, and the small spread is the interesting part: unlike the `test2` cases, this is not a configuration whose outcome hinges on a coin flip. Both completed runs fail in the same way, by the same margin, every time. Figure 6.8 shows why. The failure point is the two-point turn of Figure 4.5, where the vehicle reverses rather than rotating cleanly through the corner. The estimated path tracks the drive well until that point and leaves the road there, and with no closure firing afterwards nothing pulls it back. Swapping in a DINOv2 closer that outperforms every classical alternative on `test1` changes nothing here, because the problem is not which places the closer can recognise but that it is never given a keyframe pair to recognise them from.

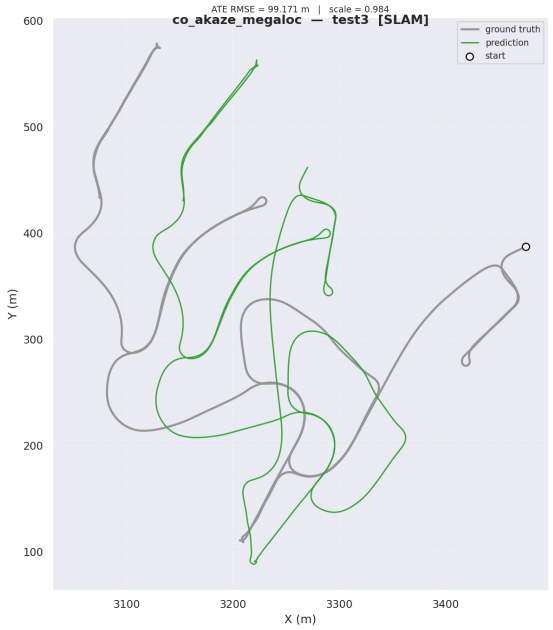


Figure 6.8: AKAZE + MegaLoc on `test3`: two completed runs, 97.44 ± 2.45 m. The divergence begins at the two-point turn. The small spread between repetitions says this is a reproducible structural failure rather than the one time error seen on `test2`.

The default beats everything this phase produced. Plain ORB2 with DBoW2 completes twelve runs of twelve at 2.52 m on `test3` and 2.57 m on `test2`. The best pair this phase produced is beaten, on both axes, by the baseline front end with a fourteen-year-old vocabulary bolted on. The caveat of Section 6.1 applies with full force, since the baseline is being compared on its home ground; but the closer ranking from Section 6.5 did *not* carry over to a different front end, and that part of the answer is not a platform artefact.

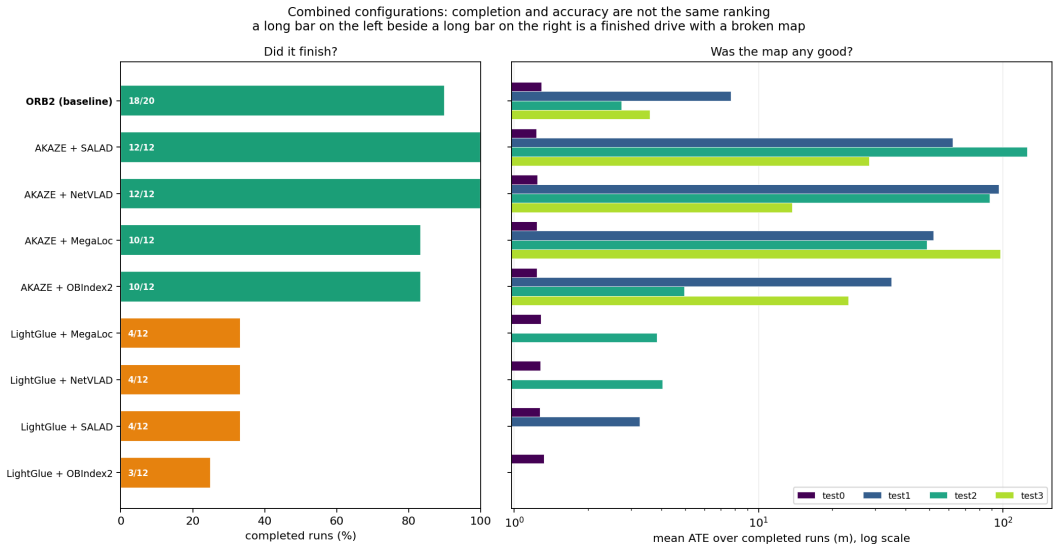


Figure 6.9: Combined configurations: completion and accuracy are not the same ranking. A long bar on the left beside a long bar on the right is a finished drive with a broken map.

Table 6.7: Machine cost of the combined pairs. Rows and columns as in Table 6.2, apart from the family column, dropped here since every row is a front-end/loop-closer pair rather than a single family.

Configuration	GPU % (rel)	CPU % (rel)	fps (rel)	peak GB	CPU/frame
ORB2 (baseline)	**5.3 (1.00×)**	36.9 (1.00×)	**13.8 (1.00×)**	19.6	2.67
AKAZE + OBIndex2	6.0 (1.13×)	41.8 (1.13×)	10.3 (0.76×)	19.2	4.05
AKAZE + MegaLoc	6.0 (1.14×)	39.0 (1.06×)	10.7 (0.79×)	19.9	3.63
AKAZE + SALAD	6.1 (1.15×)	37.2 (1.01×)	10.6 (0.77×)	21.1	3.52
AKAZE + NetVLAD	15.8 (2.97×)	49.5 (1.34×)	10.3 (0.76×)	22.4	4.79
LightGlue + OBIndex2	28.6 (5.40×)	18.8 (0.51×)	5.8 (0.41×)	13.7	3.26
LightGlue + SALAD	31.5 (5.95×)	**17.3 (0.47×)**	6.5 (0.47×)	**12.0**	**2.67**
LightGlue + MegaLoc	31.5 (5.96×)	17.6 (0.48×)	6.5 (0.47×)	13.7	2.71
LightGlue + NetVLAD	40.2 (7.58×)	28.1 (0.76×)	6.5 (0.47×)	14.6	4.29

6.8 Depth estimation ④

The depth phase replaces the right camera with a monocular stream and a neural depth network, feeding predicted depth to the tracker as a pseudo-RGB-D input.

Table 6.8: Monocular plus neural depth at the reference condition, against the stereo baseline. "DA" is DepthAnything. "Scale" is the similarity scale recovered by Umeyama alignment: 1.0 means the reconstruction is metrically correct, and a value below 1 means the map is too small by that factor before the alignment absorbs it.

Configuration	compl.	scale	`test0`	`test1`	`test2`	`test3`
ORB2 (baseline)	18/20	0.98	**1.29 (5/5)**	**7.70 (5/5)**	**2.74 (5/5)**	**3.58 (3/5)**†
ORB2 + DA V2	3/3	1.01	10.17 (3/3)	–	–	–
ORB2 + DA V3	9/12	0.75	8.30 (3/3)	27.15 (3/3)	lost @8%	20.08 (3/3)†
SuperPoint + DA V2	2/3	1.03	10.31 (2/3)	–	–	–
SuperPoint + DA V3	1/12	0.74	7.30 (1/3)	lost @71%	lost @4%	lost @29%

Against stereo the answer is one-sided, but not in the way the pilot machine suggested. With memory relieved, ORB2 with DepthAnything V3 completes `test0`, `test1` and `test3` three times of three, which the pilot machine never allowed. *Completion is no longer the problem.* Accuracy is: 8.30 m against 1.29 m on `test0`, 27.15 m against 7.70 m on `test1`, 20.08 m against 3.58 m on `test3`. A factor of roughly four to six, consistently, on every sequence.

The recovered scale explains much of it. Stereo runs come back at 0.985, metrically correct as they should be when depth is derived from a measured 0.5942 m baseline. The V3 runs come back at 0.75, 0.73 and 0.84 on the three sequences they finish, so the reconstructed world is consistently 16 to 27 % too small. The Umeyama alignment absorbs that bias before the ATE is computed, so the errors above are what *survives* the correction rather than the raw consequence of it. The slower V2 recovers scale almost perfectly (1.01 to 1.03) and still posts 10.2 m, which suggests the two generations trade global metric consistency against local depth quality in opposite directions.

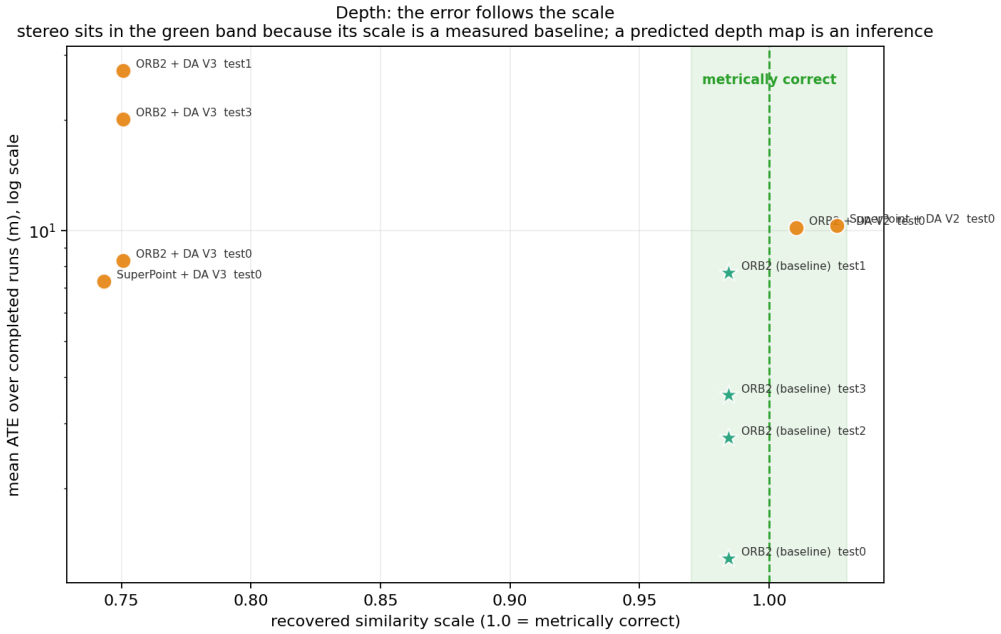


Figure 6.10: Depth: the error follows the scale. Stereo sits in the green band because its scale is a measured baseline; a predicted depth map is an inference, and the V3 generation infers a world consistently too small.

Scale is not the whole story, and `test3` shows why. On that sequence V3 recovers a scale of 0.84, much the closest to correct of the three, and still posts 20.08 m. If the bias were the only mechanism, that run should have been the best of the three rather than the second worst. Figure 6.11 shows what else is happening: the monocular trajectory has broadly the right size and shape, which is the visual counterpart of the 0.84, and what it lacks is correction. The ORB2 front end tracks visibly worse against a predicted depth map than against stereo disparity, because a smeared object boundary gives a keypoint a confidently wrong depth rather than no depth at all; and several loop closures that fire in the stereo run do not fire here, so the accumulated error is never pulled back. A trajectory of the right size that nobody straightened is a good description of what the figure shows.

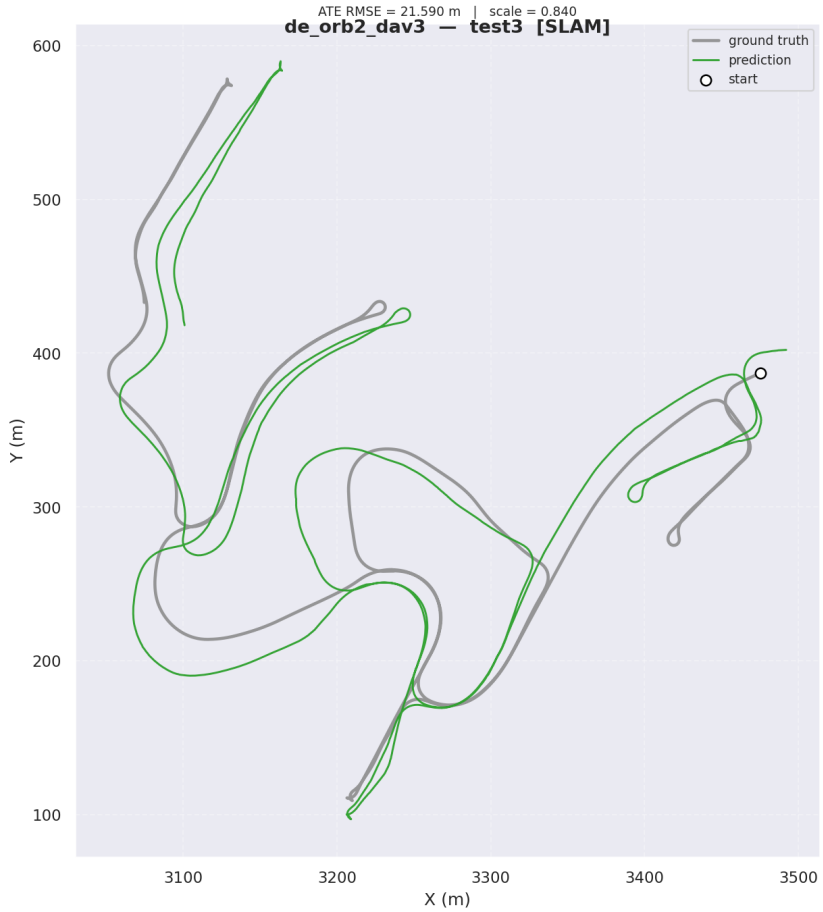


Figure 6.11: Depth on `test3`: the monocular reconstruction against ground truth and against the stereo baseline. Scale and shape are close to right; the error is in tracking and in loop closures that do not fire.

Table 6.9: Machine cost of each depth source. The V2 generation saturates the GPU and runs an order of magnitude below the camera rate; the V3 generation does neither. Columns as in Table 6.2, apart from the family column, dropped here since every row is the same family (transformer).

Configuration	GPU % (rel)	CPU % (rel)	fps (rel)	peak GB	CPU/frame
ORB2 (baseline)	**5.3 (1.00×)**	36.9 (1.00×)	**13.8 (1.00×)**	19.6	**2.67**
ORB2 + DA V3	34.3 (6.49×)	28.7 (0.78×)	10.7 (0.79×)	26.9	2.69
SuperPoint + DA V3	42.6 (8.05×)	27.3 (0.74×)	9.2 (0.68×)	13.9	2.95
ORB2 + DA V2	98.5 (19.63×)	**10.5 (0.28×)**	0.9 (0.06×)	**11.6**	12.25
SuperPoint + DA V2	98.6 (19.63×)	11.5 (0.31×)	0.8 (0.06×)	17.4	13.49

The cost table settles the choice between the two networks on its own, without reference to accuracy. DepthAnything V2 runs at 0.86 fps, roughly twelve times slower than the camera, and saturates the GPU at 98.5 %. V3 sustains 10.6 fps, faster than the camera and 77 % of the stereo pipeline's throughput, at a third of the GPU load. If neural depth is used at all on hardware of this class, only the V3 generation is viable.

6.9 How each family uses the machine

Figure 6.12 gives the shape of the whole comparison at once. Each configuration is one point, its processor load on one axis and its GPU load on the other, coloured by family. The four groups occupy four different regions of the plane and barely overlap: the classical configurations sit against the CPU axis with the GPU almost idle, the CNN group lifts off it, the transformer group climbs further, and the fully deep combinations sit furthest out on both axes at once. What the plot shows that no table does is that these are not four points on one scale of expensiveness. They are four different ways of using a machine, and which one suits a platform depends on which of its two processors is already busy.

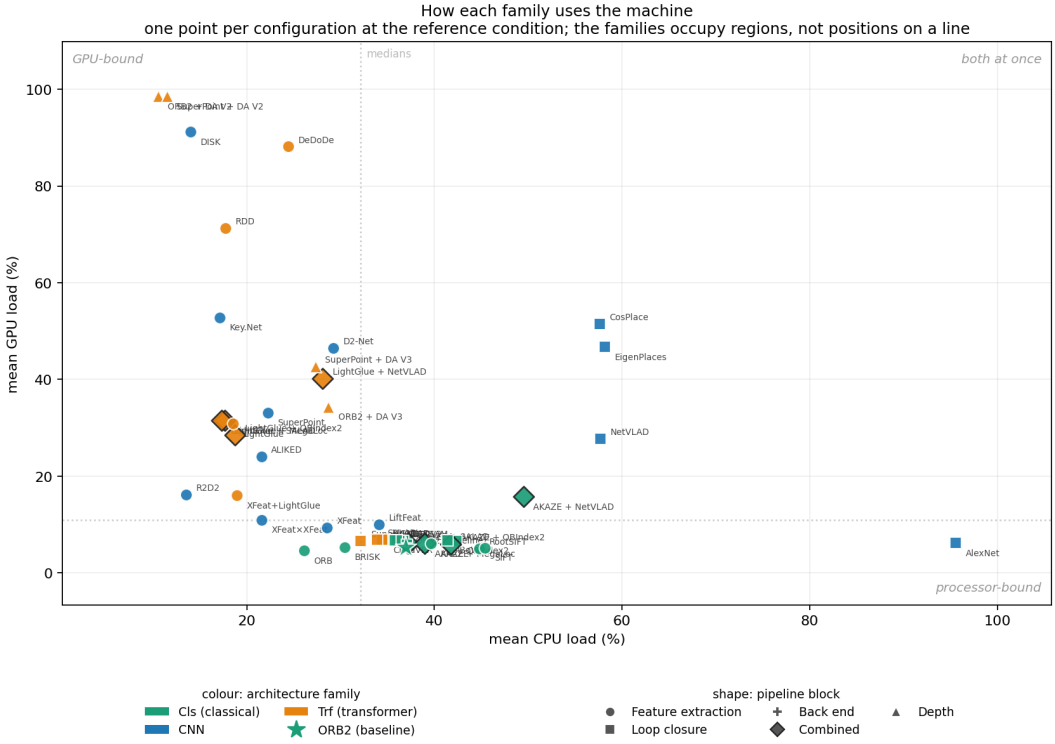


Figure 6.12: How each family uses the machine. Classical configurations load the processor and leave the GPU idle; deep ones do the reverse; the combined deep configurations load both. The families separate into regions rather than into positions on a line.

6.10 Calibration against published systems

Every figure so far measures a component inside pySLAM. This section asks how much of that is the component and how much is the bench.

No system below was re-run for this thesis. The figures are as their authors published them, on their own datasets and their own machines. Direct comparison is not possible: KITTI odometry is not KITTI-360, extraction throughput is not pipeline throughput, and an i7-4790 in 2017 is not a Ryzen 9 9950X in 2026. What the comparison can do is separate the differences with an obvious hardware or implementation explanation from the ones without.

Table 6.10: What the reference systems report, and on what. None of these numbers is comparable with this study's directly: the datasets, the resolutions and the machines all differ. The systems are chosen because each one runs a component this study also runs, so a difference in the numbers has a candidate explanation. DROID-SLAM shares nothing and is included for the opposite reason, given in the text.

System	Shared component	Hardware reported	Dataset	What is reported
ORB-SLAM2 [3]	ORB2 features, bag-of-words closer	Intel i7-4790, 16 GB, no GPU	KITTI 00–10, EuRoC, TUM	ATE and per-thread timing, 5 runs per sequence, no memory figure
ORB-SLAM3 [4]	ORB features, bag-of-words closer	Desktop CPU, no GPU	EuRoC, TUM-VI	ATE across 10 runs, multi-map and visual-inertial modes
Light-SLAM [52]	SuperPoint, LightGlue	Intel i5, RTX 3060	KITTI, EuRoC, TUM	ATE and per-frame time against ORB-SLAM2 and SuperPoint-SLAM on one machine
AirSLAM [53]	SuperPoint-family detector, LightGlue matcher	Desktop GPU and Jetson Orin	EuRoC, TartanAir, own sequences	ATE and frame rate on both a desktop and an embedded platform
iBoW-LCD [39]	the iBoW loop closer	Intel i7 2.5 GHz, 12 GB	KITTI and indoor/outdoor sets	Precision-recall, per-image query time, final vocabulary size
DROID-SLAM [5]	none	2 × RTX 3090, 24 GB VRAM each	TartanAir, EuRoC, TUM-RGBD	ATE; roughly 20 GB of VRAM and 20 fps on EuRoC as measured independently [54]

Table 6.10 collects the systems that share a component with this study: ORB features and a bag-of-words closer in the two ORB-SLAM systems, SuperPoint and LightGlue in Light-SLAM and AirSLAM, the iBoW closer in iBoW-LCD. A published number from a system with no shared component says only that a different pipeline behaved differently, which is not information.

What stands out is what is missing. None of these systems reports a memory figure. ORB-SLAM2 and ORB-SLAM3 report per-thread timing in millisecond detail and no peak resident set, Light-SLAM and AirSLAM report frame rates, and the closest any of them comes is iBoW-LCD's final vocabulary size, offered as an efficiency argument about the index rather than as a capacity requirement. DROID-SLAM is in the table for the contrast: it shares no component with this study, and it is the one system here that does state a memory figure, because at roughly 20 GB of VRAM the number is a purchasing decision rather than a footnote. Memory gets reported when it is large enough to stop somebody running the system, and stays unreported when it merely happens to fit on the author's machine. Chapter 7 is a measurement of what the second case hides.

The run counts are worth one line. ORB-SLAM2 reports the median of five runs per sequence and ORB-SLAM3 the median of ten, both because the multi-threaded system is non-deterministic. The three repetitions per cell used here sit at the bottom of the field's own convention, and the thin-repetition limitation of Section 9.7 is one this literature shares.

Table 6.11: Published throughput for the components this study also ran, beside what the same component achieved inside pySLAM at `R9-12t-28GB`. The published figures are extraction or matching only unless marked as full SLAM, and those are not directly comparable with the last column, which is whole-pipeline throughput. A **!** marks a figure measured on a GPU materially larger than the RTX 4060 Ti (8 GB) used here, which is the case where the hardware gap alone accounts for most of the difference.

Component	Hardware in the source	Reported	Scope	Here
ORB2 pipeline [3]	i7-4790, CPU only	∼40	full SLAM	**13.69**
ORB2 pipeline [52]	i5-12400F + RTX 3060	40.0	full SLAM	**13.69**
SuperPoint [27]	Intel i5-1135G7, CPU only	3.0	extraction	9.16
SuperPoint pipeline [52]	i5-12400F + RTX 3060	1.9	full SLAM†	9.16
XFeat [27]	Intel i5-1135G7, CPU only	27.1	extraction	7.60
XFeat [27]	RTX 4090 **!**	1400	extraction	7.60
DISK [27]	Intel i5-1135G7, CPU only	1.2	extraction	2.60
ALIKE‡ [27]	Intel i5-1135G7, CPU only	0.58	extraction	8.74
LightGlue [33]	RTX 3080, 1024 kpts **!**	150	matching	6.45
LightGlue [33]	i7-10700K, 512 kpts	20	matching	6.45
SP+LightGlue pipeline [52]	i5-12400F + RTX 3060	10.5	full SLAM	6.45
DROID-SLAM [54]	RTX 3090, 20 GB VRAM **!**	20	full SLAM	n/r

All figures in frames per second. **!** = measured on a GPU above this study's 8 GB card. † SuperPoint-SLAM, a separate system, not SuperPoint inside the same framework. ‡ ALIKE, the predecessor of the ALIKED variant run here.

Table 6.11 puts the published throughput figures beside this study's. Read carelessly it says the bench is slow, which is true and already stated in Section A.9. Read carefully it says three more useful things.

The baseline gap is the bench. A native C++ ORB-SLAM2 processes a KITTI frame in about 25 ms on a mid-range machine [52], roughly 40 fps, against 13.69 fps here. A factor of about three, on the same algorithm and on comparable hardware, isolates the Python orchestration and the inter-process image passing. Every absolute throughput figure in this thesis should be divided by roughly three before it is set against a native system, which is why the ratios are reported alongside the absolutes throughout.

The deep front ends' gap is not the bench, and it runs the other way. SuperPoint extracts at 3.0 fps on a laptop CPU [27]; here the whole pipeline reaches 9.16 fps, because the RTX 4060 Ti is doing work that figure did on a processor. The same holds for ALIKE at 0.58 fps against ALIKED's 8.74 here. The deep front ends were given a GPU and used it, and they still did not complete the long sequences. The marked rows are where no comparison is possible at all: XFeat's headline 1400 fps is measured on an RTX 4090 at VGA resolution and LightGlue's 150 fps on an RTX 3080 at 1024 keypoints, both in isolation rather than inside a pipeline.

One published comparison supports this study's central cost claim. The Light-SLAM figures [52] come from one native implementation on one machine, which makes the ratio between its rows meaningful even where the absolute values are not. It reports 25 ms per frame for ORB-SLAM2 against 95 ms for a SuperPoint plus LightGlue pipeline, a premium of 3.8× for the attention front end. The same premium here is 2.1×, 13.69 fps

against 6.45. The cost of moving to an attention front end is therefore not an artefact of the Python bench. It appears in native code too, and there it is larger. Where the two disagree is on SuperPoint alone, which Light-SLAM reports as slower than its own SuperPoint-plus-LightGlue pipeline while it is faster here; SuperPoint-SLAM is a separate third-party system in that comparison, so its figure measures an implementation as much as a component, which is exactly the confusion the protocol of Section 5.1 exists to avoid.

One structural point decides how far Chapter 7 carries, and it is about map pruning. The capacity finding here has a specific mechanism behind it (Section 4.7): pySLAM retains keyframes generously and never culls them within a run, so memory demand is a function of route length. Every production system culls. ORB-SLAM2 removes keyframes whose map points are largely observed by others, and ORB-SLAM3 and Stella-VSLAM inherit that policy. A culled pipeline therefore hits the capacity wall at a longer route, not at no route at all. What transfers is the shape of the relationship, that memory demand tracks distance driven rather than model size. What does not transfer is the 14 GB threshold, which is a number about this bench.

CHAPTER 7

Results II: The Hardware Ladder

This chapter reports the machine's side of the same runs: throughput, processor and memory load, and above all which failures the hardware was responsible for. It reads all four conditions on the 139 cells common to every one of them, so that a condition cannot look better or worse purely because of what it attempted.

Each condition differs from the one before by exactly one factor, so the three comparisons below are named for the factor that changed. Table 7.1 is the chapter in one page.

Table 7.1: The hardware ladder, on the 139 cells common to all four conditions. The runs column is not identical across rows, because the baseline carries extra repetitions at the reference condition. "Succ." is the raw share of completed runs; "alg." is the share over the algorithm-attributable subset (successes plus tracking losses, with memory and timeout deaths removed). The two converge as the budget grows and are identical once no run dies of memory.

Condition	runs	succ. %	alg. %	lost %	OOM	t/out	fps	peak GB	ATE m
i7-8700K, 12t, 14 GB	401	49.9	57.1	37.4	39	10	4.57	11.8	2.43
R9 9950X, 12t, 14 GB	417	56.8	59.5	38.6	12	5	9.51	11.7	2.76
R9 9950X, 12t, 28 GB	425	70.8	71.8	27.8	0	4	10.02	12.6	3.24
R9 9950X, 32t, 28 GB	417	69.5	70.7	28.8	0	5	9.37	13.1	2.87

Two columns deserve explanation before anything else is said. "Succ." is the raw share of completed runs. "Alg." divides successes by successes plus tracking losses, removing the budget deaths entirely, and it is the figure a claim about an algorithm should be scored on because it does not punish a method for being too large for the machine. The two differ by seven points on the pilot machine and by one point at 28 GB, and that convergence is the whole justification for Chapter 6 having used a single condition.

7.1 Changing the platform: a faster machine, but not for everyone

Moving from the i7-8700K to the Ryzen 9 9950X at the same 14 GB budget doubles median throughput, from 4.57 to 9.51 fps. That is the largest single change in the study on the speed axis, and it is almost the only thing it changes: completion rises by seven points, from 49.9 % to 56.8 %, and the algorithm-attributable rate barely moves at all, from 57.1 % to 59.5 %.

The speedup is not uniform. Figure 7.1 splits the per-cell throughput ratio two ways. By architecture family, the classical methods collect 2.40×, the CNN methods 1.74× and the transformer methods 1.55×. By pipeline block, loop closure gains most at 2.67× and feature extraction least at 1.65×. The rank correlation between a cell's speedup and its GPU load on the old machine is −0.73.

The mechanism is straightforward and the numbers confirm it rather than merely suggesting it: a faster processor speeds up the part of the pipeline that runs on the processor. A classical extractor is almost entirely CPU work and collects the full generational win. A deep extractor spends most of each frame inside a network on a GPU that did not change between the two conditions, so its ratio is dragged towards one. The GPU being identical across all four conditions is what makes this a controlled measurement rather than an inference.

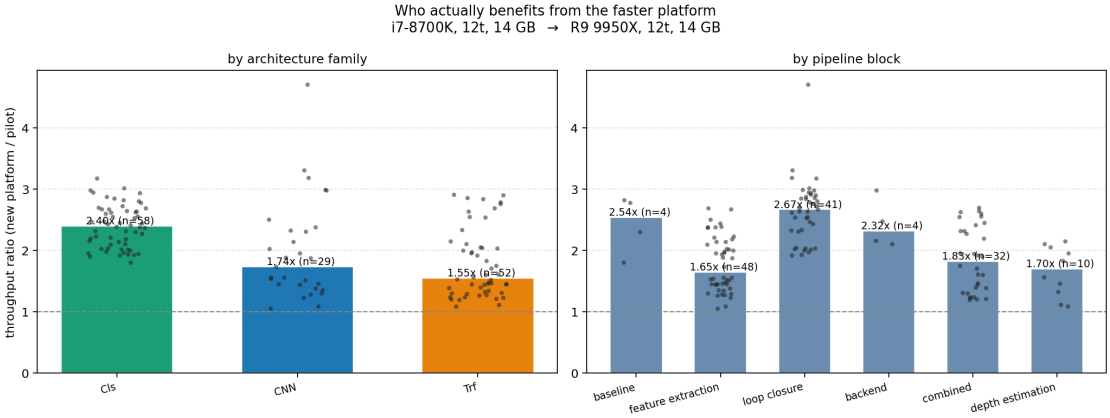


Figure 7.1: Who actually benefits from the faster platform. Level bars would mean the pipeline is uniformly CPU-bound; it is not, and that is a statement about the algorithms rather than about the hardware.

Throughput is worth seeing in absolute terms as well as in ratios, because the ratio hides what an engineer actually cares about: whether the pipeline keeps up with the camera. Figure 7.2 plots measured frames per second for every configuration across all four conditions against the 10 Hz line the sensor delivers. Almost nothing clears that line on the pilot machine. On the current machine the classical configurations clear it comfortably and most of the deep ones still do not, and no amount of memory or extra cores moves them across it. The threshold matters more than the ordering: a system at 12 fps and a system at 8 fps are not two points on a continuum, they are a system that runs and a system that falls behind and never catches up.

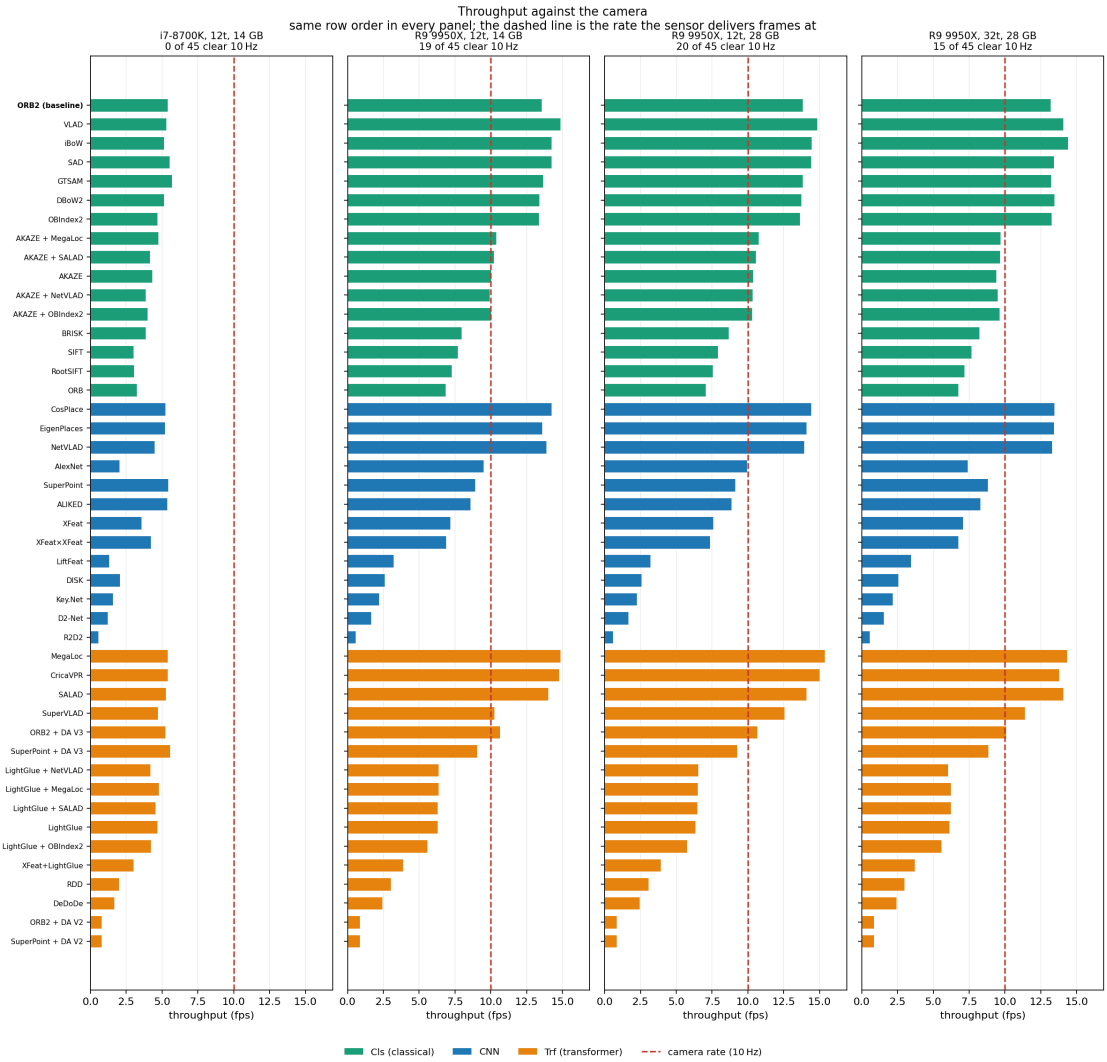


Figure 7.2: Throughput against the camera. The 10 Hz line is the one that matters: above it the pipeline keeps up, below it every frame arrives before the last one finished. The platform change moves the classical group across that line and leaves most of the deep group below it.

7.2 Changing the memory capacity: where the failures actually went

Doubling the memory budget from 14 to 28 GB, changing nothing else, is the most consequential single change in the study. Out-of-memory deaths go from twelve to zero. Completion rises from 56.8 % to 70.8 %, and the algorithm-attributable rate from 59.5 % to 71.8 %. Median throughput rises by five per cent, which is within noise and is not the point.

What makes this more than a predictable result is where the recovered runs came from. Sixty-four more runs completed at 28 GB than at 14 GB, and only twelve of those can be attributed to the eliminated out-of-memory deaths. The outcome counts reconcile exactly: +64 completions = 43 fewer tracking losses (161 to 118) + 12 fewer out-of-memory deaths + one fewer timeout + the eight additional repetitions this condition ran (425 attempts against 417). Because the two conditions did not attempt an identical

number of runs, the percentage-point gain of 14.0 is the safer figure to quote. Chapter 8 takes that apart, because it is not a hardware finding alone.

Figure 7.3 shows the change cell by cell. The right-hand panel is the argument: absolute peak memory is not comparable across two different budgets, since 13.9 GB means suffocating at 14 and comfortable at 28, so the panel plots each run against its own machine's ceiling. The rescued cells stack against 100 % on the left of that panel and spread out to the right, which is the mechanism in one picture. They were not failing for anything algorithmic. They were failing because they had nothing left.

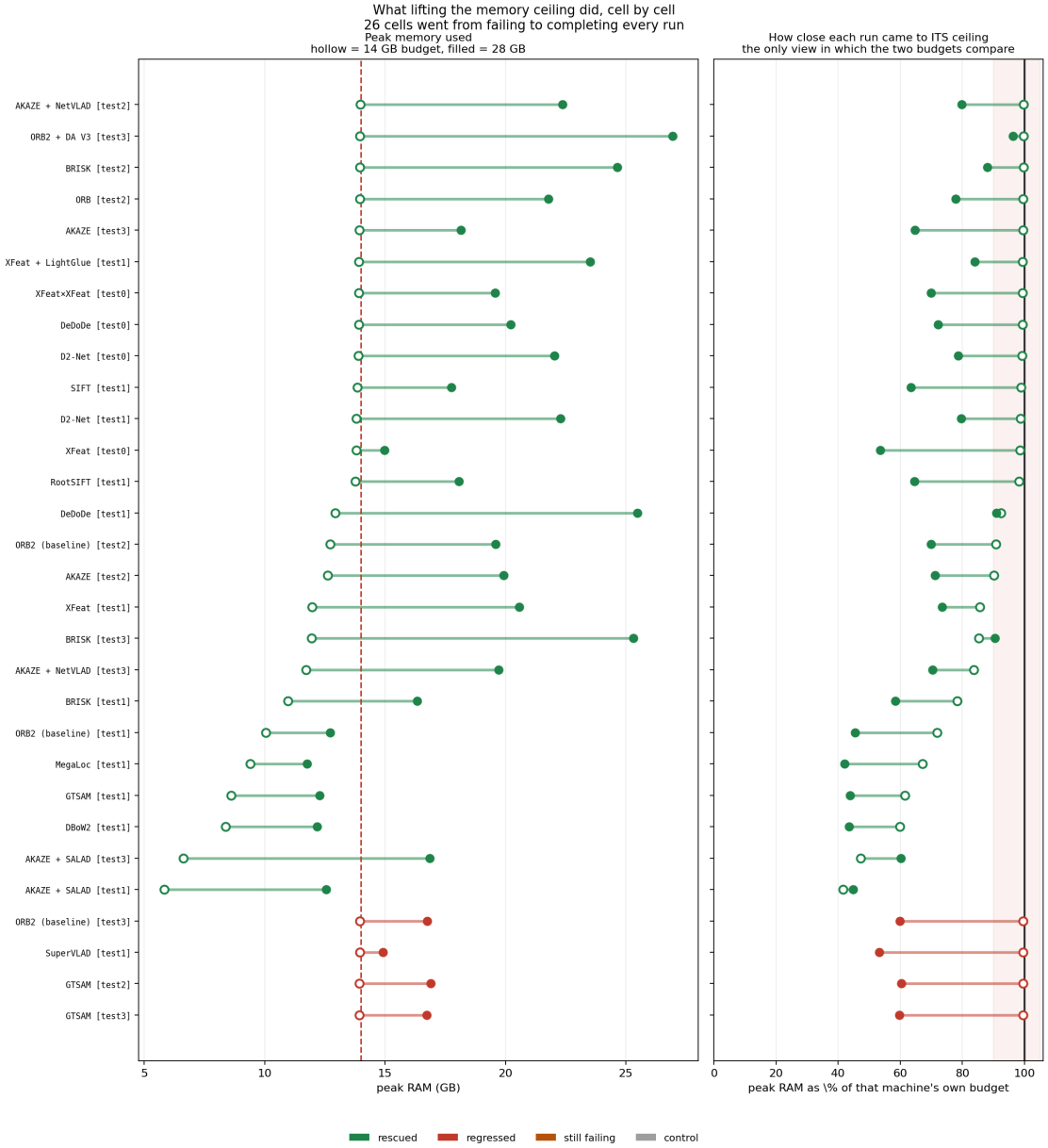


Figure 7.3: What lifting the memory ceiling did, cell by cell. Hollow markers are the 14 GB condition and filled markers the 28 GB one; the line between them is the change.

Peak memory is a summary, and summarising is what makes the failure look sudden. Figure 7.4 follows a single configuration on `test3` through the whole drive, once at 14 GB and once at 28 GB, plotting resident memory against elapsed frame. The two traces are identical for the first part of the sequence, because the map is small and the

budget is irrelevant. They separate when the map has grown enough to matter: the 14 GB trace flattens against its ceiling and stays there, and shortly afterwards the run is declared lost, while the 28 GB trace continues climbing past that point and finishes the drive. Nothing in the shape of the first trace announces a memory problem. It looks like a system that plateaued and then failed, which is exactly how the outcome label recorded it.

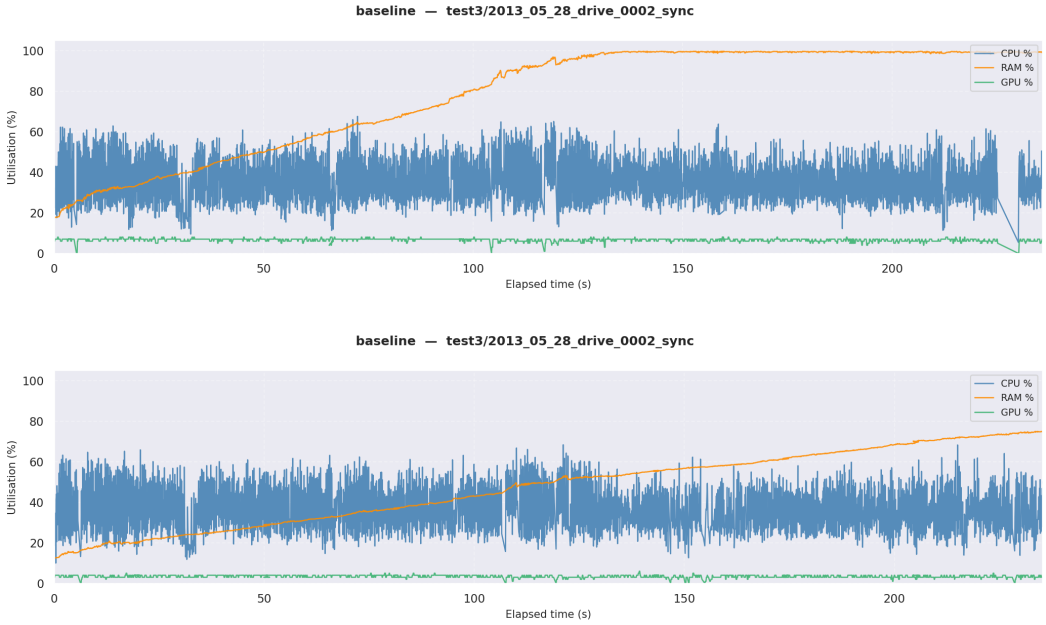


Figure 7.4: The baseline run on `test3` at two memory budgets. Above, 14 GB: memory flattens against the ceiling and the run is declared lost. Below, 28 GB: the same trace continues and the drive finishes.

7.3 Changing the core count: a negative result

Going from twelve logical processors to thirty-two, at the same 28 GB budget, makes the pipeline slightly slower. Median throughput falls from 10.02 to 9.37 fps, a loss of six per cent; the per-cell ratio has a median of 0.95, and 127 of 139 cells are slower rather than faster. Completion is unchanged within noise, 70.8 % against 69.5 %.

Figure 7.5 shows why, and the answer is not that the extra cores go unused. Average busy threads rise from 3.8 to 6.4, and peak busy threads from about seven to about fifteen, so the scheduler is genuinely spreading work more widely. It simply buys nothing: the pipeline's tracking loop is serial, and the parallel sections that exist saturate well below twelve threads. Beyond that point the additional threads add contention and scheduling overhead without adding work, and the six per cent throughput loss is what that overhead costs [55], [56].

This result also settles a loose end from Section 5.2. The platform comparison bundles the generational change together with an SMT confound, because twelve logical processors on the pilot machine is six physical cores while twelve on the current one is twelve. If thread count mattered much on this workload, that confound would be serious. It does not, so it is not.

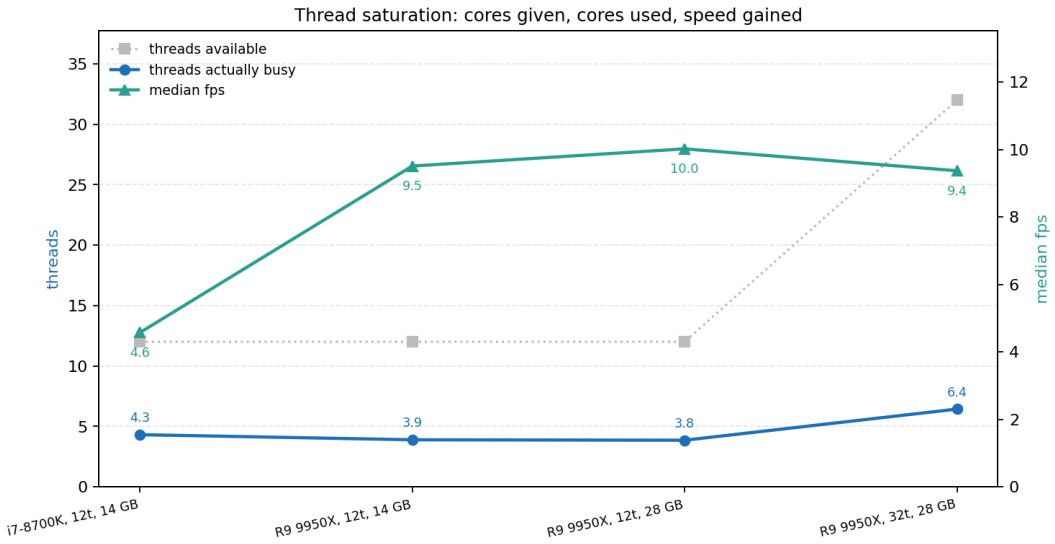


Figure 7.5: Thread saturation: cores given, cores used, speed gained. Busy threads climb on the last change while throughput does not, which is what a saturated parallel section looks like.

7.4 What does not move

Figure 7.6 summarises every configuration at every condition as four stacked bars. The per-configuration detail behind it, completed runs over attempts for all 139 cells across all four conditions, lives in Appendix D.

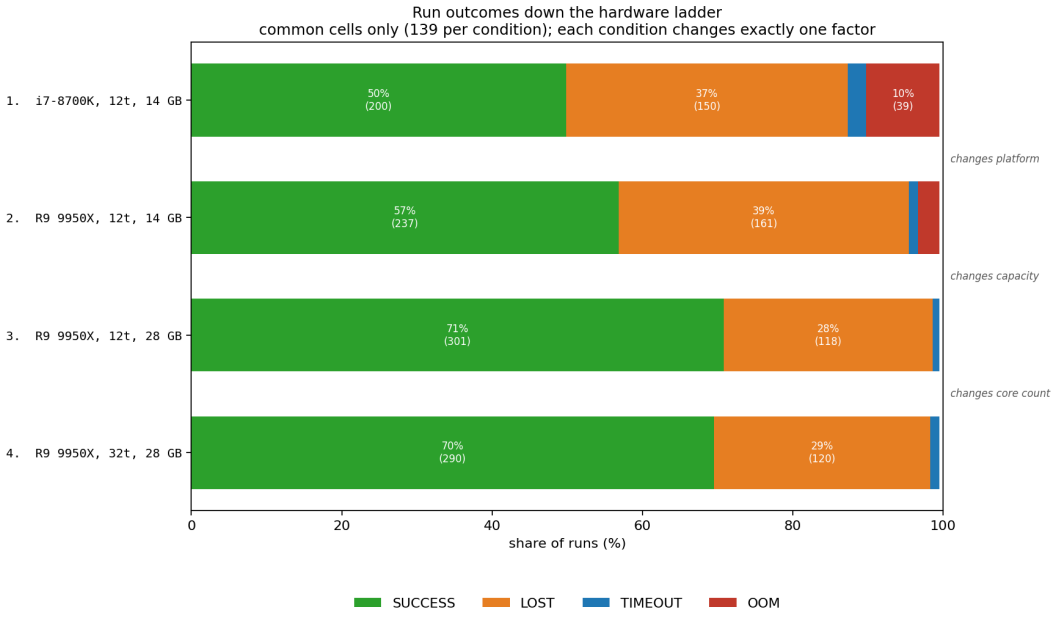


Figure 7.6: Run outcomes across the four conditions, on the common cells. Each step changes exactly one factor, named in the annotation between the bars.

The chapter closes with the quantity that did *not* move. Table 7.2 takes the 47 cells that completed at least 98 % of their sequence under every condition and reports their ATE at each. The medians are 1.287, 1.309, 1.292 and 1.302 metres, in order of increasing machine. A Friedman test, which asks whether the same items are ranked differently

across repeated conditions, gives $p = 0.24$. The correct reading is the conservative one: across the four conditions evaluated here, no hardware-dependent change in ATE was statistically detectable on this set of cells. With three repetitions per cell the test has limited power against a small effect, so this bounds any such effect rather than excluding it. Over the wider set of cells that produce an accuracy figure under all four conditions, the rank correlation between any two conditions runs from 0.89 to 0.95.

That is unsurprising in principle, since none of these methods is stochastic in a way the hardware would perturb, but it needed measuring rather than assuming, because it is exactly the assumption Chapter 6 rests on. Hardware decides *whether* you get an answer far more than it decides what the answer is.

Table 7.2: ATE on the 47 cells that completed at least 98% of their sequence under *every* condition. If accuracy moved with the hardware, no single-condition comparison would generalise. A Friedman test across the four conditions gives $p = 0.24$.

Condition	median ATE (m)	mean ATE (m)
i7-8700K, 12t, 14 GB	1.287	5.966
R9 9950X, 12t, 14 GB	1.309	5.881
R9 9950X, 12t, 28 GB	1.292	5.892
R9 9950X, 32t, 28 GB	1.302	9.664

7.5 The resource ordering

Three conditions, three factors, three outcomes. Capacity took budget failures to zero and lifted completion by nearly fourteen points. Platform speed doubled throughput, halved timeouts, and left every out-of-memory death in place: a faster way to reach the same wall. Core count did nothing, and slightly less than nothing. For this workload,

$$\boxed{\text{memory capacity} \;>\; \text{platform speed} \;>\; \text{core count}} \tag{7.1}$$

which is directly actionable when specifying a machine and, at 2026 memory prices (Appendix C), directly costable.

CHAPTER 8

Results III: Where Hardware and Algorithm Interact

Three results in this study belong to neither the component half nor the hardware half, because each is a statement about the interaction between the two. All three are visible only because the same cells were run at more than one budget and in more than one mode. A benchmark on a single machine would have reported the first as a property of loop closure, the second as an algorithmic weakness, and the third not at all. That is why they are gathered here rather than distributed through the two preceding chapters.

8.1 Loop closure is a liability when starved and an asset when fed

The same front ends were run twice, once as pure visual odometry with loop closure disabled and once in full SLAM, at both memory budgets. The four combinations produce the clearest single result in the study, and it is a reversal.

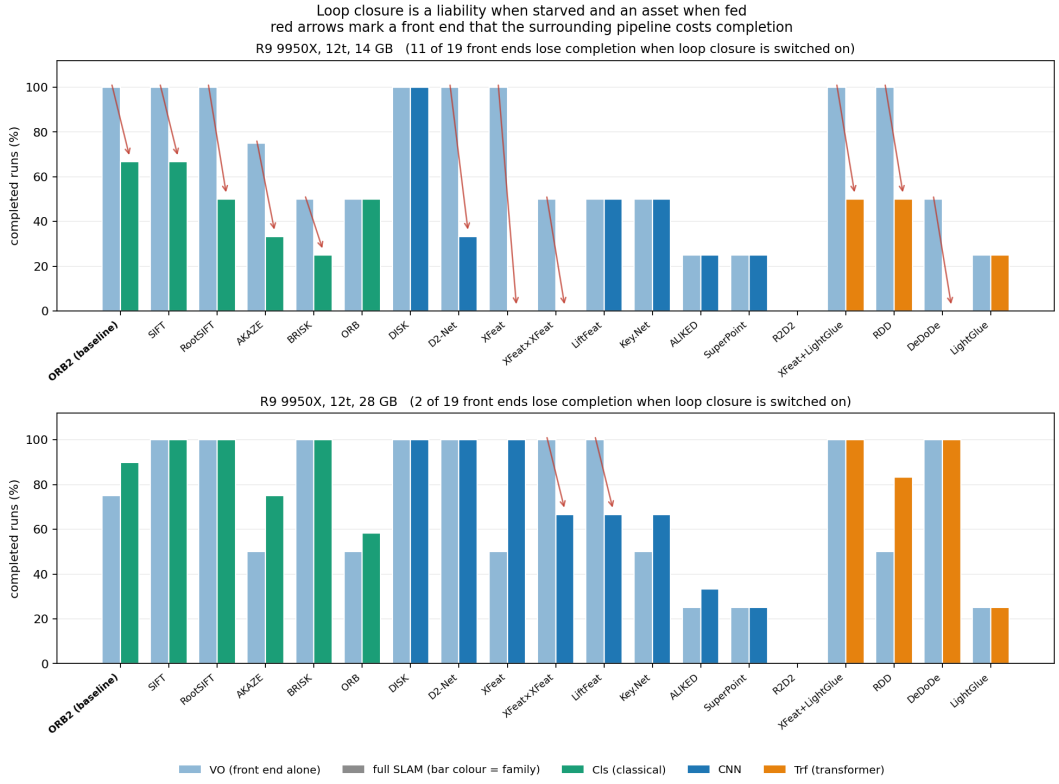


Figure 8.1: Loop closure is a liability when starved and an asset when fed. Both panels use the same ordering so a front end can be tracked between them. At 14 GB eleven of nineteen bars move down; at 28 GB only two do.

At 14 GB, eleven of the nineteen front ends complete fewer runs in SLAM than in visual odometry, and not one completes more. XFeat completes every odometry run it attempts and none of its SLAM runs. DeDoDe goes from 50 % to zero, AKAZE from 75 % to 33 %, the baseline from 100 % to 67 %. At 28 GB the same comparison has only two front ends losing ground, and seven of the rest gain: XFeat from 50 % to 100 %, RDD from 50 % to 83 %, AKAZE from 50 % to 75 %, the baseline from 75 % to 92 %. Figure 8.1 shows the comparison front end by front end, at both budgets.

That reversal has a practical consequence, and it is the answer to RQ4. Turning loop closure off is a way of buying memory without buying memory, and it works well enough to be predictive: the ranking of front ends by visual-odometry completion at 14 GB predicts their full-SLAM completion at 28 GB with a rank correlation of 0.82, while their full-SLAM completion at 14 GB predicts it at only 0.30. If you want to know which front ends will survive on a bigger machine, you learn more by running them stripped down on the small one than by running them fully loaded.

Table 8.1 gives the mechanism across all runs at once. Visual odometry completes 61.5 % of its runs at 14 GB and 63.5 % at 28 GB: it barely notices the extra memory, because it accumulates almost no history. Full SLAM goes from 56.8 % to 70.8 % over the same change.

Table 8.1: Completion with and without loop closure, at both memory budgets. Visual odometry accumulates almost no history and barely notices the extra memory; full SLAM does almost nothing else.

Mode	Condition	runs	completed %	lost	OOM
VO	R9 9950X, 12t, 14 GB	52	61.5	13	2
	R9 9950X, 12t, 28 GB	52	63.5	18	0
SLAM	R9 9950X, 12t, 14 GB	417	56.8	161	12
	R9 9950X, 12t, 28 GB	425	70.8	118	0

Loop closure and relocalisation are recovery machinery, and recovery machinery has a footprint. Starved, it competes with the map for the same scarce resource and costs more than it returns. Given room, it does what it was designed for and pulls back tracks that pure odometry abandons. The component is identical in both cases. Only the space it has to work in differs.

8.2 A failure label is not a diagnosis

The second finding is the first seen from the other side, and it is a warning about the outcome labels themselves.

Table 8.2 lists the *rescued* cells, defined here as those that completed no repetition at all at 14 GB and at least one at 28 GB. There are 28 of them, accounting for 70 recovered runs; 19 went on to complete every repetition. Of the 84 individual failures behind those 28 cells, 75 were recorded as LOST, eight as OOM and one as TIMEOUT, roughly nine algorithm-shaped verdicts for every honest resource death. The definition matters, and a stricter one gives a similar picture: requiring full completion at 28 GB rather than partial yields 26 cells and the same 70 recovered runs. Across those cells the median of the per-cell mean peak was 12.3 GB against a 14 GB ceiling before and 18.1 GB after; on per-cell median peak the same figures are 13.0 and 18.2 GB, and eleven of the 28 sat within 0.2 GB of the ceiling.

Table 8.2: The 28 cells that went from zero successful runs at 14 GB to at least one at 28 GB, accounting for 70 recovered runs; 19 recovered every repetition. The *was* column gives the modal outcome label across the three failed runs at 14 GB. At run level, 75 of the 84 failures here were recorded as LOST, 8 as OOM and 1 as TIMEOUT: roughly nine in ten carried an algorithmic label for what the 28 GB column shows to be memory pressure. Not one line of code differs between the two columns.

Variant	Seq.	was	14 GB	28 GB	peak GB @14	peak GB @28
ORB2 (baseline)	test1	LOST	0/3	5/5	10.1	12.7
GTSAM	test1	LOST	0/3	3/3	8.6	12.3
AKAZE + MegaLoc	test1	LOST	0/3	2/3	6.7	11.1
AKAZE + SALAD	test1	LOST	0/3	3/3	5.8	12.6
AKAZE + SALAD	test3	LOST	0/3	3/3	6.6	16.9
LightGlue + MegaLoc	test2	LOST	0/3	1/3	9.1	13.7
LightGlue + NetVLAD	test2	LOST	0/3	1/3	8.5	14.6
LightGlue + SALAD	test1	LOST	0/3	1/3	7.8	11.4
ORB2 + DA V3	test3	LOST	0/3	3/3	13.9	26.9
AKAZE	test2	LOST	0/3	3/3	12.6	19.9
AKAZE	test3	LOST	0/3	3/3	13.9	18.1
ALIKED	test2	LOST	0/3	1/3	6.9	12.0
BRISK	test1	LOST	0/3	3/3	11.0	16.3
BRISK	test3	LOST	0/3	3/3	12.0	25.3
D2-Net	test1	LOST	0/3	3/3	13.8	22.3
DeDoDe	test0	LOST	0/3	3/3	13.9	20.2
DeDoDe	test1	LOST	0/3	3/3	12.9	25.5
Key.Net	test1	LOST	0/3	1/3	8.2	11.7
RDD	test1	LOST	0/3	2/3	13.8	18.9
RootSIFT	test1	LOST	0/3	3/3	13.8	18.1
XFeat	test0	LOST	0/3	3/3	13.8	15.0
XFeat	test1	LOST	0/3	3/3	12.0	20.6
XFeat + LightGlue	test1	LOST	0/3	3/3	13.9	23.5
XFeat×XFeat	test0	LOST	0/3	3/3	13.9	19.6
XFeat×XFeat	test1	LOST	0/3	1/3	12.8	21.5
DBoW2	test1	LOST	0/3	3/3	8.4	12.2
BRISK	test2	OOM	0/3	3/3	13.9	24.6
LiftFeat	test1	OOM	0/3	1/3	13.9	25.3

Read on its own, the 14 GB data says those methods lose tracking, which is an algorithmic verdict. It is the wrong verdict, and the next column is the evidence: the identical code completes those sequences once the budget is raised, without a line changing. Note also which methods are in that table. It contains BRISK, AKAZE, RootSIFT and ORB2 that are the light, classical front ends. What ran out of memory was not the model but the map, which grows with route length rather than with model size, exactly as Figure 4.8 predicts.

Figure 8.2 makes the size of the mistake visible. It shows the 14 GB outcomes as they were recorded, and beside them the same runs re-labelled with what the 28 GB result indicates they actually were. Almost the entire LOST band on the left sits in cells that memory alone brought back, and the honest OOM band is a sliver by comparison. The proportion is the finding: for every failure the machine reported honestly, roughly nine were filed under the algorithm's name.

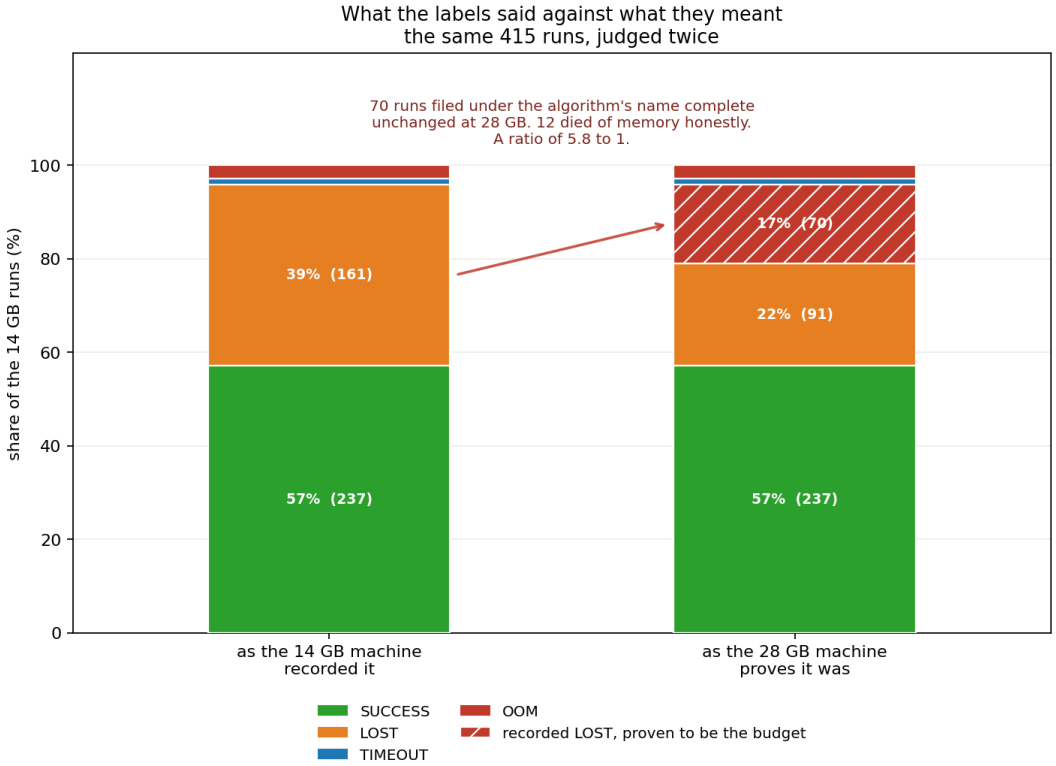


Figure 8.2: What the labels said against what they meant. Left, outcomes as the 14 GB machine recorded them; right, the same runs read against the 28 GB result. The highlighted band is budget failures filed under the algorithm.

The mechanism is that memory pressure does not usually announce itself, and Figure 7.4 showed it happening within a single run. It starves the local map, the tracker finds fewer usable correspondences, and after ten consecutive failures the run is declared lost by a rule that cannot distinguish a hard corner from an exhausted machine. Only the runs whose allocation happens to fail outright get labelled honestly. The same underlying cause therefore surfaced as OOM in only eight of the 84 failures behind the rescued cells and as LOST in seventy-five, and any analysis treating the second category as algorithmic will draw a confident wrong conclusion about a method that has no such problem.

This is the strongest methodological argument the thesis makes, and it reaches past this study. A benchmark run on one machine, close to that machine's limits, produces algorithmic-looking failures that are not algorithmic at all, and nothing in its own output reveals the difference. Only a second budget does.

8.3 Neither block can be judged without the other

The third finding is that the front end and the loop closer are not two components that can be evaluated separately and then combined. Each determines what the other is allowed to do, and the dependency runs in both directions.

A front end that stops makes the closer irrelevant. Every LightGlue pair in Section 6.7, whichever closer was attached, dies at very nearly the same point in each sequence as bare LightGlue with no closer at all. It does not matter how good the place recognition is: a closer only ever sees the keyframes the tracker survived long enough to produce, and it cannot recognise a place the vehicle never reached. The best loop closer in the study, bolted to a front end that fails in the first stretch of a drive, contributes nothing.

A closer that does not fire leaves the front end's drift in place. This direction is easier to miss, because the run finishes and therefore looks like a success. AKAZE + MegaLoc on `test3` completes and posts 97 m, and AKAZE alone on `test2` completes and posts 35 m, and in both cases the trajectory plots show the same thing: the tracker accumulates error through one hard manoeuvre and the closer never triggers to pull it back. A pipeline whose closer fires reliably tolerates a front end that drifts; one whose closer stays silent does not.

And the coupling can work in reverse. LiftFeat and XFeat×XFeat both complete every visual-odometry run they attempt, including `test1`, and both fall to one completed run in three on that same sequence once loop closure is enabled. Neither dies of memory, at a condition where nothing else is short of anything. The front end did not get worse, because it is the same code processing the same frames; what changed is what happens around it. Two mechanisms fit the data and this study cannot separate them. Both extractors run under the independent vocabulary of Section 5.1.1, so their own descriptors are never the ones the loop detector reasons about, and a closer that cannot verify a match on the tracker's own features is a closer that can accept a wrong one. The simpler alternative is that both carry the largest maps in the phase, and a badly placed loop triggers a pose-graph optimization and a map fusion over a structure that size. Either way, adding a closer made a working front end fail.

Together with the loop-closure phase, where an identical ORB2 tracker fails under one closer and completes three times of three under another, and with the cost tables, where a component's measured GPU load turns out to depend on how the platform invokes it, the conclusion is not about feature extraction or about place recognition. It is that a component's value is a property of the pipeline it sits in, and that any benchmark reporting one block in isolation is reporting a number that does not travel.

CHAPTER 9

Discussion

The three results chapters reported what happened. This chapter says what it means: first by answering the four research questions in order, then by turning the answers into a decision guide, and finally by stating what could undermine all of it.

9.1 RQ1: how the component families compare at fixed hardware

Depth of architecture bought cost without a consistent return. The organising claim of Chapter 3 was that moving along the hand-crafted → CNN → transformer axis buys accuracy and robustness, and that the bill for both arrives as compute. Only the bill arrived. On accuracy the axis is invisible where enough of every family completes, and Figure 6.1 shows why in the cleanest possible form: short-window RPE, which is the direct measure of what a front end does frame to frame, is identical across the three families. On robustness the axis does not merely vanish, it inverts.

Two things are worth separating here, because they are often conflated. The learned front ends are not *bad* at matching; they are matching a scene their training distribution covers well, and their local motion estimates are as good as anybody's. What they fail at is surviving a manoeuvre in which the correspondences run out, and what they cost is an order of magnitude of GPU load for a capability this dataset never asks for.

The cost–value frontier is flat. This follows directly from the above. Compute varies by more than an order of magnitude across the field while neither accuracy nor completion moves with it, so the frontier a reader would expect to rise is flat, and over part of its range it slopes the wrong way. Nor does assembling the best parts produce the best system, though that failure is not as clean as the accuracy axis alone suggests: the best assembled pairings match or exceed the default on completion (`AKAZE+NetVLAD` and `AKAZE+SALAD` both finish 12/12 against the baseline's 18/20) while losing to it by an order of magnitude on ATE for the long sequences, so no assembled pairing dominates on both axes, and Section 8.3 explains why components that are individually strong could not be expected to compose into a strong system.

Learned components earn their cost in specific conditions. This is the second half of RQ1, and it is where a pooled ranking breaks down. Deep place recognition does what its literature claims, but only where the difficulty it was built for is the difficulty present, and Section 4.6 identifies that condition precisely: revisits approached from a materially different heading. `test1` has them and the DINOv2 closers win there; the other three sequences do not and the cheap closers take it back. The correct conclusion is not that classical closers win, but that *a pooled ranking of loop closers is the wrong object to compute.* The useful output is a rule for when to reach for a deep closer, not a league table.

Two component choices the benchmark settles outright. The back end is a non-decision: the two solvers differ by about two centimetres and by two runs in twelve, so the choice can be made on licensing or API preference at no accuracy cost. The depth source is a decision, and it goes against the hope that monocular neural depth

is close to a substitute for stereo: V3 is fast enough to use and reconstructs the world 16 to 27 % too small, V2 recovers scale and runs twelve times slower than the camera. Monocular neural depth is a way of removing a sensor, not a way of keeping accuracy while removing one.

9.2 RQ2: what changes when the hardware changes

Better hardware scales the accuracy results rather than reordering them, and does the opposite to completion. The Spearman rank correlation between any two conditions runs from 0.89 to 0.95 on accuracy against 0.53 to 0.81 on completion, which is the split the whole thesis turns on. For the configurations that completed their sequences under all four conditions, no statistically detectable hardware-dependent change in ATE was observed; completion, by contrast, is a property of the method *and* the machine it was measured on. Section 6.10 bounds how far it should be expected to carry beyond this pipeline. A single-machine benchmark can report the first and cannot report the second, and (this is the part that matters) it has no way of telling the reader which of the two it just did.

Section 8.2 shows how that failure presents in practice. It is not that a single-machine benchmark reports completion with wide error bars; it is that it reports the *wrong cause*, confidently, in a label that looks algorithmic. Seventy-five individual runs, spread over 28 cells, were recorded as losing tracking in cells that went on to complete once the budget was raised. Any reader of the 14 GB results alone would have concluded that BRISK, AKAZE, RootSIFT and ORB2 have a robustness problem on long drives. They do not.

The practical bound this places on the literature is worth stating plainly. Accuracy comparisons published on one machine can be expected to transfer. Robustness and completion comparisons published on one machine transfer only if that machine was comfortably above the workload's memory requirement, and papers do not usually report enough to tell whether it was.

9.3 RQ3: decomposing "better hardware"

The decomposition is unambiguous and is stated in Equation 7.1. Capacity binds, platform speed helps at what it is for, and cores do nothing.

The second half of RQ3 asks what actually consumes the binding resource, and the answer is not the obvious one. A tight memory ceiling looks like it must be killing the heavy networks, so that relieving it should bring deep completion into line with classical. What the rescued set actually contains is the light classical front ends. The reason is structural: the model is loaded once and its footprint is constant, whereas the map grows monotonically with route length and is never pruned within a run (Section 4.7). On a five-minute drive the map wins that race comfortably. A deep model's weights are a fixed tax; the map is a leak.

This has a direct consequence for anyone specifying an edge platform, and it is the opposite of the intuitive one. The memory question is not "how big are the models I want to run", it is "how long is the longest route between resets". Two configurations with identical model footprints will have entirely different memory requirements if one of them drives twice as far.

A faster platform lifts only what it is for, for a related reason. It reaches the same wall sooner: it halved timeouts, which are a speed failure, and left every out-of-memory death untouched, because a memory failure is not a speed failure. And the core-count

result is simply Amdahl's law with the numbers filled in [55], [56]: a pipeline that peaks at about seven busy threads cannot be helped by being given twenty more, and the six per cent it loses is the scheduling overhead of pretending otherwise.

9.4 RQ4: is footprint reduction a substitute for capacity?

Partly, and usefully. Section 8.1 shows that disabling loop closure reorders the field in much the same way that doubling the memory does: the visual-odometry ranking at 14 GB predicts full-SLAM completion at 28 GB with $\rho_s = 0.82$, against 0.30 for full SLAM at 14 GB. Intrinsic footprint reduction and extrinsic capacity increase are therefore substitutable *as diagnostics*, which has an immediate practical use: to find out which configurations will survive on a machine you cannot afford yet, run them stripped down on the machine you have.

They are not substitutable as *deployments*, and the same section says why. Loop closure is not overhead, it is the mechanism that pulls back accumulated drift, and a pipeline that runs without it is a pipeline that will drift. What the reversal shows is that recovery machinery has a footprint, and that below some capacity threshold the footprint costs more than the recovery returns. The honest formulation is that a memory budget determines whether loop closure is worth having, which is not a sentence anyone expects to write about a SLAM component.

9.5 What transfers beyond pySLAM

Section 6.10 put this study's numbers beside the published ones. This section says what that licenses.

Transferable. Three findings have mechanisms that exist outside this bench: that memory demand scales with route length rather than model size, since every keyframe-based system accumulates a map and culling changes the slope rather than the direction; that a failure label is not a diagnosis, which follows from no single machine being able to tell a tracker that ran out of correspondences from one that ran out of headroom; and that learned front ends relocate cost from processor to GPU rather than removing it, which Table 6.11 shows holding in native code at a similar ratio.

Specific to this bench. The absolute throughput figures are roughly three times below native, and the absolute memory figures are inflated by unpruned keyframe retention and by loading two model instances in stereo. The 14 GB threshold is a number about this pipeline and this route length, not a specification.

Untested, and therefore unclaimed. Nothing here speaks to direct methods, which fail photometrically rather than through correspondence loss, so DSO and its descendants sit outside every result in Chapters 6 to 8. Nothing here speaks to end-to-end learned systems, which have no swappable seams and whose binding resource is VRAM rather than system RAM. And nothing here speaks to visual-inertial systems, which is the most consequential gap, because it cuts against the thesis's own headline. An inertial unit directly mitigates the low-overlap turn that accounts for most of the failures in this study, so if an IMU carries the pose through the manoeuvres where the deep front ends lose tracking, the completion inversion of Section 6.3 may be substantially smaller and the case for classical front ends correspondingly weaker. The accuracy and cost findings would be unaffected. Testing that is the single most valuable extension to this work.

9.6 A decision guide

Table 9.1 condenses the study into the form a person specifying a system actually needs. It is deliberately expressed in terms of the constraint rather than the method, because that is the direction the causation runs: the budget selects the component, not the other way round.

Table 9.1: What to deploy, given a constraint. Figures are for this pipeline on this dataset; the ratios should transfer further than the absolutes (Section A.9).

Constraint	**Recommendation**	**Why**
Memory is the binding limit	Classical front end (BRISK, AKAZE, ORB2) with a classical closer (DBoW2, OBIndex2)	Highest completion in the study, smallest footprint, and the pairing that beat every assembled alternative
GPU is contested (shared with detection, segmentation, a second camera)	Classical front end	Deep front ends move cost from CPU to GPU; that trade runs backwards when the GPU is the scarce resource (Section 6.4.2)
CPU is contested, GPU idle	A light CNN front end (XFeat, ALIKED) is defensible	Same trade in the favourable direction, but expect lower completion on long drives
Route contains large-heading revisits	A DINOv2 closer (CricaVPR, MegaLoc)	The one condition where deep place recognition earned its cost here (Section 4.6)
Route is long relative to available RAM	Reduce keyframe retention, or disable loop closure and accept drift	Memory demand scales with route length, not model size (Section 4.7)
Choosing a solver	Either; decide on licensing or API	The two are indistinguishable on this workload
Stereo is available but the second camera is contested	Keep it	Neural depth costs a factor of 4–6 in ATE and reconstructs the world 16–27 % too small; V2 recovers scale but runs at 0.86 fps
Specifying a machine	Buy capacity before speed, and speed before cores	Equation 7.1

One line of that table deserves emphasis because the economics have moved underneath it. When memory was cheap, “buy capacity” was the obvious recommendation and the classical preference was a mild convenience. At 2026 prices (Appendix C) capacity is often the single most expensive line item in a small robot, so a pipeline that fits in 10 GB and one that needs 22 GB differ by a component cost that can exceed the rest of the compute. That makes footprint a first-class design metric rather than an implementation detail, and it makes the classical recommendation stronger rather than weaker.

9.7 Threats to validity

Every concern that could undermine the conclusions is gathered here, stated once, rather than repeated at each result.

The platform is built for the baseline, and handicaps the deep front ends specifically. Section 6.1 states the mechanism, and three further ones compound it: the independent vocabulary makes every non-ORB front end run an ORB2 pass in addition to its own network; stereo loads two model instances rather than reusing one, doubling VRAM pressure; and the C++ projection matcher requires binary descriptors, forcing a lossy binarisation of float ones (Appendix A). *What this affects:* every comparison the baseline wins, which is most of them, and every cost ratio in Chapter 6, which should be read as an upper bound on the deep methods' expense. *What it does not affect:* the comparisons *between* swapped components, since all face the same bias; the hardware findings, which are within-configuration; and the results where a swapped component beats the baseline anyway. *Direction:* a native, tuned implementation could close the gap and possibly reverse it. What this thesis can defend is that the deep components do not pay for themselves *in this pipeline*, not that they could not in any pipeline.

The skip list is inherited from the weakest machine. Section 5.1.3 states this one in full: what was dropped, how the surviving set is biased, and what bound the family-level figures can carry. It is the single largest gap in the design and the first thing a replication should close.

One dataset, one environment, one motion profile. Section 4.8 states the four limits. *Direction:* the appearance-invariance axis on which the learned place recognisers claim their largest advantage is almost entirely untested here, which is the strongest reason to read the deep closers' single win as a statement about this dataset rather than about the methods.

Four smaller ones. The pilot condition ran before the skip list settled and cannot be re-run, so every condition-to-condition comparison reads only the 139 cells common to all four (Section 5.6.1). The platform step bundles an SMT confound, bounded by the core-count result: a workload saturating near seven threads cannot have doubled its throughput from six extra cores (Section 7.3). The current machine ran with Precision Boost Overdrive at an 85 °C limit and sat in the low eighties during long batches, so it was shedding some boost and the 2.08× median speedup is a lower bound; boost clock neither creates nor destroys memory, so the capacity finding is untouched. And three repetitions is thin, with several conclusions resting on one or two runs out of twelve, which is why medians carry their spreads, high-variance variants are flagged rather than averaged over, and differences of that size are called "within noise" throughout.

CHAPTER 10

Conclusion

I set out to ask how much machine each SLAM component needs before its accuracy becomes available, and how that answer moves when the machine moves. Four answers.

At a fixed hardware condition (RQ1), depth of architecture bought cost without a consistent return. GPU load rises by an order of magnitude from classical through CNN to transformer front ends while accuracy does not separate at all, and short-window RPE is flat across the three families, so the difference is not in estimating local motion but in surviving the manoeuvres where correspondences run out. On completion the axis inverts rather than vanishing. The ORB2 baseline came out ahead when the four sequences are read together, though never dominant on a single axis and always on its home ground: swapping only the loop closer improved ATE on three of the four sequences, D2-Net halved its error on `test1`, and five configurations completed every repetition against the baseline's 18 of 20. On loop closure a pooled ranking turns out to be the wrong object to compute, since DINOv2-based place recognition wins decisively on the one sequence with large-heading revisits and loses everywhere else. The useful output is a rule for when to reach for a deep closer, not a league table.

Across hardware conditions (RQ2), accuracy transfers and completion does not. Rank correlation between conditions runs from 0.89 to 0.95 for accuracy and 0.53 to 0.81 for completion. A single-machine benchmark can report which method is more accurate; it cannot report which method finishes, and it cannot tell its reader which of the two it just did.

Decomposed (RQ3), the ordering is memory capacity, then platform speed, then core count. Capacity removed every out-of-memory death and lifted completion by nearly fourteen points; platform speed halved timeouts and left every memory death in place; quadrupling threads made the pipeline six per cent slower. What ran out of memory was the map, not the model, so the memory question for a deployment is how long the route is, not how large the network is.

And footprint reduction substitutes for capacity (RQ4) as a diagnostic but not as a deployment. Visual-odometry completion on a small machine predicts full-SLAM completion on a large one far better than full-SLAM completion on the small machine does.

Underneath all four sits the methodological finding, which I consider the most transferable thing here. Of the 84 runs that failed at 14 GB in cells which recovered once memory was supplied, 75 had been recorded as tracking losses: an algorithmic verdict on what the recovery shows to be, at least in part, a budget problem. For every failure the machine reported honestly, roughly nine were filed under the algorithm's name, and nothing in that machine's own output distinguished them. Three repetitions per cell make this strong evidence of memory's contribution rather than proof of exclusive cause.

These answers are bounded by one dataset, one pipeline built around the baseline that wins in it, and a skip list inherited from the weakest machine; Section 9.7 states how far each reaches. What survives all three is the shape of the result. A component recommendation is meaningful only as a (component, budget) pair, and at 2026 memory prices the budget is the expensive half.

APPENDIX A

How pySLAM Works

This appendix describes the pySLAM framework [7]: what it is built from, how an image becomes a pose, and which of its design choices shaped the results in this study.

A.1 Overview and structure

pySLAM implements an ORB-SLAM2-style pipeline in Python, split into the same four stages as its reference design: **tracking**, which runs on every image and estimates the camera pose; **local mapping**, which runs on every new keyframe and refines the local map; **loop closing**, which finds revisited places and corrects drift; and an optional **dense mapping** stage, unused here.

The performance-critical geometry, meaning the projection and epipolar matchers, the optimisers, the bag-of-words indexing and the core map containers, is written in C++ and bound through pybind11. The Python layer keeps the orchestration, the configuration and the deep-learning front ends. Loop detection and global bundle adjustment each run in a separate operating system process, so they are not blocked by the Python global interpreter lock.

The defining property, and the reason this study uses it, is modularity. Feature extractors, loop closers, optimisation back ends and depth models are all selected by configuration behind a common interface, so swapping the front end is a one-line change and every other stage keeps working. Global behaviour lives in a single `Parameters` class holding roughly 200 named constants grouped by stage, a subset of which can be overridden per dataset from a YAML settings file. One of them matters here: `USE_CPP_CORE` selects whether the core classes come from the C++ module or a pure Python fallback. Both expose the same names, so the choice is invisible to the rest of the code. All results in this study use the C++ core.

A.2 Local features: the front end

The front end is the part this study varies, so it is worth describing carefully. Feature extraction is wrapped in three layers. The **feature object**, one class per method, exposes `detectAndCompute` and `compute` on a single image; handcrafted methods delegate to OpenCV and learned ones run a network on the GPU. The **feature manager** owns the detector, the descriptor, the scale-space policy and the keypoint filters, deciding whether a method needs a pyramid built by pySLAM or produces its own multi-scale keypoints. The **feature tracker** pairs a manager with a matcher and exposes the matching interface used by tracking, local mapping and loop closing.

Every descriptor type is registered in a table recording its norm, Hamming for binary and L_2 for float, and a reference maximum descriptor distance. That distance is the gate used by all the projection matchers: a candidate match is rejected if the descriptor distance exceeds it. Each new front end must therefore supply its own, and pySLAM additionally adapts the threshold at run time from the match statistics observed during initialisation.

Ready-made tracker configurations, currently 44, cover handcrafted methods (Shi-Tomasi, FAST, ORB, ORB2, BRISK, KAZE, AKAZE, SIFT, RootSIFT, SURF and various descriptor pairings), learned sparse methods (SuperPoint, XFeat, DISK, ALIKED, D2-Net, R2D2, KeyNet and the descriptor-only HardNet, SOSNet, L2-Net and TFeat paired with an ORB2 detector), learned matchers (LightGlue over several detectors, and the XFeat sparse matcher), and detector-free matchers (LoFTR, EfficientLoFTR, MASt3R). The last group produces matches without repeatable keypoints, which is why they are usable for visual odometry but not inside the full SLAM loop, where a map point needs persistent keypoints to accumulate observations against.

A.3 Frames, keyframes and initialisation

A `Frame` is built from one image, a stereo pair, or an image plus a depth map. Construction runs the front end and stores everything as flat arrays of length N: pixel and undistorted coordinates, normalised coordinates, octaves, sizes, orientations, descriptors, per-keypoint depths and, for stereo, the matched horizontal coordinate in the right image. This flat layout is what makes the C++ matchers efficient. For stereo input, detection runs on both images in parallel threads and stereo matching pairs keypoints along image rows, valid because the images are rectified. Keypoints without a valid depth keep a negative marker and are treated as monocular observations, which matters because the pose optimiser applies different chi-square thresholds to the two.

A `KeyFrame` extends a frame with the map bookkeeping: observations, the weighted covisibility graph, the spanning tree, loop edges and erase flags. A keyframe holds its own images and an independent descriptor set, which makes it the main contributor to memory growth (Section A.9).

Initialisation bootstraps a map from a pair of buffered frames. For monocular input it matches them, estimates relative pose from the essential matrix with RANSAC, triangulates, runs a short bundle adjustment and accepts only if the reprojection error decreased and enough points survived; a minimum baseline-to-depth ratio prevents a stationary start producing a degenerate map, and the median depth is normalised to fix the arbitrary scale. For stereo and RGBD the keypoints are unprojected directly with their measured depth and the second pose comes from EPnP, so the map is created at metric scale with no essential-matrix step.

A.4 Tracking

Tracking runs once per image and is the only stage on the critical path of the frame rate. If the constant-velocity motion model is valid it predicts the current pose, otherwise the previous pose is reused. Map points observed in the previous frame are then projected into the current one and matched within a search radius under the descriptor gate and ratio test; if too few matches survive, tracking falls back to descriptor matching against the reference keyframe. Either path ends in a motion-only bundle adjustment with a robust kernel, which also marks outliers.

The local map is then rebuilt around the current frame from the best covisible keyframes of the reference keyframe, its points are projected and matched against the still-unmatched keypoints, and a second pose optimisation runs over the enlarged set. Tracking is declared successful only if enough inlier map points survive this step.

The keyframe decision follows ORB-SLAM2 and combines several triggers: a maximum frame gap since the last keyframe, a minimum gap plus an idle local mapper, weak tracking relative to the reference keyframe, too few tracked close points alongside many

untracked ones for non-monocular sensors, and optionally weak image coverage or a minimum travelled distance. A keyframe is created only if a trigger fires *and* the frame still tracks enough points but proportionally fewer than its reference. Frame poses are stored relative to their reference keyframe, which is what allows the full trajectory to be recovered after loop closing has corrected the keyframe poses.

Two waits exist inside tracking for practical reasons: it pauses while loop closing applies a correction, and it will optionally wait for the local mapper to become idle. Both trade throughput for map consistency.

A.5 Local mapping

Local mapping consumes keyframes from a queue on its own thread and performs six steps: associate the new keyframe's keypoints with their map points and refresh the covisibility connections; cull recently created map points that were found in too few of the frames from which they should have been visible, or that still have too few observations after two keyframes; triangulate new points against the best neighbouring keyframes along epipolar lines, accepting only after checks on parallax, positive depth, reprojection error and scale consistency; fuse duplicate points between the current keyframe and its neighbours; run local bundle adjustment over the covisible window with the rest of the map fixed; and cull keyframes whose observations are largely redundant, meaning about ninety per cent of their points are seen by at least three others at the same or finer scale.

Fusion and bundle adjustment are skipped when the queue is not empty, so a backlog degrades map quality gracefully rather than blocking the pipeline.

A.6 Loop closing

Detection runs in a separate process. Keyframes arrive through a queue as an image plus, optionally, the front-end descriptors, and the process returns candidate identifiers together with the computed global descriptor. Running detection out of process is what lets a GPU place-recognition network be used without stalling tracking.

The available global descriptors fall into two groups. The first aggregates local descriptors: DBoW2, DBoW3, OBIndex2, iBoW, VLAD and HDC over DELF. These need a vocabulary, and the shipped vocabularies are built on ORB descriptors. The second runs a network directly on the image: SAD, AlexNet, NetVLAD, CosPlace, EigenPlaces, MegaLoc and the three added in this work. These need no vocabulary.

Each aggregating configuration exists in two variants, and the distinction is central to this study. In the default variant the detector reuses the front-end descriptors, which requires them to be compatible with the loaded vocabulary. In the *independent* variant the detector allocates its own ORB2 feature manager and re-extracts descriptors from every keyframe image. The independent variant is what makes a non-ORB front end usable with a bag-of-words closer at all, at the cost of a second extraction pass per keyframe.

A candidate is then filtered three ways before it is trusted. Group consistency requires it to be supported by consistent covisible groups over several consecutive detections, which removes isolated false positives. Geometry verification matches it to the current keyframe at the keypoint level and runs a Sim(3) solver under RANSAC, with scale fixed for stereo and free for monocular, using a successful solution to guide a second denser matching pass. Only a candidate keeping enough inliers throughout is accepted.

Correction then stops the local mapper, propagates the Sim(3) correction to the covisible keyframes, moves their map points through the uncorrected pose and back

through the corrected one, fuses the two sides of the loop, rebuilds the covisibility graph, optimises the essential graph in Sim(3) to distribute the drift over the whole trajectory, and launches global bundle adjustment in yet another process.

When tracking is lost, relocalisation queries the same place-recognition database, estimates a pose with PnP inside RANSAC for each candidate, and searches additional map points by projection at a coarse then a fine radius if too few inliers survive. The motion model is reset rather than updated, because the history before the loss is no longer meaningful.

A.7 Back ends, the C++ core and parallelism

Four optimisation problems appear in the system: motion-only bundle adjustment during tracking, local and large-window bundle adjustment in local mapping, Sim(3) refinement during geometry verification, and essential-graph plus global bundle adjustment during loop correction. Each can be solved with g2o or GTSAM, selected by separate flags. g2o is the default throughout, and the GTSAM front-end paths are marked experimental in the source.

The C++ core mirrors the Python core class by class: the data classes, the projection and epipolar matchers, the rotation histogram, the tracking and local mapping inner loops, both optimiser wrappers and JSON serialisation. Two consequences show up in practice. The matchers are written against the flat keypoint arrays of a frame, so a front end must fill those arrays in the expected layout. And the projection matcher was written for binary descriptors, so a float descriptor has to be binarised before it can pass through that path (Section A.9).

The concurrency model mixes threads and processes deliberately. Threads are used where the work sits inside C++ or a library that releases the interpreter lock: local mapping, large-window bundle adjustment, the loop-closing supervisor, stereo detection and parallel matching. Processes are used where the work is Python or GPU bound: loop detection, global bundle adjustment, and the optional semantic and volumetric stages, none of which this study enables. Communication goes through managed queues carrying keyframe data and images, so the cost of this design grows with image resolution.

A.8 Modules added in this work

Three joint detector-descriptor front ends were added: **LiftFeat**, **RDD** and **DeDoDe v2**. Each consists of a feature class exposing `detectAndCompute` and `compute` with an internal lock and a one-frame cache, registrations in the detector and descriptor enumerations, a branch in the feature manager, an entry in the descriptor property table giving the norm and maximum descriptor distance, and a ready-made tracker configuration. All three produce unoriented keypoints, so the rotation histogram filter is bypassed for them, and all three return L_2 descriptors. The maximum descriptor distances were not inherited from the papers but measured, each set to three times the median absolute deviation of the distances observed over matched keypoints, which is the rule pySLAM applies to its own descriptors.

Three place-recognition models were added: **SALAD**, **CricaVPR** and **SuperVLAD**, each a subclass of the place-recognition base class with an entry in the global descriptor enumeration and a configuration carrying no vocabulary, so they pair with any front end. **EfficientLoFTR** was added as a detector-free matcher, which required the early-exit checks that reject empty descriptor arrays to become conditional on the matcher being detector-free.

Three integration problems had to be solved. The DINOv2 backbone requires image dimensions divisible by fourteen, so images are resized up to the next multiple before the network; the resulting descriptors are image-level and never mapped back to pixels, so this is harmless, whereas EfficientLoFTR has the same constraint at a factor of thirty-two and *does* map coordinates back, so its match coordinates are rescaled by the resize ratio. Model construction had to be deferred past process start-up, with the loop detector built inside the detection process rather than the parent, both to avoid pickling a Torch model across the boundary and to stop the parent waiting on a weight download; the Torch hub cache directory is set explicitly for the same reason. And the C++ projection matcher read the right-image coordinate array unconditionally during point fusion, which is empty for monocular input, so the read went out of bounds; a bounds check now returns the negative marker the surrounding code already reads as "no stereo observation".

A.9 Why the framework shapes the results

The architecture above explains why the absolute numbers in this study undershoot native implementations. Each mechanism below was visible in the results.

Python orchestration and inter-process image passing. These cap throughput. On the pilot machine nothing reached the dataset's 10 fps; on the current machine the classical configurations clear it comfortably, which suggests the ceiling was the processor rather than the language, at least for the CPU-bound half of the field.

Keyframe retention. Keyframes are retained generously, and with the independent DBoW3 vocabulary each costs roughly 8 to 12 MB, because it holds its own images and two descriptor sets. Since the map grows steadily with route length, the memory requirement is a function of how far the vehicle drives rather than of how hard the scene is. This is the mechanism behind the entire capacity finding of Section 7.2.

Stereo model duplication. For thread safety, stereo processing builds a second feature tracker so the left and right images can be extracted in parallel. This loads two copies of the network and doubles VRAM pressure on an 8 GB card. A dual-pass approach through one instance would halve VRAM usage at the cost of latency; the framework does not do this, and the cost falls entirely on the deep front ends.

The independent vocabulary. It re-extracts ORB2 descriptors per keyframe, so every non-ORB front end pays for its own network *plus* an ORB2 pass. This is a fairness compromise rather than a defect, since it is what allows a deep extractor to be paired with a working loop closer at all. It does mean the deep front ends are measured with a handicap a purpose-built deep pipeline would not carry.

Relocalisation model re-instantiation. The relocalisation path re-instantiates the place-recognition model instead of sharing the instance already loaded in the detection process, felt as SALAD reloading mid-run.

The binary descriptor assumption. The C++ projection matcher was written for binary descriptors, forcing a lossy binarisation of DeDoDe's float descriptors. This affects absolute performance, but relative comparisons stay fair because every method passes

through the same pipeline. Native deployments of the recommended configurations should therefore yield better throughput and more memory headroom than reported here.

APPENDIX B

Environment and Known Issues

None of the results in this study were produced by pySLAM's own entry points. They came from a separate harness, `src_pc`, which sits beside the pySLAM checkout, imports it as a library, and owns everything pySLAM leaves to the caller: which configurations exist, how a run is bounded, what it leaves behind, and how a few hundred runs on four machines become one table. This appendix describes that harness and what is known to be wrong with it. The machines themselves are in Appendix C.

B.1 The guest configuration

Every condition ran inside a WSL2 Ubuntu guest pinned by `.wslconfig`: twelve logical processors, 14 GB of memory, 8 GB of swap and the `dropcache` automatic memory-reclaim policy on the pilot condition, the same values for conditions 2 and 3, and thirty-two processors for condition 4. Swap size and the reclaim policy are held constant across all four, so neither can contribute to a difference between conditions.

That reclaim setting matters more than it looks. WSL2's guest releases page cache back to Windows lazily, and early batches showed later runs starting with less effective memory than earlier ones. The three-minute cooldown between runs exists specifically to let reclaim catch up, and it is one of the reasons native Linux would be the better host for this workload.

B.2 Organisation and dataset preparation

The harness is a small Python package with three parts and one rule: no value is written twice. `configs` holds the single settings module together with the generated camera and sequence YAML files, so image width, capture rate, feature count, ORB pyramid settings, depth cutoff, metric windows, timeouts and cooldowns are defined there and nowhere else. `experiments` holds the variant table and the two runners, one executing a single sequence in-process and one executing a batch of them as subprocesses. `utils` holds the path definitions, dataset preparation, the metrics implementation, the writers, and the aggregation pipeline. A separate module defines every directory so nothing hard-codes a path, and both runners locate the pySLAM checkout the same way, as a sibling directory prepended to `sys.path` before any pySLAM import. There is no installation step, which is convenient during development and is also the reason for one of the failure modes in Section B.6.

`prep_project.py` is the run-once setup, and it exists so that no file is ever placed by hand. It reads the raw KITTI-360 download and produces a processed tree in pySLAM's KITTI layout: rectified left and right images, a per-sequence timestamp file, and ground truth written twice, in KITTI format for pySLAM's own reader and in TUM format for the harness's metrics.

The camera file is generated rather than copied. Intrinsics come from the dataset's own calibration, and everything is scaled to the target width of 1408 pixels with the height following so the aspect ratio holds. The baseline is recovered in metres before

scaling and re-multiplied by the scaled focal length, so the physical baseline survives the resize. The file also carries the depth threshold and the `ORBextractor.*` block, which is how the feature count and pyramid settings actually reach pySLAM's extractor; Section B.6 explains why that route matters. Per-sequence statistics and frame counts are written at the same time, and the batch runner reads those counts back later, which is how a hard-killed run can still be reported as a percentage of its sequence.

B.3 The variant table

`variants.py` is the experimental design expressed as a dictionary. Every entry names a pySLAM feature-tracker configuration and a loop-detector configuration by attribute name, plus the phase, a folder-safe short name, the back end, the feature count, the depth cutoff, an optional neural depth model, and a list of sequences to skip. One function is the only place that touches pySLAM's configuration namespaces, so adding a front end or a closer to the study is a one-line addition here, provided the module has been registered inside pySLAM as described in Section A.8.

Which loop detector a front end gets follows one rule: ORB-family descriptors use the shipped vocabulary, everything else uses the independent variant that extracts its own ORB2 descriptors per keyframe, for the fairness reason in Section A.9.

The baseline is exactly what the ORB2, DBoW3 and g2o entries would each produce. Rather than run the identical configuration three times, it runs once per mode and sequence into a shared folder that the three phases point at. A phase batch tops that folder up to the requested repetition count and reuses whatever earlier batches finished, counting only runs that left a dashboard, so a crashed baseline is retried rather than silently reused.

B.4 Running one sequence

`slam_runner.py` executes a single (variant, sequence) pair in-process, and its contact surface with pySLAM is deliberately narrow: build a config from the generated YAML, build the dataset, ground truth and camera from it, construct `Slam` with the variant's two configurations, then call `slam.track` once per frame. Only three of pySLAM's roughly two hundred global parameters are set at run time: the GTSAM flag, the stereo depth cutoff, and the front-end depth flag for the monocular variants. Everything else arrives through the generated YAML or is fixed inside the C++ core, which is a correction rather than a design choice for reasons Section B.6 records.

Two sensor paths exist. The stereo path passes both images and lets pySLAM do the matching. The monocular-plus-depth path passes the left image with a depth map from a pySLAM depth estimator, optionally shadow-filtered, and declares the sensor type as RGBD while leaving the dataset's own type alone. That combination is intentional: the RGBD front end is what consumes a predicted depth map, while the dataset still reports itself as stereo so pySLAM's evaluation does not solve away a scale the depth model has already fixed. Loop closure is switched off by passing no loop configuration at all, which is also what separates the two output trees.

Each run writes a per-frame row to `progress.csv` covering tracking state, keyframe and map-point counts, loss and relocalisation counters, approximate frame rate, and CPU, RAM and GPU load sampled at that frame. The file is line-buffered and flushed after every row, which is the mechanism by which a run killed by the kernel still contributes real measurements. Accuracy is then computed twice by two routes: pySLAM's own

evaluation on the associated final trajectory, and independently by the harness from the final TUM trajectory and ground truth, with a Umeyama alignment plus RPE over a short and a long window. The harness always solves for scale and reports the factor as a diagnostic rather than a correction, since on a stereo run it should come out at essentially one and a value that does not is scale drift worth looking at.

Teardown is ordered and defensive. pySLAM's multiprocessing shutdown can raise or hang, most often as a broken pipe to a worker that already died, so every teardown step is wrapped separately and none may skip the metric writing that follows. A run that finished its real work but died in shutdown must not be indistinguishable from one that timed out.

B.5 Batch execution, limits and aggregation

`batch_runner.py` builds a plan from the requested phases, sequences and repetition count, then executes each entry as a separate subprocess. Isolation is the point: pySLAM spawns processes of its own, and a leaked worker holds memory the next run needs. Each run is launched into its own process group, and however it ends, the whole group is killed in a `finally` block, followed by a sweep of the process table for anything whose command line still names the checkout or the runner.

Every run is bounded twice. The runner checks a twenty-minute wall-clock limit inside the frame loop, with the clock starting at the first frame actually processed so that model loading does not count against it; because the check happens between frames it cannot interrupt a single pathological frame, so the batch runner adds a hard limit two minutes later, interrupting the process group first and killing after a thirty-second grace period. A third timer covers the case the other two handle badly: once a run's dashboard appears on disk the heavy work is done and only shutdown remains, so a hung teardown is killed after thirty seconds rather than consuming the remaining hard timeout. A run is declared lost after ten consecutive frames without tracking. There is no in-loop memory guard; out-of-memory deaths are not predicted but recognised afterwards from the kill signal, and the per-frame log is what keeps those runs countable.

Outcome classification uses two independent records. The dashboard is the truth when it exists; when the process was killed first, the last row of `progress.csv` gives the frame reached and the flag row gives the exit status. Every run appends one row to `run_flags.csv`, and failures are additionally appended to an errors file with the full command, so any of them can be reproduced by copying one line.

Aggregation is a two-stage pipeline and both stages rebuild from scratch every time, because a stale figure produced under an older definition of "success" is indistinguishable from a fresh one and will end up in a thesis. The first stage aggregates one results tree, recovering metrics from runs that never wrote a dashboard and publishing accuracy over completers only. The second runs the first over every tree, discovers the conditions and the ladder between them, and emits one long-form CSV whose rows are labelled as individual runs, per-cell aggregates or per-condition summaries, each run-level row carrying a provenance value that distinguishes runs backed by both a flag row and a directory from those missing one or the other. Orphans are kept so the counts reconcile and excluded from every metric.

Asset building is split along one mechanical line: the offline script reads only that CSV and the ground-truth poses, so it runs in seconds with no GPU and no pySLAM checkout, while the online script is the only one importing pySLAM and exists for the figures that need a model loaded or a frame decoded. The moment one figure in the offline script imported pySLAM, a missing weight file would take down forty figures that

never needed a GPU.

B.6 Known issues and honest mistakes

For the benefit of anyone extending the project, the notable potholes.

Parameters that were never read. An early version of the runner set ORB pyramid and FAST thresholds on pySLAM's global `Parameters` object. Those attribute names do not exist there, so the assignments created new attributes nothing read: silent no-ops for as long as they were in the code. The same was true of the map-memory caps, which live in the C++ core's own header and cannot be reached from Python while that core is in use. Both are gone, and the extractor settings now travel through the generated camera YAML, which pySLAM's extractor actually reads back. The lesson is that assigning to a configuration object in Python proves nothing; the only honest check is whether the value appears in the run's own log.

Capping map memory does not work. A memory-capping option intended to save RAM starved the local map in hard sections and made tracking worse, and it is permanently disabled. In miniature it is the same lesson as the capacity finding: constraining memory does not degrade this system gracefully, it degrades it as tracking loss, wearing the wrong label on the way out.

The teardown killer used to shoot live runs. The teardown-grace timer originally accepted either the dashboard or the final trajectory file as evidence a run had finished. The trajectory writer creates that file, empty, at start-up, so the evidence appeared within seconds of launch and healthy long runs were killed thirty seconds in and recorded as complete. Only the dashboard counts now, because it is written exactly once, at the very end. Any "is it done yet" check has to key on an artefact that cannot exist early.

Extractors that ignore the feature cap. SIFT and LiftFeat ignore the platform's feature-count cap and emit 5,000 to 7,000 keypoints per frame, which is why their long runs ended in out-of-memory kills at 14 GB. This is a property of the models worth knowing rather than a bug in the bench, and it is also what the skip list is for: those variants are excluded from the long sequences rather than allowed to burn a twenty-minute slot to die.

A skip list that skips everything. KAZE has every sequence in its skip list. It is still in the variant table and still appears in the design, but it never executes and contributes no data. A variant skipped everywhere should be read as a documented exclusion rather than as a result.

One flag is wired up wrong. The batch runner's map-saving switch appends an empty argument instead of the flag it means to pass, which the argument parser would reject. It has gone unnoticed because map saving is off by default and the switch is never used; it is recorded here rather than quietly fixed in the text.

Grayscale saves nothing. Loading images as grayscale saves no memory, because OpenCV promotes them back to three channels on read, and the study runs RGB

throughout. The switch remains in the settings because it also selects the directory layout and the two colour flags, and those three must move together.

Two smaller ones. iBoW's detector queue can overflow when the front end outruns the loop-closing process, a real-time mismatch reported here as a finding rather than fixed. And the DINOv2-family closers must be lazy-initialised to avoid `torch.hub` network calls inside the spawned subprocess, with CricaVPR additionally requiring the xFormers build matching the installed PyTorch.

The C++ core must be rebuilt after an update. Because the harness imports pySLAM from a sibling checkout rather than an installed package, pulling a new version leaves the compiled cores stale and the Python and pybind11 layers desynchronised until they are cleanly rebuilt. Nothing detects this; the symptom is an import or signature error at start-up, or worse, a silently different code path.

Two ATE numbers per run. pySLAM's evaluation and the harness's own metrics are computed independently and need not agree exactly, since they associate poses differently and treat the monocular case differently. The harness's number is the one reported. Where the two disagree noticeably, the association window is the first thing to check.

APPENDIX C

The Two Machines

Section 7.1 treats the first step of the ladder as a single factor called “platform”. This appendix opens it up, because the two machines differ in more than one way. It closes with what those differences cost in 2026, which turns out to be part of the argument rather than a footnote to it.

C.1 The two systems

Only the processor and the memory changed between the pilot machine and the current one (Table C.1). The GPU, the storage and the operating system were held identical, which makes them controls rather than variables.

Table C.1: The two machines in full. Everything below the midrule is identical across all four hardware conditions.

	Pilot machine (condition 1)	Current machine (conditions 2–4)
CPU	Intel i7-8700K	AMD Ryzen 9 9950X
Launched	2017	2024
Microarchitecture	Coffee Lake (14 nm)	Zen 5 (4 nm)
Physical cores / threads	6 / 12	16 / 32
Threads given to the guest	12	12 (conditions 2–3), 32 (condition 4)
Base / boost clock	3.7 / 4.7 GHz	4.3 / 5.7 GHz
L3 cache	12 MB	64 MB
Vector width	AVX2 (256-bit)	AVX-512 (512-bit)
Memory kit	G.Skill Ripjaws V 16 GB (2×8 GB) DDR4-3200 CL16	Patriot Viper 32 GB (2×16 GB) DDR5-6000 CL30, auto-tuned by the board to 6200 MT/s
Guest memory budget	14 GB	14 GB (condition 2), 28 GB (conditions 3–4)
GPU	NVIDIA RTX 4060 Ti, 8 GB	
Storage	Samsung 990 PRO, 2 TB NVMe	
Host OS	Windows 10 Pro, Ubuntu guest under WSL2	

Two details of the memory configuration deserve stating plainly. The current kit runs above its rated speed: the board’s automatic tuning lifts the DDR5-6000 EXPO profile to 6200 MT/s, and every current-machine run was executed in that state. Anyone repeating the work at stock 6000 MT/s should expect marginally lower throughput and identical completion outcomes, since the capacity finding shows that bandwidth is not what decides survival. The second detail is that the pilot machine had 16 GB physically installed and gave 14 GB of it to the guest, leaving Windows roughly 2 GB, whereas

the current machine carves the same 14 GB out of 32 GB. The same nominal budget is therefore not the same effective budget.

C.2 The processors

Seven years and four process nodes separate these parts, and the gap shows up in four places that matter here.

Per-core throughput. Clock speed contributes perhaps 20 %; the rest is instructions per cycle. Zen 5 executes substantially more work per cycle than Coffee Lake on the branchy, pointer-chasing code that dominates a SLAM tracking loop: descriptor matching, map-point projection, and the sparse linear algebra inside the bundle adjustment. The measured 2.4× on CPU-bound classical variants is consistent with a combined clock-and-IPC effect rather than either alone.

Cache. The 12 MB to 64 MB L3 jump is, for this workload, probably as important as the core itself. The local map, the covisibility graph and a keyframe's descriptor set are exactly the kind of working set that either fits in last-level cache or goes to DRAM on every access.

Vector width. Hamming-distance matching over binary descriptors and the dense inner loops of float-descriptor matching both vectorise well. AVX-512 doubles the theoretical width, and although OpenCV's dispatch does not always take it, this is another reason the classical binary front ends collected the largest speedup of any family (2.40×).

Power. The speedup is not free, and for a thesis about deployment the watts belong in the table. The i7-8700K is a 95 W part and measures close to it, around 86 W at the package under full load [57]. The 9950X is rated 170 W with a 230 W package ceiling, and measures roughly 200 W at stock in an all-core stress test, or about 235 W with Precision Boost Overdrive enabled [58]. Roughly 2.4× the throughput for roughly 2.3× the power ceiling is close to a wash on work per joule, which is the ordering to carry into an embedded decision: the platform step is a throughput upgrade, not an efficiency one.

What the processor change did *not* do is create memory. That is the central asymmetry of Chapter 7: it is a faster way to reach the same wall.

C.3 DDR4 against DDR5

The memory generation changes with the platform and cannot be separated from it. On paper the two kits are far apart: DDR4-3200 CL16 against DDR5-6200 CL30. Peak bandwidth roughly doubles, and DDR5 splits each module into two independent 32-bit subchannels, which improves access parallelism for the scattered, small reads a sparse SLAM map generates. Absolute latency barely moves, because the higher CAS latency in clock cycles cancels much of the faster clock. For this workload, bandwidth plausibly contributes to the observed speedup alongside IPC and cache, and I cannot apportion the three with the experiments I ran.

What is clear is the negative result, and it is the more useful one. **Memory speed and memory volume are different resources, and only volume rescued runs.** The platform step changed the memory technology, the bandwidth, the timings and the access parallelism all at once, at a fixed 14 GB budget, and left twelve out-of-memory kills in place. The capacity step changed nothing about the memory except how much of it there was, and removed all twelve. A benchmark reporting only "faster memory" as the upgrade would have named the wrong variable.

There is a practical corollary for anyone specifying an edge platform. Fast memory is worth buying for throughput, and it is usually bundled with the faster processor anyway. But if the choice is between a higher-bandwidth kit and a larger one at the same price, this workload wants the larger one, because its failure mode is exhaustion rather than starvation.

C.4 What capacity costs in 2026

Every recommendation in Section 9.6 assumes memory can be bought. Through late 2025 and into 2026 that assumption became expensive. Manufacturing capacity shifted toward high-bandwidth memory for AI accelerators, consumer DDR5 supply tightened, and desktop kit prices rose steeply rather than following the long decline the market had been used to. Figure C.1 plots the price history of the exact kit the current machine runs.

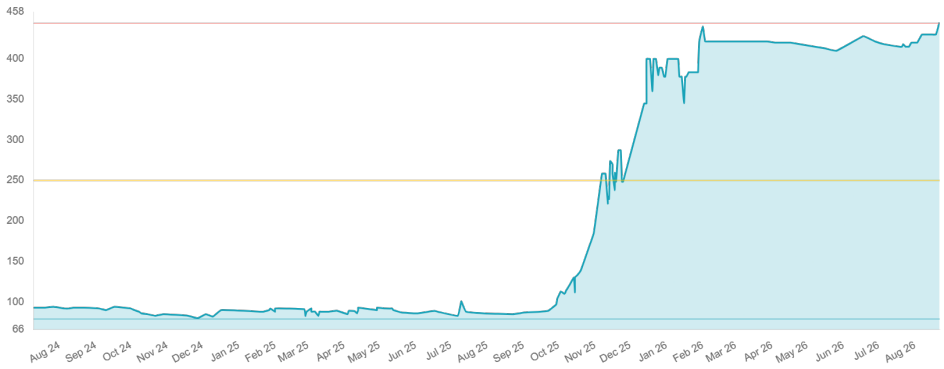


Figure C.1: Retail price history of this study's memory kit, a 32 GB (2×16 GB) Patriot Viper Venom DDR5-6000 CL30 pair. Source: [59].

The concrete number from this project makes the point better than an index does. The kit in the current machine was bought *second hand* for roughly 300 EUR in 2026. For comparison, the pilot machine's entire 16 GB DDR4-3200 kit cost a small fraction of that when new, and the same 32 GB DDR5 kit sold new for around a third of the 2026 figure two years earlier. The storage told a similar story: the Samsung 990 PRO 2 TB used in both machines has roughly doubled in price over the same period, for the same underlying reason.

This changes how the capacity finding should be read. Doubling the guest from 14 GB to 28 GB eliminated every out-of-memory kill and rescued 26 cells worth 66 runs. Stated in 2024 terms that is a cheap fix and the obvious recommendation. Stated in 2026 terms it is the single most expensive line item in the machine, and the recommendation has to be defended rather than assumed. Three consequences follow: footprint becomes a first-class design metric alongside trajectory error; shrinking the pipeline is worth more than it used to be, since it is the substitute whose price is falling rather than rising; and the classical recommendation of Section 9.6 gets stronger, not weaker.

One caveat. Memory pricing is volatile and these figures are a snapshot of mid-2026; the technical findings do not depend on them, and if supply normalises the recommendations soften back toward "buy the headroom". What does not change is the ordering: capacity binds this workload, and knowing how much of it each algorithm consumes is useful at any price.

C.5 The confound I could not remove

The pilot machine exposed 12 logical processors from 6 physical cores with SMT; the current machine exposes 12 from 12 physical cores. SMT threads share execution resources, so twelve SMT threads are worth meaningfully less than twelve real cores, and part of the platform speedup is therefore core-quality rather than core-quantity. I cannot eliminate this, because the pilot machine is no longer available to me.

What I can do is bound it, and Section 7.3 does. Going from 12 real cores to 32 on the same machine changed throughput by $-6\,\%$, because the pipeline peaks at about seven busy threads. A workload that cannot use more than seven threads cannot have doubled its throughput because it was given six more. The platform effect is per-core performance, cache and bandwidth, not parallelism.

APPENDIX D

The Full Run Matrix

Table D.1 is the per-configuration detail behind Figure 7.6, and behind every completion figure quoted in Chapters 6 to 8. It lists all 139 full-SLAM cells that exist under all four hardware conditions, as completed runs over attempted runs.

Read a row left to right and it says what one configuration did with a faster machine, then with twice the memory, then with more cores. Three patterns are worth looking for. A row that is grey on the left and bold on the right is a cell the capacity step rescued, and Section 8.2 is about how those failures were labelled at the time. A row that is grey across all four columns failed for a reason no machine in this study fixes. And a row that is bold across all four is one of the cells behind the invariance result of Section 7.4.

Three conventions govern what is counted. A cell is one (variant, sequence) pair, so a variant that attempted all four sequences contributes four rows and one the skip list restricted to `test0` and `test1` contributes two. Probe cells, the single exploratory runs on the pilot machine where the norm is three or more, are excluded, which is why the pilot column totals 401 runs rather than the 424 it attempted: the 23 missing runs are 23 probe cells at one run each. And the baseline carries five repetitions instead of three at the reference condition, which is why its denominators in that column read 5 rather than 3.

The visual-odometry pass is not tabulated here. Its cell set is not common across the ladder, since the pilot attempted 80 VO runs against 52 on each later condition, so no matched four-column table exists for it; Section 8.1 reports the VO comparison on the cells that do match.

Table D.1: The full run matrix: completed runs over attempted runs, for every cell at every hardware condition.

Configuration	Seq.	Fam.	**Cond. 1** i7 · 12t 14 GB	**Cond. 2** R9 · 12t 14 GB	**Cond. 3** R9 · 12t 28 GB	**Cond. 4** R9 · 32t 28 GB
Baseline						
ORB2 (baseline)	`test0`	Cls	**3/3**	**3/3**	**5/5**	**3/3**
ORB2 (baseline)	`test1`	Cls	**3/3**	0/3	**5/5**	**3/3**
ORB2 (baseline)	`test2`	Cls	2/3	2/3	**5/5**	**3/3**
ORB2 (baseline)	`test3`	Cls	0/3	**3/3**	3/5	**3/3**
Feature extraction ①						
AKAZE	`test0`	Cls	3/4	**3/3**	**3/3**	**3/3**
AKAZE	`test1`	Cls	0/3	1/3	0/3	2/3
AKAZE	`test2`	Cls	0/3	0/3	**3/3**	**3/3**
AKAZE	`test3`	Cls	0/3	0/3	**3/3**	**3/3**
BRISK	`test0`	Cls	**3/3**	**3/3**	**3/3**	**3/3**
BRISK	`test1`	Cls	**3/3**	0/3	**3/3**	**3/3**
BRISK	`test2`	Cls	0/3	0/3	**3/3**	**3/3**
BRISK	`test3`	Cls	0/3	0/3	**3/3**	**3/3**
ORB	`test0`	Cls	**3/3**	**3/3**	**3/3**	**3/3**
ORB	`test1`	Cls	2/3	1/3	1/3	1/3
ORB	`test2`	Cls	0/3	2/3	**3/3**	2/3
ORB	`test3`	Cls	0/3	0/3	0/3	0/3
RootSIFT	`test0`	Cls	**3/3**	**3/3**	**3/3**	**3/3**
RootSIFT	`test1`	Cls	0/3	0/3	**3/3**	**3/3**

continued on next page

Table D.1 continued from previous page

Configuration	Seq.	Fam.	Cond. 1 i7 · 12t 14 GB	Cond. 2 R9 · 12t 14 GB	Cond. 3 R9 · 12t 28 GB	Cond. 4 R9 · 32t 28 GB
SIFT	test0	Cls	**3/3**	**3/3**	**3/3**	**3/3**
SIFT	test1	Cls	0/3	1/3	**3/3**	**3/3**
ALIKED	test0	CNN	**3/3**	**3/3**	**3/3**	**3/3**
ALIKED	test1	CNN	0/3	0/3	0/3	0/3
ALIKED	test2	CNN	0/3	0/3	1/3	0/3
ALIKED	test3	CNN	0/3	0/3	0/3	0/3
D2-Net	test0	CNN	0/3	2/3	**3/3**	**3/3**
D2-Net	test1	CNN	0/3	0/3	**3/3**	**3/3**
DISK	test0	CNN	**3/3**	**3/3**	**3/3**	**3/3**
DISK	test1	CNN	**3/3**	**3/3**	**3/3**	**3/3**
Key.Net	test0	CNN	**3/3**	**3/3**	**3/3**	**3/3**
Key.Net	test1	CNN	0/3	0/3	1/3	0/3
LiftFeat*	test0	CNN	2/3	**3/3**	**3/3**	**3/3**
LiftFeat*	test1	CNN	0/2	0/3	1/3	1/3
R2D2	test0	CNN	0/3	0/3	0/3	0/3
R2D2	test1	CNN	0/3	0/3	0/3	0/3
SuperPoint	test0	CNN	**3/3**	**3/3**	**3/3**	**3/3**
SuperPoint	test1	CNN	0/3	0/3	0/3	0/3
SuperPoint	test2	CNN	0/3	0/3	0/3	0/3
SuperPoint	test3	CNN	0/3	0/3	0/3	0/3
XFeat	test0	CNN	2/3	0/3	**3/3**	**3/3**
XFeat	test1	CNN	0/3	0/3	**3/3**	2/3
XFeat×XFeat	test0	CNN	0/3	0/3	**3/3**	2/3
XFeat×XFeat	test1	CNN	0/2	0/3	1/3	**3/3**
DeDoDe*	test0	Trf	0/3	0/3	**3/3**	**3/3**
DeDoDe*	test1	Trf	0/3	0/3	**3/3**	**3/3**
LightGlue	test0	Trf	**3/3**	**3/3**	**3/3**	**3/3**
LightGlue	test1	Trf	0/3	0/3	0/3	0/3
LightGlue	test2	Trf	0/3	0/3	0/3	0/3
LightGlue	test3	Trf	0/3	0/3	0/3	0/3
RDD*	test0	Trf	**3/3**	**3/3**	**3/3**	**3/3**
RDD*	test1	Trf	0/3	0/3	2/3	1/3
XFeat+LightGlue	test0	Trf	**3/3**	**3/3**	**3/3**	**3/3**
XFeat+LightGlue	test1	Trf	0/3	0/3	**3/3**	2/3
Loop closure ②						
DBoW2	test0	Cls	**3/3**	**3/3**	**3/3**	**3/3**
DBoW2	test1	Cls	1/3	0/3	**3/3**	**3/3**
DBoW2	test2	Cls	2/3	**3/3**	**3/3**	**3/3**
DBoW2	test3	Cls	0/3	**3/3**	**3/3**	**3/3**
iBoW	test0	Cls	2/3	**3/3**	**3/3**	**3/3**
iBoW	test1	Cls	**3/3**	**3/3**	2/3	2/3
OBIndex2	test0	Cls	**3/3**	**3/3**	**3/3**	**3/3**
OBIndex2	test1	Cls	**3/3**	**3/3**	**3/3**	**3/3**
OBIndex2	test2	Cls	**3/3**	**3/3**	**3/3**	**3/3**
OBIndex2	test3	Cls	0/3	**3/3**	**3/3**	1/3
SAD	test0	Cls	**3/3**	**3/3**	**3/3**	**3/3**
SAD	test1	Cls	**3/3**	**3/3**	**3/3**	**3/3**
SAD	test2	Cls	**3/3**	**3/3**	**3/3**	**3/3**
SAD	test3	Cls	0/3	**3/3**	0/3	1/3
VLAD	test0	Cls	**3/3**	**3/3**	**3/3**	**3/3**
VLAD	test1	Cls	**3/3**	**3/3**	**3/3**	0/3
VLAD	test2	Cls	**3/3**	**3/3**	**3/3**	**3/3**
VLAD	test3	Cls	0/3	1/3	2/3	2/3
AlexNet	test0	CNN	**3/3**	**3/3**	**3/3**	**3/3**
CosPlace	test0	CNN	**3/3**	**3/3**	**3/3**	**3/3**
CosPlace	test1	CNN	**3/3**	**3/3**	2/3	**3/3**
EigenPlaces	test0	CNN	**3/3**	**3/3**	**3/3**	**3/3**
EigenPlaces	test1	CNN	2/3	2/3	2/3	**3/3**
NetVLAD	test0	CNN	**3/3**	**3/3**	**3/3**	**3/3**
NetVLAD	test1	CNN	**3/3**	**3/3**	2/3	**3/3**
CricaVPR*	test0	Trf	**3/3**	**3/3**	**3/3**	**3/3**
CricaVPR*	test1	Trf	2/3	**3/3**	**3/3**	2/3
CricaVPR*	test2	Trf	**3/3**	**3/3**	**3/3**	**3/3**
CricaVPR*	test3	Trf	2/3	1/3	1/3	2/3

continued on next page

Table D.1 continued from previous page

Configuration	Seq.	Fam.	Cond. 1 i7 · 12t 14 GB	Cond. 2 R9 · 12t 14 GB	Cond. 3 R9 · 12t 28 GB	Cond. 4 R9 · 32t 28 GB
MegaLoc	test0	Trf	**3/3**	**3/3**	**3/3**	**3/3**
MegaLoc	test1	Trf	2/3	2/3	**3/3**	**3/3**
MegaLoc	test2	Trf	**3/3**	**3/3**	**3/3**	**3/3**
MegaLoc	test3	Trf	0/3	2/3	1/3	2/3
SALAD*	test0	Trf	**3/3**	**3/3**	**3/3**	**3/3**
SALAD*	test1	Trf	1/3	2/3	1/3	**3/3**
SALAD*	test2	Trf	**3/3**	**3/3**	**3/3**	**3/3**
SALAD*	test3	Trf	1/3	1/3	2/3	**3/3**
SuperVLAD*	test0	Trf	1/3	**3/3**	**3/3**	**3/3**
SuperVLAD*	test1	Trf	0/3	**3/3**	0/3	1/3
SuperVLAD*	test2	Trf	0/3	**3/3**	**3/3**	1/3
SuperVLAD*	test3	Trf	0/3	2/3	1/3	2/3
Back end ③						
GTSAM	test0	Cls	**3/3**	**3/3**	**3/3**	**3/3**
GTSAM	test1	Cls	2/3	0/3	**3/3**	**3/3**
GTSAM	test2	Cls	**3/3**	**3/3**	1/3	2/3
GTSAM	test3	Cls	0/3	**3/3**	2/3	**3/3**
Combined ① + ②						
AKAZE + MegaLoc	test0	Cls	**3/3**	**3/3**	**3/3**	**3/3**
AKAZE + MegaLoc	test1	Cls	**3/3**	0/3	2/3	1/3
AKAZE + MegaLoc	test2	Cls	**3/3**	**3/3**	**3/3**	**3/3**
AKAZE + MegaLoc	test3	Cls	0/3	**3/3**	2/3	2/3
AKAZE + NetVLAD	test0	Cls	**3/3**	**3/3**	**3/3**	**3/3**
AKAZE + NetVLAD	test1	Cls	1/3	**3/3**	**3/3**	2/3
AKAZE + NetVLAD	test2	Cls	0/3	2/3	**3/3**	**3/3**
AKAZE + NetVLAD	test3	Cls	**3/3**	1/3	**3/3**	**3/3**
AKAZE + OBIndex2	test0	Cls	**3/3**	**3/3**	**3/3**	**3/3**
AKAZE + OBIndex2	test1	Cls	2/3	**3/3**	1/3	2/3
AKAZE + OBIndex2	test2	Cls	**3/3**	**3/3**	**3/3**	**3/3**
AKAZE + OBIndex2	test3	Cls	**3/3**	**3/3**	**3/3**	**3/3**
AKAZE + SALAD	test0	Cls	**3/3**	**3/3**	**3/3**	**3/3**
AKAZE + SALAD	test1	Cls	2/3	0/3	**3/3**	**3/3**
AKAZE + SALAD	test2	Cls	2/3	**3/3**	**3/3**	**3/3**
AKAZE + SALAD	test3	Cls	2/3	0/3	**3/3**	2/3
LightGlue + MegaLoc	test0	Trf	**2/2**	**3/3**	**3/3**	**3/3**
LightGlue + MegaLoc	test1	Trf	0/2	0/3	0/3	0/3
LightGlue + MegaLoc	test2	Trf	0/2	0/3	1/3	0/3
LightGlue + MegaLoc	test3	Trf	0/2	0/3	0/3	0/3
LightGlue + NetVLAD	test0	Trf	**2/2**	**3/3**	**3/3**	**3/3**
LightGlue + NetVLAD	test1	Trf	0/2	0/3	0/3	0/3
LightGlue + NetVLAD	test2	Trf	0/2	0/3	1/3	0/3
LightGlue + NetVLAD	test3	Trf	0/2	0/3	0/3	0/3
LightGlue + OBIndex2	test0	Trf	**2/2**	**3/3**	**3/3**	**3/3**
LightGlue + OBIndex2	test1	Trf	0/2	1/3	0/3	1/3
LightGlue + OBIndex2	test2	Trf	0/2	0/3	0/3	0/3
LightGlue + OBIndex2	test3	Trf	0/2	0/3	0/3	0/3
LightGlue + SALAD	test0	Trf	0/2	**3/3**	**3/3**	**3/3**
LightGlue + SALAD	test1	Trf	0/2	0/3	1/3	0/3
LightGlue + SALAD	test2	Trf	0/2	0/3	0/3	0/3
LightGlue + SALAD	test3	Trf	0/2	0/3	0/3	0/3
Depth estimation ④						
ORB2 + DA V2	test0	Trf	1/3	**3/3**	**3/3**	**3/3**
ORB2 + DA V3	test0	Trf	**3/3**	**3/3**	**3/3**	**3/3**
ORB2 + DA V3	test1	Trf	**3/3**	**3/3**	**3/3**	**3/3**
ORB2 + DA V3	test2	Trf	0/3	0/3	0/3	0/3
ORB2 + DA V3	test3	Trf	0/3	0/3	**3/3**	0/3
SuperPoint + DA V2	test0	Trf	0/3	2/3	2/3	2/3
SuperPoint + DA V3	test0	Trf	2/3	2/3	1/3	1/3
SuperPoint + DA V3	test1	Trf	0/3	0/3	0/3	0/3
SuperPoint + DA V3	test2	Trf	0/4	0/3	0/3	0/3
SuperPoint + DA V3	test3	Trf	0/3	0/3	0/3	0/3
All 139 cells			**200/401**	**237/417**	**301/425**	**290/417**

Bibliography

[1] D. Sharafutdinov et al., "Comparison of modern open-source visual SLAM approaches," *Journal of Intelligent & Robotic Systems*, vol. 107, p. 43, 2023.

[2] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: A versatile and accurate monocular SLAM system," *IEEE Transactions on Robotics*, vol. 31 no. 5, pp. 1147–1163, 2015.

[3] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras," *IEEE Transactions on Robotics*, vol. 33 no. 5, pp. 1255–1262, 2017.

[4] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM," *IEEE Transactions on Robotics*, vol. 37 no. 6, pp. 1874–1890, 2021.

[5] Z. Teed and J. Deng, "DROID-SLAM: Deep visual SLAM for monocular, stereo, and RGB-D cameras," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 16 558–16 569.

[6] F. Tosi et al., "How NeRFs and 3D Gaussian splatting are reshaping SLAM: A survey," *arXiv preprint arXiv:2402.13255*, 2024.

[7] L. Freda, "pySLAM: An open-source, modular, and extensible framework for SLAM," *arXiv preprint arXiv:2502.11955*, 2025.

[8] Y. Liao, J. Xie, and A. Geiger, "KITTI-360: A novel dataset and benchmarks for urban scene understanding in 2D and 3D," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45 no. 3, pp. 3292–3310, 2022.

[9] R. Smith, M. Self, and P. Cheeseman, "Estimating uncertain spatial relationships in robotics," in *Autonomous Robot Vehicles*, I. J. Cox and G. T. Wilfong, Eds., Springer, 1990, pp. 167–193.

[10] A. J. Davison, I. D. Reid, N. D. Molton, and O. Stasse, "MonoSLAM: Real-time single camera SLAM," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 29 no. 6, pp. 1052–1067, 2007.

[11] M. Montemerlo, S. Thrun, D. Koller, and B. Wegbreit, "FastSLAM: A factored solution to the simultaneous localization and mapping problem," in *AAAI National Conference on Artificial Intelligence*, 2002, pp. 593–598.

[12] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Visual SLAM: Why filter?" *Image and Vision Computing*, vol. 30 no. 2, pp. 65–77, 2012.

[13] G. Klein and D. Murray, "Parallel tracking and mapping for small AR workspaces," in *IEEE and ACM International Symposium on Mixed and Augmented Reality (ISMAR)*, 2007, pp. 225–234.

[14] J. Engel, T. Schöps, and D. Cremers, "LSD-SLAM: Large-scale direct monocular SLAM," in *European Conference on Computer Vision (ECCV)*, 2014, pp. 834–849.

[15] J. Engel, V. Koltun, and D. Cremers, "Direct sparse odometry," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40 no. 3, pp. 611–625, 2018.

[16] C. Forster, M. Pizzoli, and D. Scaramuzza, "SVO: Fast semi-direct monocular visual odometry," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 15–22.

[17] D. Gálvez-López and J. D. Tardós, "Bags of binary words for fast place recognition in image sequences," *IEEE Transactions on Robotics*, vol. 28 no. 5, pp. 1188–1197, 2012.

[18] R. Kümmerle, G. Grisetti, H. Strasdat, K. Konolige, and W. Burgard, "G2o: A general framework for graph optimization," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2011, pp. 3607–3613.

[19] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in *IEEE International Conference on Computer Vision (ICCV)*, 2011, pp. 2564–2571.

[20] Y. Liu et al., "LiftFeat: 3D geometry-aware local feature matching," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.

[21] G. Chen, T. Fu, H. Chen, W. Teng, H. Xiao, and Y. Zhao, "RDD: Robust feature detector and descriptor using deformable transformer," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[22] J. Edstedt, G. Bökman, M. Wadenbäck, and M. Felsberg, "DeDoDe: Detect, don't describe, describe, don't detect for local feature matching," in *International Conference on 3D Vision (3DV)*, 2024.

[23] S. Izquierdo and J. Civera, "Optimal transport aggregation for visual place recognition," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 17 658–17 668.

[24] F. Lu et al., "CricaVPR: Cross-image correlation-aware representation learning for visual place recognition," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[25] F. Lu et al., "SuperVLAD: Compact and robust image descriptors for visual place recognition," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[26] D. DeTone, T. Malisiewicz, and A. Rabinovich, "SuperPoint: Self-supervised interest point detection and description," in *IEEE/CVF CVPR Workshops*, 2018, pp. 224–236.

[27] G. Potje, F. Cadar, A. Araujo, R. Martins, and E. R. Nascimento, "XFeat: Accelerated features for lightweight image matching," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 2682–2691.

[28] X. Zhao, X. Wu, W. Chen, P. C. Y. Chen, Q. Xu, and Z. Li, "ALIKED: A lighter keypoint and descriptor extraction network via deformable transformation," *IEEE Transactions on Instrumentation and Measurement*, vol. 72, 2023.

[29] M. Tyszkiewicz, P. Fua, and E. Trulls, "DISK: Learning local features with policy gradient," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 14 254–14 265.

[30] J. Revaud, C. De Souza, M. Humenberger, and P. Weinzaepfel, "R2D2: Reliable and repeatable detector and descriptor," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[31] M. Dusmanu et al., "D2-Net: A trainable CNN for joint description and detection of local features," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 8092–8101.

[32] A. Barroso-Laguna, E. Riba, D. Ponsa, and K. Mikolajczyk, "Key.Net: Keypoint detection by handcrafted and learned CNN filters," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 5836–5844.

[33] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local feature matching at light speed," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 17 627–17 638.

[34] S. Leutenegger, M. Chli, and R. Y. Siegwart, "BRISK: Binary robust invariant scalable keypoints," in *IEEE International Conference on Computer Vision (ICCV)*, 2011, pp. 2548–2555.

[35] P. F. Alcantarilla, J. Nuevo, and A. Bartoli, "Fast explicit diffusion for accelerated features in nonlinear scale spaces," in *British Machine Vision Conference (BMVC)*, 2013.

[36] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60 no. 2, pp. 91–110, 2004.

[37] R. Arandjelović and A. Zisserman, "Three things everyone should know to improve object retrieval," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012, pp. 2911–2918.

[38] P. F. Alcantarilla, A. Bartoli, and A. J. Davison, "KAZE features," in *European Conference on Computer Vision (ECCV)*, 2012, pp. 214–227.

[39] E. Garcia-Fidalgo and A. Ortiz, "iBoW-LCD: An appearance-based loop-closure detection approach using incremental bags of binary words," *IEEE Robotics and Automation Letters*, vol. 3 no. 4, pp. 3051–3057, 2018.

[40] H. Jégou, M. Douze, C. Schmid, and P. Pérez, "Aggregating local descriptors into a compact image representation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2010, pp. 3304–3311.

[41] R. Arandjelović, P. Gronat, A. Torii, T. Pajdla, and J. Sivic, "NetVLAD: CNN architecture for weakly supervised place recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 5297–5307.

[42] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, 2012, pp. 1097–1105.

[43] G. Berton, C. Masone, and B. Caputo, "Rethinking visual geo-localization for large-scale applications," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 4878–4888.

[44] G. Berton, G. Trivigno, B. Caputo, and C. Masone, "EigenPlaces: Training viewpoint robust models for visual place recognition," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 11 080–11 090.

[45] M. Oquab et al., "DINOv2: Learning robust visual features without supervision," *Transactions on Machine Learning Research (TMLR)*, 2024.

[46] G. Berton and C. Masone, "MegaLoc: One retrieval to place them all," *arXiv preprint arXiv:2502.17237*, 2025.

[47] F. Dellaert and M. Kaess, "Factor graphs for robot perception," *Foundations and Trends in Robotics*, vol. 6 no. 1-2, pp. 1–139, 2017.

[48] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, "Depth anything V2," *arXiv preprint arXiv:2406.09414*, 2024.

[49] Depth Anything Team, *Depth anything V3*, Model release used via the pySLAM depth-estimation interface, 2025.

[50] S. Umeyama, "Least-squares estimation of transformation parameters between two point patterns," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13 no. 4, pp. 376–380, 1991.

[51] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012, pp. 573–580.

[52] Z. Zhang, J. Li, X. Li, and Y. Lin, "Light-SLAM: A robust deep-learning visual SLAM system based on LightGlue under challenging lighting conditions," *IEEE Transactions on Intelligent Transportation Systems*, 2025.

[53] K. Xu, Y. Hao, S. Yuan, C. Wang, and L. Xie, "AirSLAM: An efficient and illumination-robust point-line visual SLAM system," *IEEE Transactions on Robotics*, vol. 41, pp. 1673–1692, 2025.

[54] L. Lipson, Z. Teed, and J. Deng, "Deep patch visual SLAM," in *European Conference on Computer Vision (ECCV)*, 2024.

[55] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.

[56] M. D. Hill and M. R. Marty, "Amdahl's law in the multicore era," *Computer*, vol. 41 no. 7, pp. 33–38, 2008.

[57] I. Cutress. "The Coffee Lake review: Core i7-8700K and Core i5-8400 — power consumption," Accessed: Aug. 24, 2026. [Online]. Available: https://at-web1.www.anandtech.com/show/11859/the-anandtech-coffee-lake-review-8700k-and-8400-initial-numbers/5

[58] P. Alcorn. "AMD Ryzen 9 9950X review: Zen 5 at full power — power consumption, overclocking, thermals," Accessed: Aug. 24, 2026. [Online]. Available: https://www.tomshardware.com/pc-components/cpus/amd-ryzen-9-9950x-cpu-review/4

[59] PriceHistory.app. "Price history for patriot memory viper venom DDR5 RAM 32 gb." Consumer DDR5 memory pricing trend, 2024–2026, Accessed: Aug. 24, 2026. [Online]. Available: https://pricehistory.app/p/patriot-memory-viper-venom-ddr5-ram-32gb-j7JIksmb